



EMAC FW Driver User Guide

Product Version 1.6.0

September 2017

© 1996-2017 Cadence Design Systems, Inc. All rights reserved.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 800.862.4522. All other trademarks are the property of their respective holders.

Restricted Permission: This document is protected by copyright law and international treaties and contains trade secrets and proprietary information owned by Cadence. Unauthorized reproduction or distribution of this document, or any portion of it, may result in civil and criminal penalties. Except as specified in this permission statement, this document may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This document contains the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only in accordance with, a written agreement between Cadence and its customer.

Unless otherwise agreed to by Cadence in writing, this statement grants Cadence customers permission to print one (1) hard copy of this document subject to the following conditions:

1. This document may not be modified in any way.
2. Any authorized copy of this document or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement.
3. The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration.

Disclaimer: INFORMATION IN THIS DOCUMENT IS SUBJECT TO CHANGE WITHOUT NOTICE AND DOES NOT REPRESENT A COMMITMENT ON THE PART OF CADENCE. EXCEPT AS MAY BE EXPLICITLY SET FORTH IN A WRITTEN AGREEMENT BETWEEN CADENCE AND ITS CUSTOMER, CADENCE DOES NOT MAKE, AND EXPRESSLY DISCLAIMS, ANY REPRESENTATIONS OR WARRANTIES AS TO THE COMPLETENESS, ACCURACY OR USEFULNESS OF THE INFORMATION CONTAINED IN THIS DOCUMENT. CADENCE DOES NOT WARRANT THAT USE OF SUCH INFORMATION WILL NOT INFRINGE ANY THIRD PARTY RIGHTS, AND CADENCE DISCLAIMS ALL IMPLIED WARRANTIES, INCLUDING MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. CADENCE DOES NOT ASSUME ANY LIABILITY FOR DAMAGES OR COSTS OF ANY KIND THAT MAY RESULT FROM USE OF SUCH INFORMATION. CADENCE CUSTOMER HAS COMPLETE CONTROL AND FINAL DECISION-MAKING AUTHORITY OVER ALL ASPECTS OF THE DEVELOPMENT, MANUFACTURE, SALE AND USE OF CUSTOMER'S PRODUCT, INCLUDING, BUT NOT LIMITED TO, ALL DECISIONS WITH REGARD TO DESIGN, PRODUCTION, TESTING, ASSEMBLY, QUALIFICATION, CERTIFICATION, INTEGRATION OF CADENCE PRODUCTS, INSTRUCTIONS FOR USE, LABELING AND DISTRIBUTION, AND CADENCE EXPRESSLY DISAVOWS ANY RESPONSIBILITY WITH REGARD TO ANY SUCH DECISIONS REGARDING CUSTOMER'S PRODUCT.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227- 14 and DFAR252.227-7013 et seq. or its successor.

EMAC FW Driver User Guide: User Guide

Cadence Design Systems

Firmware Identifier:edd.soma-2017-09-29-ee5e2d658805800cb47531e2c1448a90

Hardware Identifier:015fd689f2e4ddb88dc401f8c26532e

Table of Contents

1. Acronyms	1
2. Document Purpose	2
3. Description	3
3.1. Core Driver Features	3
3.2. Naming conventions	4
3.3. System Usage	4
4. Core Driver Usage Model	5
4.1. Description	5
4.2. Probing the Hardware	5
4.3. Initialisation	6
4.4. Event Handling	7
4.5. Transmit Operation	8
4.5.1. Large Segment Offload	10
4.6. Receive Operation	11
4.6.1. Detecting discarded fragments	11
4.6.2. Receive Segment Coalescing	14
4.7. Interspersing Express Traffic (802.3br)	16
4.8. ASF - Active Safety Feature	17
4.9. Quitting Driver	17
4.10. PCS Auto-negotiation	18
4.11. Core Driver Interrupt/Event Types	18
5. Configuration and Hardware Operation Information	20
5.1. Introduction	20
6. Dynamic Data Structures	24
6.1. Introduction	24
6.2. CEDI_Config	24
6.3. CEDI_SysReq	26
6.4. CEDI_TxStatus	26
6.5. CEDI_RxStatus	27
6.6. CEDI_BuffAddr	27
6.7. CEDI_TimeStampData	28
6.8. CEDI_TransmitLaunchTime	28
6.9. CEDI_TxDescData	28
6.10. CEDI_TxDescStat	29
6.11. CEDI_RxDescData	29
6.12. CEDI_RxDescStat	29
6.13. CEDI_qTxBufParams	30
6.14. CEDI_FrameSize	31
6.15. CEDI_MacAddress	31
6.16. CEDI_Statistics	31
6.17. CEDI_NumScreeners	33
6.18. CEDI_T1Screen	34
6.19. CEDI_T2Screen	34
6.20. CEDI_T2Compare	35
6.21. CEDI_1588TimerVal	35
6.22. CEDI_TsuTimerVal	35
6.23. CEDI_TimerIncrement	36
6.24. CEDI_AnAdvPage	36
6.25. CEDI_defLpAbility	36

6.26. CEDI_sgmLpAbility	37
6.27. CEDI_LpAbility	37
6.28. CEDI_LpAbilityPage	38
6.29. CEDI_AnNextPage	38
6.30. CEDI_LpNextPage	38
6.31. CEDI_LpPage	39
6.32. CEDI_LpPageRx	39
6.33. CEDI_NetAnStatus	39
6.34. CEDI_WakeOnLanReg	40
6.35. CEDI_LpiStats	40
6.36. CEDI_EnstTimeConfig	40
6.37. CEDI_DesignCfg	41
6.38. CEDI_Callbacks	44
6.39. CEDI_QueueSegAlloc	45
6.40. CEDI_FrameEliminationConfig	45
6.41. CEDI_FrameEliminationTagConfig	46
6.42. CEDI_FrameEliminationStats	46
6.43. CEDI_PreemptionConfig	46
6.44. CEDI_MmslStats	47
6.45. CEDI_MmslStatus	47
6.46. CEDI_LockupConfig	47
6.47. CEDI_RxQFlushConfig	48
6.48. CEDI_Type2ScreenerRateLimit	49
6.49. CEDI_AxiQosConfig	49
6.50. CEDI_AsfInfo	49
6.51. CEDI_BusWidth	49
6.52. CEDI_DmaDatBLen	50
6.53. CEDI_MdcClkDiv	50
6.54. CEDI_MdioDevType	50
6.55. CEDI_Interface	51
6.56. CEDI_TxTsMode	51
6.57. CEDI_RxTsMode	51
6.58. CEDI_LinkFaultIndication	52
6.59. CEDI_RedundancyTagSize	52
6.60. CEDI_IfSpeed	52
6.61. CEDI_TxFreeDStat	53
6.62. CEDI_RxRdStat	53
6.63. CEDI_T2Offset	53
6.64. CEDI_SegmentsPerQueue	54
6.65. CEDI_TxSchedType	55
6.66. CEDI_PauseCap	55
6.67. CEDI_LoopbackMode	55
6.68. CEDI_RemoteFault	56
6.69. CEDI_MacType	56
6.70. CEDI_SeqNumIdentification	56
6.71. CEDI_FragSize	57
6.72. CEDI_RespondStatus	57
6.73. CEDI_VerifyStatus	58
6.74. CEDI_RxQFlushMode	58
6.75. CEDI_CbPhyManComplete	58
6.76. CEDI_CbTxEvent	59

6.77. CEDI_CbTxError	59
6.78. CEDI_CbRxFrame	59
6.79. CEDI_CbRxError	60
6.80. CEDI_CbHrespError	60
6.81. CEDI_CbLpPageRx	60
6.82. CEDI_CbAnComplete	61
6.83. CEDI_CbLinkChange	61
6.84. CEDI_CbLockupEvent	61
6.85. CEDI_CbTsuEvent	62
6.86. CEDI_CbPauseEvent	62
6.87. CEDI_CbPtpPriFrameTx	62
6.88. CEDI_CbPtpPeerFrameTx	63
6.89. CEDI_CbPtpPriFrameRx	63
6.90. CEDI_CbPtpPeerFrameRx	63
6.91. CEDI_CbLpiStatus	64
6.92. CEDI_CbWolEvent	64
6.93. CEDI_CbExtInpIntr	64
6.94. CEDI_CbMmslEvent	65
7. Driver API Object	66
7.1. Introduction	66
7.2. CEDI_GetInstance	66
7.3. probe	66
7.4. init	67
7.5. destroy	67
7.6. start	68
7.7. stop	68
7.8. isr	69
7.9. setEventEnable	69
7.10. getEventEnable	70
7.11. setIntrptModerate	71
7.12. getIntrptModerate	71
7.13. setIntrptModerateThreshold	72
7.14. getIntrptModerateThreshold	73
7.15. setIfSpeed	73
7.16. getIfSpeed	74
7.17. setJumboFramesRx	75
7.18. getJumboFramesRx	75
7.19. setJumboFrameRxMaxLen	76
7.20. getJumboFrameRxMaxLen	76
7.21. setUniDirEnable	77
7.22. getUniDirEnable	77
7.23. setTxChecksumOffload	78
7.24. getTxChecksumOffload	78
7.25. setRxBufOffset	79
7.26. getRxBufOffset	79
7.27. set1536ByteFramesRx	80
7.28. get1536ByteFramesRx	80
7.29. setRxChecksumOffload	81
7.30. getRxChecksumOffload	81
7.31. setFcsRemove	82
7.32. getFcsRemove	82

7.33. setTxPartialStFwd	83
7.34. getTxPartialStFwd	84
7.35. setRxPartialStFwd	84
7.36. getRxPartialStFwd	85
7.37. setRxDmaDataAddrMask	86
7.38. getRxDmaDataAddrMask	86
7.39. setRxBadPreamble	87
7.40. getRxBadPreamble	87
7.41. setFullDuplex	88
7.42. getFullDuplex	89
7.43. setIgnoreFcsRx	89
7.44. getIgnoreFcsRx	90
7.45. setRxHalfDuplexInTx	90
7.46. getRxHalfDuplexInTx	91
7.47. getIfCapabilities	91
7.48. setLoopback	92
7.49. getLoopback	92
7.50. calcMaxTxFrameSize	93
7.51. queueTxBuf	93
7.52. qTxBuf	94
7.53. deQTxBuf	95
7.54. txDescFree	96
7.55. freeTxDesc	96
7.56. getTxDescStat	97
7.57. getTxDescSize	98
7.58. resetTxQ	98
7.59. startTx	99
7.60. stopTx	99
7.61. abortTx	99
7.62. transmitting	100
7.63. enableTx	100
7.64. getTxEnabled	101
7.65. getTxStatus	101
7.66. clearTxStatus	102
7.67. enableCbs	102
7.68. disableCbs	103
7.69. getCbsQSetting	103
7.70. setIpgStretch	104
7.71. getIpgStretch	105
7.72. calcMaxRxFrameSize	106
7.73. addRxBuf	106
7.74. numRxBufs	107
7.75. numRxUsed	108
7.76. readRxBuf	108
7.77. getRxDescStat	109
7.78. getRxDescSize	110
7.79. rxEnabled	110
7.80. enableRx	111
7.81. disableRx	111
7.82. removeRxBuf	111
7.83. resetRxQ	112

7.84. getRxStatus	113
7.85. clearRxStatus	113
7.86. setHdrDataSplit	114
7.87. getHdrDataSplit	114
7.88. setRscEnable	115
7.89. getRscEnable	116
7.90. setRscClearMask	116
7.91. setSpecificAddr	117
7.92. getSpecificAddr	118
7.93. setSpecificAddr1Mask	119
7.94. getSpecificAddr1Mask	119
7.95. disableSpecAddr	120
7.96. setTypeIdMatch	120
7.97. getTypeIdMatch	121
7.98. setUnicastEnable	122
7.99. getUnicastEnable	122
7.100. setMulticastEnable	123
7.101. getMulticastEnable	123
7.102. setNoBroadcast	124
7.103. getNoBroadcast	124
7.104. setVlanOnly	125
7.105. getVlanOnly	125
7.106. setStackedVlanReg	126
7.107. getStackedVlanReg	126
7.108. setCopyAllFrames	127
7.109. getCopyAllFrames	127
7.110. setHashAddr	128
7.111. getHashAddr	129
7.112. setLenErrDiscard	129
7.113. getLenErrDiscard	130
7.114. getNumScreenRegs	130
7.115. setType1ScreenReg	131
7.116. getType1ScreenReg	131
7.117. setType2ScreenReg	132
7.118. getType2ScreenReg	133
7.119. setType2EthertypeReg	133
7.120. getType2EthertypeReg	134
7.121. setType2CompareReg	135
7.122. getType2CompareReg	135
7.123. setPauseEnable	136
7.124. getPauseEnable	136
7.125. txPauseFrame	137
7.126. txZeroQPause	137
7.127. getRxPauseQuantum	138
7.128. setTxPauseQuantum	138
7.129. getTxPauseQuantum	139
7.130. setCopyPauseDisable	139
7.131. getCopyPauseDisable	140
7.132. setPfcPriorityBasedPauseRx	140
7.133. getPfcPriorityBasedPauseRx	141
7.134. txPfcPriorityBasedPause	141

7.135. setTxPfcPauseFrameFields	142
7.136. getTxPfcPauseFrameFields	143
7.137. setEnableMultiPfcPauseQuantum	143
7.138. getEnableMultiPfcPauseQuantum	144
7.139. setUnicastPtpDetect	144
7.140. getUnicastPtpDetect	145
7.141. setPtpRxUnicastIpAddr	145
7.142. getPtpRxUnicastIpAddr	146
7.143. setPtpTxUnicastIpAddr	147
7.144. getPtpTxUnicastIpAddr	147
7.145. set1588Timer	148
7.146. get1588Timer	148
7.147. adjust1588Timer	149
7.148. set1588TimerInc	150
7.149. get1588TimerInc	150
7.150. setTsuTimerCompVal	151
7.151. getTsuTimerCompVal	151
7.152. getPtpFrameTxTime	152
7.153. getPtpFrameRxTime	153
7.154. getPtpPeerFrameTxTime	153
7.155. getPtpPeerFrameRxTime	154
7.156. get1588SyncStrobeTime	154
7.157. setExtTsuPortEnable	155
7.158. getExtTsuPortEnable	155
7.159. set1588OneStepTxSyncEnable	156
7.160. get1588OneStepTxSyncEnable	157
7.161. setDescTimeStampMode	157
7.162. getDescTimeStampMode	158
7.163. setStoreRxTimeStamp	158
7.164. getStoreRxTimeStamp	159
7.165. resetPcs	160
7.166. getPcsReady	160
7.167. startAutoNegotiation	161
7.168. setAutoNegEnable	161
7.169. getAutoNegEnable	162
7.170. getLinkStatus	163
7.171. getAnRemoteFault	163
7.172. getAnComplete	164
7.173. setAnAdvPage	164
7.174. getAnAdvPage	165
7.175. getLpAbilityPage	166
7.176. getPageRx	166
7.177. setNextPageTx	167
7.178. getNextPageTx	167
7.179. getLpNextPage	168
7.180. getPhyId	168
7.181. setMdioEnable	169
7.182. getMdioEnable	170
7.183. phyStartMdioWrite	170
7.184. phyStartMdioRead	171
7.185. getMdioReadData	172

7.186. getMdioIdle	172
7.187. readStats	173
7.188. clearStats	173
7.189. takeSnapshot	174
7.190. setReadSnapshot	174
7.191. getReadSnapshot	175
7.192. setWakeOnLanReg	175
7.193. getWakeOnLanReg	176
7.194. setLpiTxEnable	176
7.195. getLpiTxEnable	177
7.196. getLpiStats	178
7.197. getDesignConfig	178
7.198. setWriteStatsEnable	179
7.199. getWriteStatsEnable	179
7.200. incStatsRegs	180
7.201. setRxBackPressure	180
7.202. getRxBackPressure	181
7.203. setCollisionTest	181
7.204. getCollisionTest	182
7.205. setRetryTest	182
7.206. getRetryTest	183
7.207. writeUserOutputs	183
7.208. readUserOutputs	184
7.209. setUserOutPin	185
7.210. readUserInputs	185
7.211. getUserInPin	186
7.212. getMdioInState	187
7.213. readReg	187
7.214. writeReg	188
7.215. setTxQueueNum	188
7.216. setRxQueueNum	189
7.217. setTxQueueSegAlloc	190
7.218. setTxQueueScheduler	190
7.219. getTxQueueScheduler	191
7.220. setDwrrWeighting	191
7.221. setEtsBandAlloc	192
7.222. getDwrrWeighting	193
7.223. getEtsBandAlloc	193
7.224. setEnstTimeConfig	194
7.225. getEnstTimeConfig	195
7.226. setEnstEnable	195
7.227. getEnstEnable	196
7.228. getEnstSupported	196
7.229. setReportingBadFCS	197
7.230. getReportingBadFCS	198
7.231. setPtpSingleStep	198
7.232. getPtpSingleStep	199
7.233. setMiiOnRgmii	199
7.234. getMiiOnRgmii	200
7.235. setCbsIdleSlope	200
7.236. getCbsIdleSlope	201

7.237. getLinkFaultIndication	202
7.238. setFrameEliminationTagSize	202
7.239. getFrameEliminationTagSize	203
7.240. setFrameEliminationEnable	203
7.241. getFrameEliminationEnable	204
7.242. setFrameEliminationSeqRecRstTmrEnable	205
7.243. getFrameEliminationSeqRecRstTmrEnable	205
7.244. setFrameEliminationConfig	206
7.245. getFrameEliminationConfig	207
7.246. setFrameEliminationTagConfig	207
7.247. getFrameEliminationTagConfig	208
7.248. setFrameEliminationTimeoutConfig	208
7.249. getFrameEliminationStats	209
7.250. getEmac	210
7.251. getPmac	210
7.252. getMacType	211
7.253. setPreemptionConfig	211
7.254. getPreemptionConfig	212
7.255. preemptionRestartVerification	212
7.256. setMmslEventEnable	213
7.257. getMmslEventEnable	213
7.258. readMmslStats	214
7.259. readMmslStatus	215
7.260. setLockupConfig	215
7.261. getLockupConfig	216
7.262. setType1ScreenRegDropEnable	216
7.263. getType1ScreenRegDropEnable	217
7.264. setType2ScreenRegDropEnable	218
7.265. getType2ScreenRegDropEnable	218
7.266. setRxQFlushConfig	219
7.267. getRxQFlushConfig	220
7.268. getRxDmaFlushedPacketsCount	220
7.269. setType2ScreenerRateLimit	221
7.270. getType2ScreenerRateLimit	222
7.271. getRxType2RateLimitTriggered	222
7.272. setRxWatermark	223
7.273. getRxWatermark	224
7.274. setAxiQosConfig	224
7.275. getAxiQosConfig	225
7.276. getAsfInfo	226

List of Figures

4.1. Callback example showing MDIO read data operation	8
4.2. Tx descriptor list example	9
4.3. Example of Rx descriptor list operation:	12
4.4. Rx example (continued) showing discarded frame fragments:	13

List of Tables

4.1. Callback Functions and Events	18
5.1. Configuration and Hardware Operation Information	20

Chapter 1. Acronyms

API	Application Programming Interface.
Core Driver	Cadence Firmware component that provides IP programming abstraction.
CPS	Cadence Platform Services. A set of basic platform specific access functions acting as hardware abstraction layer for the core driver to operate. This is described further in the porting guide, and there are examples in the reference code for ARM bare metal (<code>reference_code/tests/cps_emac_bm.c</code>) and for linux (<code>reference_code/lrx_mod/src/cps_v2_linux.c</code>).
DMA	Direct Memory Access
EDD	Ethernet DMA-MAC Driver
EMAC	Ethernet MAC hardware IP
LSO	Large Segment Offload
MAC	Media Access Control
RSC	Receive Segment Coalescing
TCP	Transmission Control Protocol
TSO	TCP Segment Offload
UDP	User Datagram Protocol
UFO	UDP Fragment Offload

Chapter 2. Document Purpose

This document is intended to serve as an overview and a reference for customers looking to use the Cadence EMAC Firmware Core Driver for the Cadence EMAC IP Core.

Chapter 3. Description

The FW provided by Cadence is of two types:

1. Production Code: Code built to production standards. i.e. it is fully verified with positive and negative testing.
2. Reference Code: This is example code for the platform specific portions of the customers' code base. Reference code is only provided as an example to help the customer with his environment. Ideally code snippets from this will be looked at, and the customer will create his own versions of the reference code that is specific to his environment.

3.1. Core Driver Features

Supports all features of the Core:

1. IEEE 1588 v1, v2
2. IEEE 802.1AS
3. NIC functions including checksum offload for TCP, IP and UDP, large segment Tx offload for TCP (TSO) and UDP (UFO), Rx segment coalescing (RSC) and header-data splitting on Rx
4. internal FIFO DMA or packet-buffering DMA, with partial-store-and-forward mode for low latency
5. QoS/VLAN/Multi-queue functions
6. Statistics

The *Init* function configures a number of parameters on creation of the driver instance, e.g. register base address, the size of descriptor lists, the callback functions

Provide event polling API for upper layer ISR - the driver's *isr()* function should be called in response to a hardware interrupt, and will notify of events by means of client-defined callback functions, e.g. for frame transmitted, transmission error, pause event, etc.

Support descriptor memory at different memory locations from private data

- TX/RX can also be separated in different memory location/type

Driver provides functions for:

- queueing frame data buffers for transmit and receive
- buffer queue management (e.g. *queueTxBuf*, *freeTxDesc*, *resetTxQ*, *addRxBuf*, *readRxBuf*, *removeRxBuf*)
- Tx/Rx control (e.g. *startTx*, *stopTx*, *enableRx*, *disableRx*)
- status (e.g. *transmitting*, *getTxStatus*, *clearTxStatus*, *txDescFree*, *getTxDescStatus*, *getRxStatus*, *clearRxStatus*, *rxEnabled*, *getRxDescStatus*)
- recovery from errors (e.g. *abortTx*, *resetTxQ*, *resetRxQ*)
- up to 16 buffer queues for Tx and Rx to support prioritised queueing, with screening to assign Rx priority based on QoS/destination/VLAN priority/ethertype or other user-defined compare

Support re-direct DMA Rx address

- Top bits (31:28) of Rx DMA address can be multiplexed to alternative location on a packet-by-packet basis

Detailed data structures and APIs are listed in the later chapters of the document.

3.2. Naming conventions

The EMAC Core Driver API will use the prefix `CEDI_` (Cadence EMAC Driver Interface) for public top-level names of functions and data objects/definitions.

3.3. System Usage

The EMAC Core Driver is supplied with Linux driver reference code which illustrates the use of the core driver in the wider system context. For more details on this see the Linux driver Readme file.

The reference code also includes sample test code extracted from the functional test suite for the core driver. This is provided as an example of how to use the driver API for basic Tx/Rx operations. See the README file for further information.

Chapter 4. Core Driver Usage Model

4.1. Description

In order to support multiple instances and provide the minimum name space pollution, the Core Driver is implemented as a structure that contains a function pointer table.

A public header is provided to define the API object structure and the data structures used. The file is contained in the include directory of the distribution as *cedi.h*.

The API requires a private data pointer that provides the driver with instance information. It is critical that the upper software layer abides by the programming model.

Every API call apart from *probe* requires the private data address as the first parameter, to identify the driver instance and provide access to the internal state of that instance.

There is no resource protection mechanism provided in the EMAC Core Driver. It is expected that the higher level software stack will be responsible in protecting calls to the core driver where appropriate, for example when queuing a multi-buffer frame for transmission. In critical areas such as the calling of the *isr* function, usage is expected to be single threaded. Multi-threaded processing of a received frame is possible, provided that each priority queue handles its own buffer freeing and recycling after any callback function has returned.

4.2. Probing the Hardware

The client code must obtain the object pointer to the core driver. It can be retrieved by issuing the *CEDI_GetInstance()* function. This provides access to all of the API functions, but initially only *probe* may be called.

Before initialising the core driver, the client must first invoke the *probe* function. The *probe* function returns the dynamic memory requirements for the driver, depending on the configuration specified. The configuration information is passed to *probe* in an *CEDI_Config* struct, although at this stage only those fields which affect the requirements need to be set, as indicated in the struct description. The client may repeat *probe* calls to customise configuration depending on available system resources.

The client code then must allocate the memory to the sizes returned by the probe function. One of these allocations is for the driver's private data, with other blocks being required for transmit (Tx) and receive (Rx) descriptor lists and a copy of the Tx/Rx statistics. If multiple priority queues are to be configured, the descriptor lists for all Tx queues are allocated in one contiguous block, with another block for the Rx queue descriptors.

e.g. with 3 Tx and Rx queues of 80 descriptors each:

```
...
CEDI_Config cfg;
CEDI_SysReq req;

/* get API function pointers for this driver instance */
CEDI_OBJ *emacObj = CEDI_GetInstance();
uint16_t numDescriptors[16] = {80,80,80,0,0,0,0,0,0,0,0,0,0,0,0,0};

/* Initialise config structure fields required by probe */
cfg.regBase = 0xFFF30000; /* IP core registers base address */
cfg.rxQs = 3;
cfg.txQs = 3;
```

```
memcpy(cfg.txQlen, numDescriptors, sizeof(numDescriptors));
memcpy(cfg.txQlen, numDescriptors, sizeof(numDescriptors));

emacObj->probe(&cfg, &req);

printf("privateData object requires %u bytes\n", req.pDataSize);
printf("Tx descriptors require %u bytes\n", req.txDescListSize);
printf("Rx descriptors require %u bytes\n", req.rxDescListSize);
...
```

4.3. Initialisation

After this stage, the client must reserve the necessary memory areas for the driver's private data, the transmit descriptor lists, receive descriptor lists, statistics data and hardware configuration (if required). The base addresses of descriptor lists must be aligned to the data word boundary, i.e. 32-bit boundary for 32-bit DMA addressing or 64-bit boundary for 64-bit DMA addressing. The CEDI_Config struct should be filled out with the locations of these memory blocks (except the private data), and values for all the other initial configuration fields.

The driver can now be initialised with an *init* call, which will:

- initialise the driver instance and state
- configure the hardware as specified in the *config* parameter fields
- construct up the Tx and Rx descriptor lists
- clear the statistics, Tx Status and Rx Status registers

e.g. *init* example, following on from *probe* example above:

```
...
static void _ev_phyManComplete(CEDI_PrivateData *pD, uint8_t read,
                               uint16_t readData) {
    ...
}

static void _ev_rxFrame(CEDI_PrivateData *pD, uint8_t queueNum) {
    ...
}

CEDI_Callbacks core_callbacks = {
    .phyManComplete = _ev_phyManComplete,
    .rxFrame        = _ev_rxFrame,
    // etc.
};

CEDI_PrivateData *privData = malloc(req.privDataSize);
CEDI_Statistics *statRegs = (CEDI_Statistics *)malloc(req.statsSize);

...

/* allocate Tx and Rx descriptor rings */
cfg->rxQAddr = NCPS_allocHWMem(req.rxDescListSize);
cfg->txQAddr = NCPS_allocHWMem(req.txDescListSize);
if (!cfg->rxQAddr || !cfg->txQAddr) {
```

```

    printf("Unable to allocate space for descriptors\n");
    return ERR_1;
}

/* fill in rest of config struct */
cfg->dmaBusWidth = CEDI_DMA_BUS_WIDTH_64;
/* enable all Tx & Rx interrupts on start */
cfg->intrEnable = CEDI_EV_TX_COMPLETE | CEDI_EV_TX_USED_READ |
                 CEDI_EV_TX_UNDERRUN | CEDI_EV_TX_RETRY_EX_LATE_COLL |
                 CEDI_EV_TX_FR_CORRUPT | CEDI_EV_RX_COMPLETE |
                 CEDI_EV_RX_USED_READ | CEDI_EV_RX_OVERRUN |
                 CEDI_EV_HRESP_NOT_OK;
cfg->rxPktBufSize = 3;
... /* etc. */

cfg->statsRegs = (uintptr_t)statRegs;

if (0 != (result = emacObj->init(privData, &cfg, &core_callbacks))) {
    printf("Error in emacObj->init: returned %d\n", result);
}

```

At this point, any further setup not provided by CEDI_Config should be done, e.g. specific addresses, other receive filtering, adding receive buffers to the descriptor list(s).

When ready, *start* should be called to enable the interrupts specified in the CEDI_Config *intrEnable* field.

4.4. Event Handling

The Core Driver notifies upper layer software of new events and state changes by means of the *isr* API function. The *isr* function may be invoked by upper layer software as part of the interrupt processing being handled by the upper layer software. Alternatively, if interrupt-driven operation is not desired, *isr* can be called by frequent polling. In the latter case, the EMAC interrupt should not lead to a read of the interrupt status register by the client software, as this must be interpreted and cleared by the Core Driver's *isr* function.

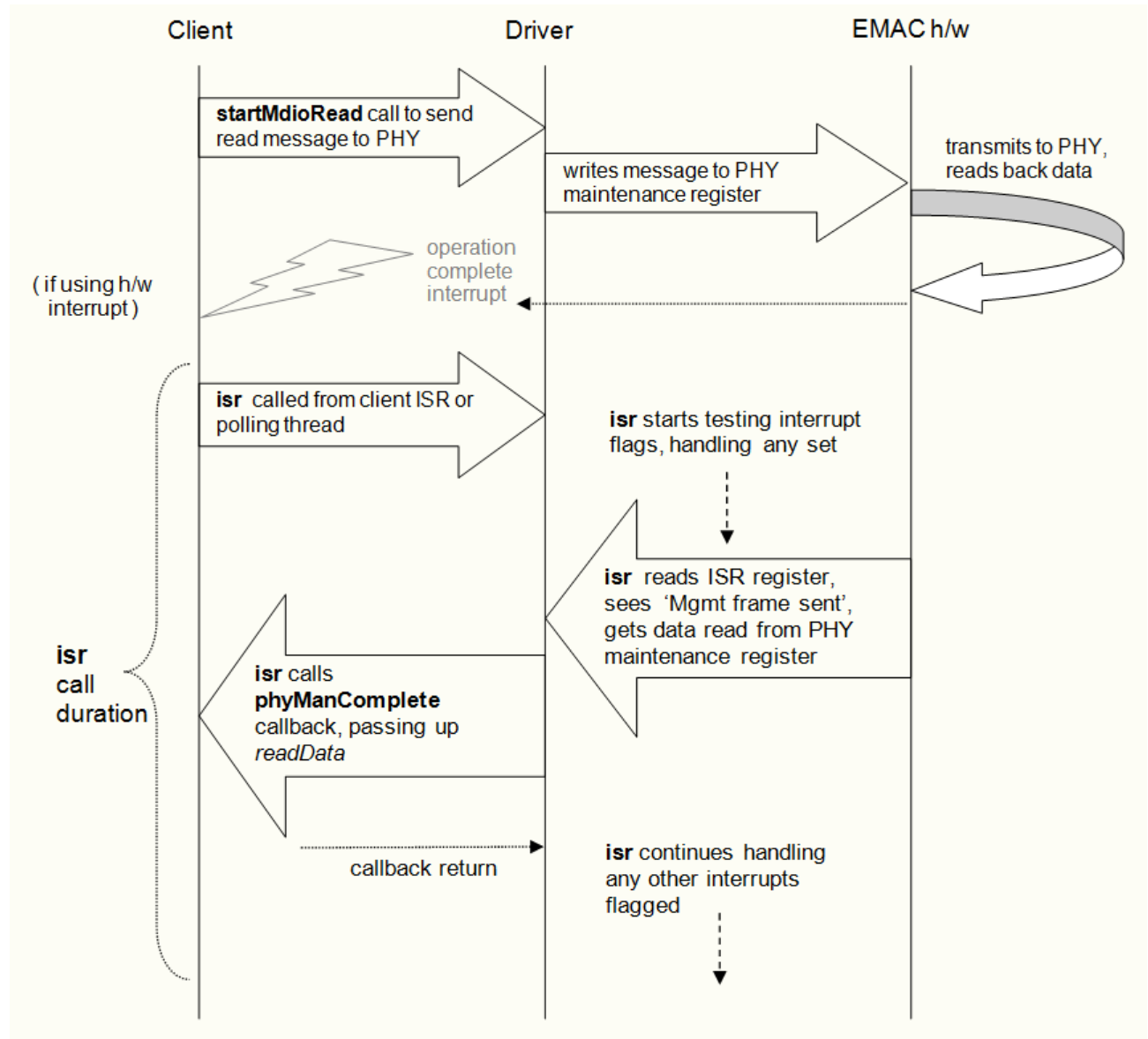
The invocation of the *isr* function of the Core Driver will result in a callback to the function registered as part of the *init* process IF there is an enabled event or error condition that is encountered by the Core driver. It is the responsibility of the upper layer software to deal with any errors that are reported by the Core driver.

Actions performed by the upper layer during a callback function should be kept to a minimum, as the function must return before further *isr* event-checking can proceed. Data passed by pointer in callbacks (e.g. PTP event time, auto-negotiation page data) is held in the driver's private data and remains available after the callback return, but may be overwritten on detection of the next event of a similar type.

In addition to the events enabled by the *start* call, dynamic enabling and disabling of the EMAC interrupts/events is available through the *setEventEnable* function. The events and callbacks are described at the end of this chapter and in the chapter "Dynamic Data Structures".

If multiple priority queues are in use, the Tx and Rx status registers will be processed in descending order of priority.

The following diagram illustrates the callback mechanism in relation to a PHY management read :

Figure 4.1. Callback example showing MDIO read data operation

4.5. Transmit Operation

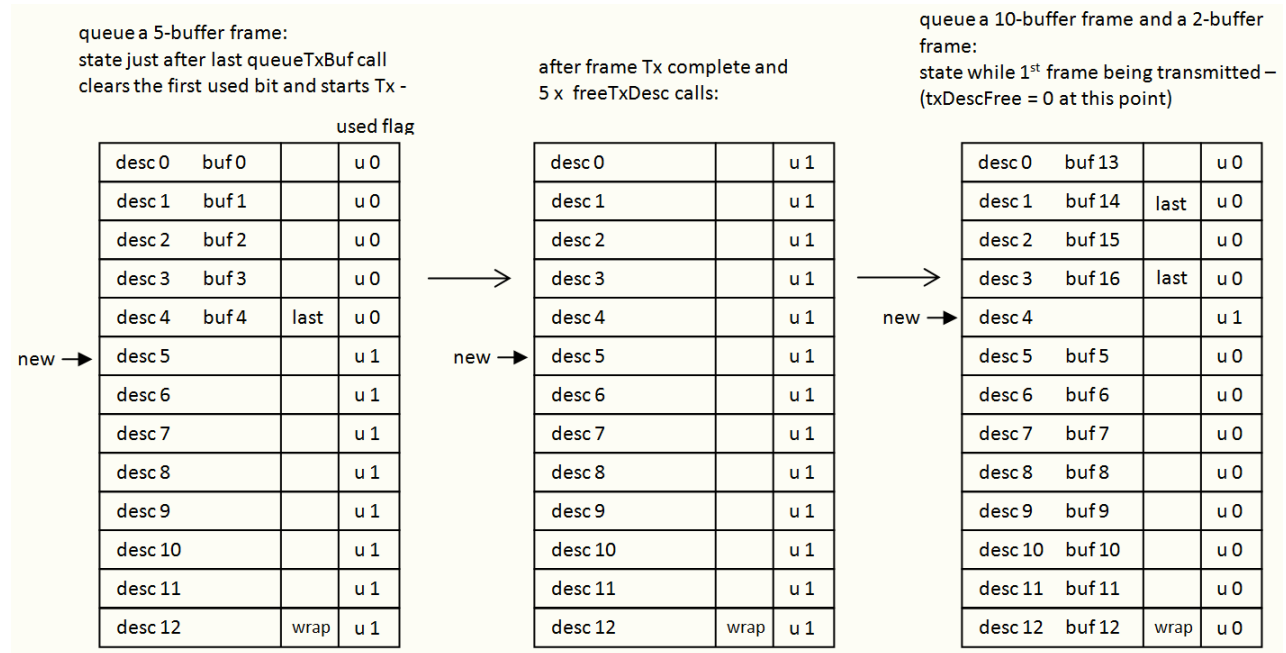
init will initialise the buffer descriptor lists and internal pointers, and the `CEDI_EV_TX_COMPLETE` (a frame has been transmitted) and/or `CEDI_EV_TX_USED_READ` (all transmit queues empty) events should be enabled. Other transmit events may also be enabled for error handling.

Frame transmission simply requires calling `queueTxBuf` for each of the buffers which make up a frame to be transmitted, and setting the `lastBuffer` flag for the last call. When the driver sees this flag, it will start transmission automatically.

The *isr* callback `txEvent` is used to notify that transmission has completed successfully. In response to this, software should call `freeTxDesc` to free the descriptor(s) used the frame, and receive the buffer address for frame identification and the descriptor status. When queueing buffers the number of free descriptors can be checked by calling `txDescFree`.

e.g. after init with txQLen = 12 (Note: an extra descriptor is added to list to prevent transmit reads from wrapping round before the used flags are set after transmission) :

Figure 4.2. Tx descriptor list example



In a concurrent processing environment, the client will need to arrange mutex locking to protect each transmit queue between calls of *queueTxBuf* while queueing a multi-buffer frame.

To reduce interrupt frequency, software can disable the CEDI_EV_TX_COMPLETE event and simply poll on *freeTxDesc* after queueing frames to transmit.

Simple transmit example, using only "polling" methods to monitor Tx progress, but assuming TxError callback is defined to flag errors:

```
...
/* enable cfg->intrEnable events */
emacObj->start(privData);
emacObj->enableTx(privData);
...

qNum = 0;
framesQd = 0;
txOk = 0;

/* disable Tx events for polling */
emacObj->setEventEnable(privData,
    CEDI_EV_TX_COMPLETE | CEDI_EV_TX_USED_READ, 0, qNum);

/* Tx "numFrames" single-buffer frames from txBuf[]
    buffer address array */
for (t = 10000; (!txOk) && t; --t) {
```

```

if (framesQd < numFrames) {
    /* add another frame to the Tx queue if space available */
    if (0 != (result = emacObj->txDescFree(privData, qNum, &nFree)))
        printf("Error calling txDescFree (frame=%u): returned %u\n",
               framesQd, result);

    else if (nFree) {
        if (0 != (result = emacObj->queueTxBuf(privData, qNum,
            &txBuf[framesQd % 10], txLen, CEDI_TXB_LAST_BUFF)))
            printf("Error queueing Tx frame (frame=%u): returned %u\n",
                   framesQd, result);
        else
            framesQd++;
    }
}

/* free up descriptors when possible */
if (EINVAL == (result =
    emacObj->freeTxDesc(privData, qNum, &freeBuf,
    &txDescStat, &txStatus)))
    printf("Error freeing Tx descriptor: returned %u\n", result);

else
    /* done when queue empty */
    txOk = (result == ENOENT) && (framesQd == numFrames);

/* check for errors/other events */
emacObj->isr(privData);

/* see if callbacks have set any flags */
if (0 != (error = (txUnderrun | txRetryExc | txFrameCorr)))
    print("Transmit error detected - code = %X\n", error);
}

```

4.5.1. Large Segment Offload

LSO operation is enabled through the use of the parameters supplied with a *qTxBuf* call. Each large TCP or UDP frame to be segmented/fragmented must have its L2/L3/L4 headers queued in one Tx descriptor buffer using one *qTxBuf* call, followed by the data payload in further *qTxBuf* calls.

For TSO, the *qTxBufParams flags* field should have the value *CEDI_TXB_TCP_ENCAP* (OR'd with *CEDI_TXB_TSO_AUTO_SEQ* if required - see description in Chapter 5), *mssMfs* should be set to MSS and *tcpStream* should be set as required (selects a TCP stream for maintaining sequence numbering). Some of these parameters apply only to the first (header) descriptor, and others to later (data) descriptors, but they should all be set for each *qTxBuf* call in a frame, and the driver will apply them when required.

For UFO, the *qTxBufParams flags* field should have the value *CEDI_TXB_UDP_ENCAP*, and the *mssMfs* is used for MFS (Maximum Fragment Size).

Example of queueing a TCP frame for transmitting by LSO, with header and a single data buffer:

```

CEDI_qTxBufParams txParams;
...
// queue header for Tx
txParams.queueNum = qNum;

```

```

txParams.bufAdd = &txBuf[0]; /* CEDI_BuffAddr pointing to buffer containing header */
txParams.length = hdrSize;
txParams.flags = CEDI_TXB_TCP_ENCAP;
txParams.mssMfs = 1400;
txParams.tcpStream = stream;

if (0!=(result = emacObj->qTxBuf(privData, &txParams))) {
    printf("Error queueing Tx header: returned %u\n",result);
    return EINVAL;
}

// queue data for Tx
txParams.bufAdd = &txBuf[1]; /* CEDI_BuffAddr pointing to buffer containing data */
txParams.length = bufSize;
txParams.flags = CEDI_TXB_TCP_ENCAP | CEDI_TXB_LAST_BUFF;

if (0!=(result = emacObj->qTxBuf(privData, &txParams))) {
    printf("Error queueing Tx data: returned %u\n",result);
    return EINVAL;
}

```

4.6. Receive Operation

The following description details only operation of a single receive priority queue:

After initialisation, *addRxBuf* should be called the required number of times to add receive buffers into the descriptor list. When ready, enable reception with *enableRx*.

When a frame has been received, the software will be notified via the *rxFrame* callback from *isr*. In this callback, software can call *numRxUsed* to check for the number of data buffers to be read.

To read data, the *readRxBuf* function should be called. This must be supplied with a new buffer to replace the full buffer being returned. Its return status indicates whether any data was available, and whether the Start Of Frame (SOF) and End Of Frame (EOF) flags were set - see *readRxBuf* for details.

When a buffer is read with EOF set, the frame status including matching details can be obtained using the *rxDescStat* pointer parameter.

4.6.1. Detecting discarded fragments

If an error, such as oversize frame, occurs during reception of a multi-buffer frame, the start of the discarded frame will be left in buffers. The *readRxBuf* status value must be checked to detect any frame fragment left, which will show as a frame starting with SOF but no EOF occurring before the next SOF is found:

- after status = CEDI_RXDATA_SOF_ONLY, software is waiting for status = CEDI_RXDATA_EOF_ONLY, to complete the frame

- if software then gets status = CEDI_RXDATA_SOF_EOF or CEDI_RXDATA_SOF_ONLY, it discards the previous buffers belonging to an incomplete frame

e.g. after init with rxQLen = 16 (Note: an extra descriptor is added to list to prevent transmit reads from wrapping round before the used flags are set after transmission) :

Figure 4.3. Example of Rx descriptor list operation:

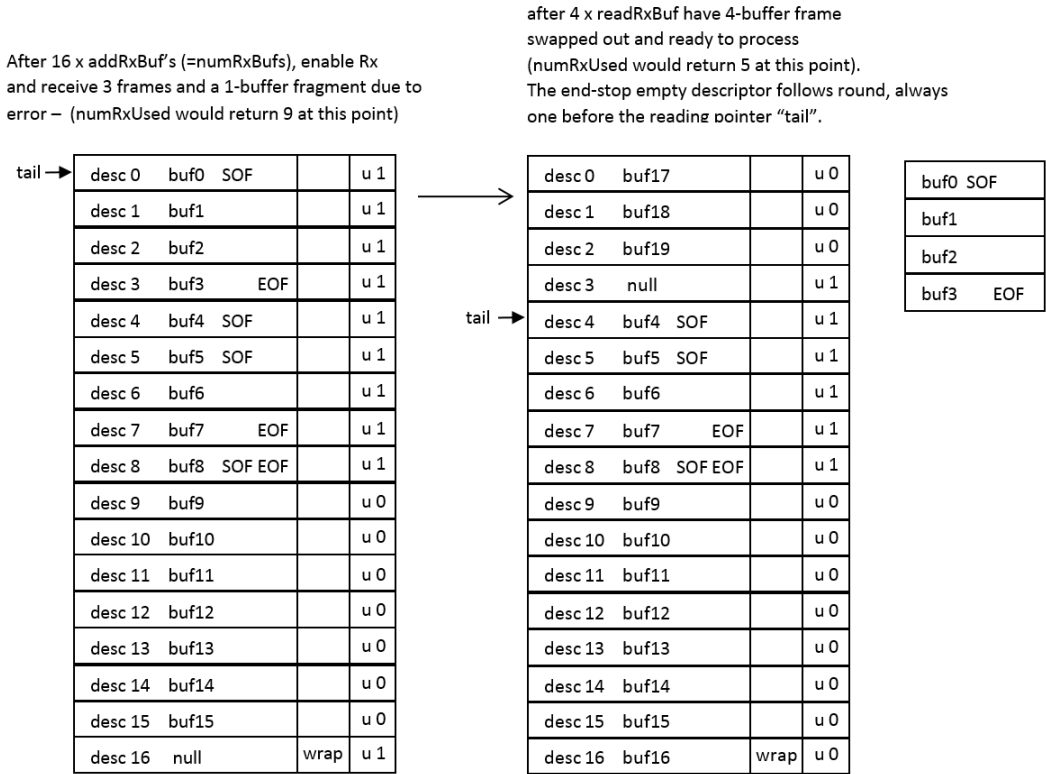
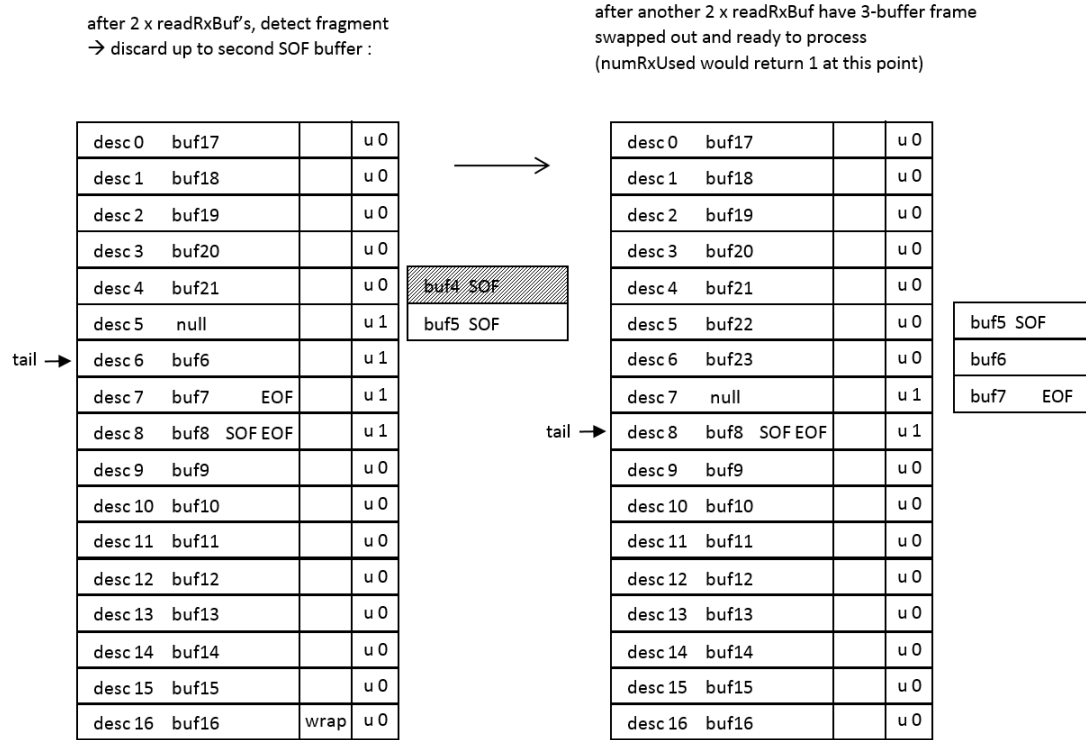


Figure 4.4. Rx example (continued) showing discarded frame fragments:

Note this API is usable in a NAPI mode, only requiring that the *rxFrame* event is disabled when Rx buffers are to be read, using polling with *readRxBuf* calls to process the Rx queue, and re-enabling the interrupt when no more Rx data is available (*status=CEDI_RXDATA_NODATA*). Freeing Tx descriptors can also be done by polling with *freeTxDesc* until *descData->status= CEDI_TXDATA_NONE_FREED*.

Simple example of a receive loop for reading from descriptor list for queue 0, assuming an array of buffer pointers in "nBuf":

```
...
rxBufs = 0;
rxFrames = 0;
newIndex = 0;

/* allocate a buffer to swap in when read data */
if (0!=allocBuffer(rxBufLenBytes, &nBuf[newIndex]))
    printf("Error allocating new Rx buffer\n");
else
{
    /* poll numRxUsed until read all frames */
    while (emacObj->numRxUsed(privData, 0))
    {
        if (0!=(result = emacObj->readRxBuf(privData, 0,
            &nBuf[newIndex], 0, &rxDStat, &readStatus)))
        {
            printf("Error readRxBuf returned %u\n", result);
        }
        else
        {
            printf("Error readRxBuf returned %u\n", result);
        }
    }
}
```

```

{
    if (readStatus!=CEDI_RXDATA_NODATA) {
        /* data was read, and
           nBuf[newIndex] now points to it */

        /* fragment detection */
        if ((rxBufs>0) &&
            ((readStatus==CEDI_RXDATA_SOF_ONLY) ||
             (readStatus==CEDI_RXDATA_SOF_EOF)))
        {
            /* discard buffers read in previous frame */
            ...
        }

        if ((readStatus==CEDI_RXDATA_SOF_EOF) ||
            (readStatus==CEDI_RXDATA_EOF_ONLY)) {
            rxFrames++;
            rxBufs = 0;
        }
        else
            rxBufs++;

        /* increment buffer index */
        newIndex = (newIndex+1) % privData->cfg.rxQLen[0];

        /* Allocate another buffer to swap in */
        if (0!=allocBuffer(rxBufLenBytes, &nBuf[newIndex]))
            printf("Error allocating new Rx buffer\n");
    }
}
}
}
}

```

4.6.2. Receive Segment Coalescing

RSC operation is enabled on a per-queue basis using the *setRscEnable* function (must have more than one Rx queue present in the hardware configuration). Header/data splitting must also be enabled for RSC (*setHdrDataSplit*)

Filtering a TCP stream into a specific Rx queue for coalescing is done using the type 2 screener registers combined with three compare registers with masking disabled, which provides a total of 12 bytes comparison, sufficient to match source and destination IP addresses and ports.

Example of how to configure screener & compare registers to direct a specific TCP stream using a 4-tuple combination of IP addresses and port numbers:

```

...

/* need header-splitting enabled */
if (EOK!=emacObj->setHdrDataSplit(privData[emacB], 1)) {
    printf("Error - setHdrDataSplit call failed!\n");
    goto done;
}

rxQNum = 1;
screenReg = 0;
compReg = 0;

/* offset 12 bytes after EtherType for IP source address */
cmp0.offsetPosition = CEDI_T2COMP_OFF_ETYPE;

```

```

cmp0.offsetVal = 12;          /* in bytes */
cmp0.disableMask = 1;        /* enable 4-byte compare */
cmp0.compValue = (tcpHdrP1.srcIPAddr.byte[3] << 8) | tcpHdrP1.srcIPAddr.byte[2];
cmp0.compMask = (tcpHdrP1.srcIPAddr.byte[1] << 8) | tcpHdrP1.srcIPAddr.byte[0];

if (0!=(result = emacObj->setType2CompareReg(privData[emacB], compReg, &cmp0))) {
    printf("Error return from setType2CompareReg enable - value: %d\n", result);
    goto done;
}
/* offset 16 bytes after EtherType for IP dest address */
cmp1.offsetPosition = CEDI_T2COMP_OFF_ETYPE;
cmp1.offsetVal = 16;          /* in bytes */
cmp1.disableMask = 1;        /* enable 4-byte compare */
cmp1.compValue = (tcpHdrP1.dstIPAddr.byte[3] << 8) | tcpHdrP1.dstIPAddr.byte[2];
cmp1.compMask = (tcpHdrP1.dstIPAddr.byte[1] << 8) | tcpHdrP1.dstIPAddr.byte[0];

if (0!=(result = emacObj->setType2CompareReg(
    privData[emacB], compReg+1, &cmp1))) {
    printf("Error return from setType2CompareReg: %d\n", result);
    goto done;
}
/* offset 0 bytes after IP header for TCP source & dest ports */
cmp2.offsetPosition = CEDI_T2COMP_OFF_IPHDR;
cmp2.offsetVal = 0;          /* in bytes */
cmp2.disableMask = 1;        /* enable 4-byte compare */
cmp2.compMask = (byte0(tcpHdrP1.srcPort)<<8) | byte1(tcpHdrP1.srcPort);
cmp2.compValue = (byte0(tcpHdrP1.dstPort)<<8) | byte1(tcpHdrP1.dstPort);

if (0!=(result = emacObj->setType2CompareReg(privData[emacB],
    compReg+2, &cmp2))) {
    printf("Error return from setType2CompareReg: %d\n", result);
    goto done;
}

/* set up screening Type2 register to match 4-tuple */
scrn1.vlanEnable = 0;
scrn1.eTypeEnable = 0;
scrn1.compAEnable = 1;
scrn1.compAIndex = compReg;
scrn1.compBEnable = 1;
scrn1.compBIndex = compReg+1;
scrn1.compCEnable = 1;
scrn1.compCIndex = compReg+2;
scrn1.qNum = rxQNum;

if (0!=(result = emacObj->setType2ScreenReg(privData[emacB],
    screenReg, &scrn1))) {
    printf("Error return from setType2ScreenReg enable reg %u - value: %d\n",
        screenReg, result);
    goto done;
}

if (EOK!=emacObj->setRscEnable(privData[emacB], rxQNum, 1)) {
    printf("Error - setRscEnable call failed!\n");
    goto done;
}
result = emacObj->getRscEnable(privData[emacB], rxQNum, &enabled);

```

4.7. Interspersing Express Traffic (802.3br)

To enable *Interspersing Express Traffic* support in Core Driver, *incExpressTraffic* field in *CEDI_Config* struct (which is passed to *probe* function) has to be set. The *probe* may clear that field, indicating that device does not support *Interspersing Express Traffic*.

Operation of *Interspersing Express Traffic* relies on existence of 2 independent MAC cores: preemptible MAC (*pMAC*) and express MAC (*eMAC*). Those are represented in driver by 2 separate private data instances. The *probe* call returns the dynamic memory requirements for the driver, which covers memory required for both private data instances. That area will be splitted between private data instances by the driver itself.

After *init* call, pointer points to *pMAC*'s private data. *eMAC*'s private data pointer may be acquired using *getEmac* function. The same functions are used to control both *pMAC* and *eMAC*, simply by providing corresponding private data pointer. Some functions are exception to this, as they either handle both MACs at once internally, or are not valid for *eMAC* at all. Function, that handles both MACs at once, expects private data pointer corresponding to *pMAC*.

Functions, that handle of both MACs at once:

- probe
- init
- destroy
- start
- stop
- isr
- setIfSpeed
- setFullDuplex

Functions, that are not valid for *eMAC*:

- setUniDirEnable
- resetPcs
- getPcsReady
- startAutoNegotiation
- setAutoNegEnable
- getAutoNegEnable
- getLinkStatus
- getAnRemoteFault
- getAnComplete
- setAnAdvPage
- getAnAdvPage
- getLpAbilityPage
- getPageRx
- setNextPageTx
- getNextPageTx
- getLpNextPage
- getPhyId
- setMdioEnable
- getMdioEnable
- phyStartMdioWrite
- phyStartMdioRead
- getMdioReadData
- getMdioIdle

- setCollisionTest
- getCollisionTest
- writeUserOutputs
- readUserOutputs
- setUserOutPin
- readUserInputs
- setMiiOnRgmii
- getMdioInState
- set1588Timer
- adjust1588Timer
- set1588TimerInc
- get1588TimerInc
- setExtTsuPortEnable
- getExtTsuPortEnable

In addition, all functions that rely on existence of multiple queues are not valid for *eMAC* as well, since it only has 1 TX queue and 1 RX queue. Such functionalities include, but are not limited to:

- Type 1 screener registers
- Receive Side Coalescing (RSC)
- TX priority schedulers

If device supports *Interspersing Express Traffic*, then additional module, called MMSL (MAC Merge Sub Layer) exists in that device, and can be configured from Core Driver. There can be only one MMSL per device. Following functions are responsible for interaction with MMSL:

- setPreemptionConfig
- getPreemptionConfig
- preemptionRestartVerification
- setMmslEventEnable
- getMmslEventEnable
- readMmslStats
- readMmslStatus

4.8. ASF - Active Safety Feature

ASF functionality is not covered by GEM Core Driver. There is a ASF Core Driver included in this package. ASF should be initialized and configured by ASF Core Driver functions. ASF Core Driver implements own isr function to handle ASF interrupts. There is a function in GEM Core Driver which can be used to get number of ASF modules in GEM. There could be 0, 1 or 2 ASF modules. If GEM contains at least one ASF module, then ASF register base is returned by this function. The function name is getAsfInfo.

4.9. Quitting Driver

The *stop* function can be called to end the driver's active operation.

Transmission will be halted (allowing frame tx to complete), receive will be reset, the descriptor queues will be cleared, PCS will be reset and all events disabled.

At the end of the driver instance's life, *destroy* should be called to terminate use.

This stops any ongoing activity, disables the hardware and de-initialises the driver. The driver instance cannot be used after this call.

4.10. PCS Auto-negotiation

The full PCS auto-negotiation functionality is only relevant to a EMAC configuration which requires the PCS for a non-SGMII PHY interface, i.e. 1000BASE-X.

(For MII/GMII interfaces, the auto-negotiation operation is carried out by the PHY, and can be monitored through the PHY management interface. For SGMII use, the results of the auto-negotiation can be read from the link partner ability register using *getLpAbilityPage* .)

The proper handling of PCS auto-negotiation requires that the `CEDI_EV_PCS_LP_PAGE_RX` and `CEDI_EV_PCS_AN_COMPLETE` events are enabled.

The process can be initiated by calling *startAutoNegotiation*, and is continued by responding to the following callbacks. The first *lpPageRx* callback will include the link partner's abilities settings.

If the client side needs to send next pages, the next page register must be written by a *setNextPageTx* call within 10 ms of the previous page being notified by the link partner page received interrupt. Next pages received from the link partner will be returned by further *lpPageRx* callbacks.

When client side has no more next pages to transmit, the driver will automatically write a null message page to transmit if necessary.

Completion of auto-negotiation is indicated by an *anComplete* callback, which will report the network resolution status.

4.11. Core Driver Interrupt/Event Types

Table 4.1. Callback Functions and Events

Callback Function	Reports Event(s)	Description
PhyManComplete	CEDI_PHY_MAN_FRAME	PHY Management Frame has completed transmission (and reception, for read operations) on MDIO interface. Core driver checks whether write or read, and copies in PHY data if read
TxEvt	CEDI_EV_TX_COMPLETE, CEDI_EV_TX_USED_READ	Can indicate two types of event simultaneously: TX_COMPLETE indicates a frame has completed transmission on the specified queue, and TX_USED_READ indicates no more frames queued
TxError	CEDI_EV_TX_UNDERRUN, _RETRY_EX_LATE_COLL, CEDI_EV_TX_FR_CORRUPT	Three possible Tx error conditions - queueNum is not set for CEDI_EV_TX_UNDERRUN
RxFrameComplete	CEDI_EV_RX_COMPLETE	A frame has been received and stored in memory
RxError	CEDI_EV_RX_USED_READ, CEDI_EV_RX_OVERRUN	A Rx error has occurred
HrespError	CEDI_EV_HRESP_NOT_OK	DMA HResp error, probably due to incorrect DMA memory configuration

Callback Function	Reports Event(s)	Description
		such as buffer address pointing to a non-existent memory region, and may be notified at the same time as another associated error (e.g. CEDI_EV_TX_UNDERRUN) or on its own.
PauseEvent	CEDI_EV_PAUSE_FRAME_TX, CEDI_EV_PAUSE_TIME_ZERO, CEDI_EV_PAUSE_NZ_QU_RX	PAUSE_TIME_ZERO means either pause time register has decremented to zero, or a zero quantum pause frame received. CEDI_EV_PAUSE_NZ_QU_RX means pause frame with non-zero quantum received
PtpPrimaryFrameTx	CEDI_EV_PTP_TX_DLY_REQ, CEDI_EV_PTP_TX_SYNC	PTP Primary Frame has been transmitted. The captured time is supplied.
PtpPeerFrameTx	CEDI_EV_PTP_TX_PDLY_REQ, CEDI_EV_PTP_TX_PDLY_RSP	PTP Peer Frame has been transmitted. The captured time is supplied.
PtpPrimaryFrameRx	CEDI_EV_PTP_RX_DLY_REQ, CEDI_EV_PTP_RX_SYNC	PTP Primary Frame has been received. The captured time is supplied.
PTPPeerFrameRx	CEDI_EV_PTP_RX_PDLY_REQ, CEDI_EV_PTP_RX_PDLY_RSP	PTP Peer Frame has been received. The captured time is supplied.
TsuEvent	CEDI_EV_TSU_SEC_INC, CEDI_EV_TSU_TIME_MATCH	Time Stamp Unit events - seconds register increment and/or timer count match
LpiStatus	CEDI_EV_LPI_CH_RX	LPI indication status bit has changed
WolEvent	CEDI_EV_WOL_RX	A Wake on LAN event has been received
ExtInpIntrEvent	CEDI_EV_EXT_INTR	External Input Interrupt - a rising edge detected on the ext_interrupt_in pin
LpPageRx	CEDI_EV_PCS_LP_PAGE_RX	PCS auto-negotiation page received from the link partner. The link partner base or next page data is supplied in an CEDI_LpPageRx struct.
AnComplete	CEDI_EV_PCS_AN_COMPLETE	PCS auto-negotiation has completed. Network resolution status is provided in an CEDI_NetAnStatus struct.
LinkChange	_EV_PCS_LINK_CHANGE_DET	Link status has changed state, supplied via linkState parameter. If auto-negotiation not enabled, the synchronistaion status is supplied.

Chapter 5. Configuration and Hardware Operation Information

5.1. Introduction

The following definitions specify the driver operation environment that is defined by hardware configuration or client code.

These defines are located in the header file of the core driver.

Table 5.1. Configuration and Hardware Operation Information

#define	value	description
General config.		
CEDI_MAX_RX_QUEUES	(16U)	Max number of Rx priority queues supported by driver
CEDI_MAX_TX_QUEUES	(16U)	Max number of Tx priority queues supported by driver.
CEDI_ALL_QUEUES	(255U)	Allows en/disabling of events on all priority queues.
CEDI_MAX_RBQ_LENGTH	(1000U)	Max size of each Rx buffer descriptor list.
CEDI_MAX_TBQ_LENGTH	(1000U)	max size of each Tx buffer descriptor list
Bit-flags to specify DMA endianism.		
CEDI_END_SWAP_DESC	(0x01U)	
CEDI_END_SWAP_DATA	(0x02U)	
Bit-flags to specify DMA config register bits 24-27; see EMAC User Guide for details.		
CEDI_CFG_DMA_DISC_RXP	(0x01U)	
CEDI_CFG_DMA_FRCE_RX_BRST	(0x02U)	
CEDI_CFG_DMA_FRCE_TX_BRST	(0x04U)	
Bit-flags to specify checksum offload operation.		
CEDI_CFG_CHK_OFF_TX	(0x01U)	
CEDI_CFG_CHK_OFF_RX	(0x02U)	
Bit-flags for selecting interrupts to enable/disable/read.		
CEDI_EV_TX_COMPLETE	(0x00000001)	Frame has been transmitted successfully
CEDI_EV_TX_USED_READ	(0x00000002)	Used bit set has been read in Tx descriptor list.
CEDI_EV_TX_UNDERRUN	(0x00000004)	Tx underrun.
CEDI_EV_TX_RETRY_EX_LATE_COLL	(0x00000008)	Retry limit exceeded or late collision.
CEDI_EV_TX_FR_CORRUPT	(0x00000010)	Tx frame corruption.
CEDI_EV_RX_COMPLETE	(0x00000020)	Frame received successfully and stored.

#define	value	description
CEDI_EV_RX_USED_READ	(0x00000040)	Used bit set has been read in Rx descriptor list.
CEDI_EV_RX_OVERRUN	(0x00000080)	Rx overrun error.
CEDI_EV_HRESP_NOT_OK	(0x00000100)	DMA hresp not OK.
CEDI_EV_PCS_LP_PAGE_RX	(0x00000200)	PCS auto-negotiation link partner page received.
CEDI_EV_PCS_AN_COMPLETE	(0x00000400)	PCS auto-negotiation has completed.
CEDI_EV_PCS_LINK_CHANGE_DET	(0x00000800)	Link status change detected by PCS.
CEDI_EV_PAUSE_FRAME_TX	(0x00001000)	Pause frame transmitted.
CEDI_EV_PAUSE_TIME_ZERO	(0x00002000)	Pause time reaches zero or zeroq pause frame received.
CEDI_EV_PAUSE_NZ_QU_RX	(0x00004000)	Pause frame with non-zero quantum received.
CEDI_EV_PTP_TX_DLY_REQ	(0x00008000)	PTP delay_req frame transmitted.
CEDI_EV_PTP_TX_SYNC	(0x00010000)	PTP sync frame transmitted.
CEDI_EV_PTP_TX_PDLY_REQ	(0x00020000)	PTP pdelay_req frame transmitted.
CEDI_EV_PTP_TX_PDLY_RSP	(0x00040000)	PTP pdelay_resp frame transmitted.
CEDI_EV_PTP_RX_DLY_REQ	(0x00080000)	PTP delay_req frame received.
CEDI_EV_PTP_RX_SYNC	(0x00100000)	PTP sync frame received.
CEDI_EV_PTP_RX_PDLY_REQ	(0x00200000)	PTP pdelay_req frame received.
CEDI_EV_PTP_RX_PDLY_RSP	(0x00400000)	PTP pdelay_resp frame received.
CEDI_EV_TSU_SEC_INC	(0x00800000)	TSU seconds register increment.
CEDI_EV_TSU_TIME_MATCH	(0x01000000)	TSU timer count match.
CEDI_EV_MAN_FRAME	(0x02000000)	Management Frame Sent.
CEDI_EV_LPI_CH_RX	(0x04000000)	LPI indication status bit change received.
CEDI_EV_WOL_RX	(0x08000000)	Wake on LAN event received.
CEDI_EV_EXT_INTR	(0x10000000)	External input interrupt detected.
CEDI_EV_RX_LOCKUP	(0x40000000)	RX lockup has been detected.
CEDI_EV_TX_LOCKUP	(0x80000000)	TX lockup has been detected.
CEDI_CAP_LSO	(0x00000001)	Bit-flags for indicating functionality supported by the driver/EMAC 3 REMOVE_ME Large Segmentation Offload.
CEDI_CAP_RSC	(0x00000002)	Receive Side Coalescing offload.
CEDI_CAP_RSS	(0x00000004)	Receive Side Scaling offload.
Bit-flags for selecting Tx Status fields to reset.		
CEDI_TXS_USED_READ	(0x0001)	

#define	value	description
CEDI_TXS_COLLISION	(0x0002)	
CEDI_TXS_RETRY_EXC	(0x0004)	
CEDI_TXS_FRAME_ERR	(0x0010)	
CEDI_TXS_TX_COMPLETE	(0x0020)	
CEDI_TXS_UNDERRUN	(0x0040)	
CEDI_TXS_LATE_COLL	(0x0080)	
CEDI_TXS_HRESP_ERR	(0x0100)	
Bit-flags for selecting Rx Status fields to reset.		
CEDI_RXS_NO_BUFF	(0x0001)	
CEDI_RXS_FRAME_RX	(0x0002)	
CEDI_RXS_OVERRUN	(0x0004)	
CEDI_RXS_HRESP_ERR	(0x0008)	
Transmit IP/TCP/UDP checksum generation offload errors reported in Tx descriptor status.		
CEDI_TXD_CHKOFF_NO_ERROR	(0)	
CEDI_TXD_CHKOFF_VLAN_HDR_ERR	(1)	
CEDI_TXD_CHKOFF_SNAP_HDR_ERR	(2)	
CEDI_TXD_CHKOFF_INVALID_IP	(3)	
CEDI_TXD_CHKOFF_INVALID_PKT	(4)	
CEDI_TXD_CHKOFF_PKT_FRAGMENT	(5)	
CEDI_TXD_CHKOFF_NON_TCP_UDP	(6)	
CEDI_TXD_CHKOFF_PREM_END_PKT	(7)	
Bit-flags to control queueTxBuf/qTxBuf behaviour.		
CEDI_TXB_LAST_BUFF	(0x01)	Indicates last buffer in frame
CEDI_TXB_NO_AUTO_CRC	(0x02)	EMAC will not calculate & insert frame CRC.
CEDI_TXB_NO_AUTO_START	(0x04)	Prevents transmission being started automatically after queueing the last buffer in the frame; has no effect if CEDI_TXB_LAST_BUFF not also present.
CEDI_TXB_TCP_ENCAP	(0x08)	select TCP encapsulation for the current frame - must be set for first and second buffers in frame, ie header and first data buffers
CEDI_TXB_UDP_ENCAP	(0x10)	select UFO fragmentation for the current frame - only relevant for first buffer in frame, which includes the header only; Ignored if CEDI_TXB_TCP_ENCAP is set.

#define	value	description
CEDI_TXB_TSO_AUTO_SEQ	(0x20)	select auto sequence-number generation, based on the current value of the EMAC sequence number counter for this stream
CEDI_TXB_ENABLE_TLT	(0x40)	Enable Transmit Launch Time.
CEDI_TXD_LMASK	(0x4000 - 1)	Max length accepted by queueTxBuf.
CEDI_TCP_FLG_FIN	(0x0001)	TCP flags (standard, some used to control RSC) 5 REMOVE_ME .
CEDI_TCP_FLG_SYN	(0x0002)	
CEDI_TCP_FLG_RST	(0x0004)	
CEDI_TCP_FLG_PSH	(0x0008)	
CEDI_TCP_FLG_ACK	(0x0010)	
CEDI_TCP_FLG_URG	(0x0020)	
Bit-flags to control MDIO write/read operations.		
CEDI_MDIO_FLG_CLAUSE_45	(0x01)	
CEDI_MDIO_FLG_SET_ADDR	(0x02)	
CEDI_MDIO_FLG_INC_ADDR	(0x04)	
CEDI_QUANTA_PRIORITY_MAX	8	PFC Priority based pause frame support: Maximum number of quantum priorities supported.
Bit-flags for selecting MMSL-related interrupts to enable/disable/read.		
CEDI_MMSL_EV_RCV_R_ERR	(0x01)	
CEDI_MMSL_EV_RCV_V_ERR	(0x02)	
CEDI_MMSL_EV_SMDS_ERR	(0x04)	
CEDI_MMSL_EV_SMD_C_ERR	(0x08)	
CEDI_MMSL_EV_FR_COUNT_ERR	(0x10)	
CEDI_MMSL_EV_SMD_ERR	(0x20)	

Chapter 6. Dynamic Data Structures

6.1. Introduction

This section defines the data structures used by the driver to provide hardware information, modification and dynamic operation of the driver.

These data structures are defined in the header file of the core driver and utilized by the API.

6.2. CEDI_Config

Type: struct

Description

Configuration parameters passed to probe & init.

Note that only the first eight fields are required for the probe call; all are required by init

Fields

regBase	base address of EMAC registers
rxQs	number of Rx priority queues, up to 1 more than highest defined dma_priority_queueN value (for example, =4 if highest is dma_priority_queue3)
txQs	number of Tx priority queues, same limit as for rxQs
txQLen	number of descriptors in each Tx descriptor queue
eTxQLen	number of Tx descriptors in express MAC
rxQLen	number of descriptors in each Rx descriptor queue
eRxQLen	number of Rx descriptors in express MAC
dmaAddrBusWidth	DMA address bus width. 0=32b , 1=64b
enTxExtBD	enable Tx extended BD mode
enRxExtBD	enable Rx extended BD mode
dmaEndianism	sets endian swap for data buffers and descriptors accesses: "OR'd combination of CEDI_END_SWAP_DESC and CEDI_END_SWAP_DATA"
dmaBusWidth	DMA bus width: 32, 64 or 128 bits.
intrEnable	events to enable on start, OR together bit-flags defined in Interrupt control
upper32BuffTxQAddr	Upper 32 bits of physical address of Tx descriptor queues.

	Used only if DMA configured for 64-bit addressing. This is the single upper address used for Tx & Rx descriptors by older MAC h/w, in GEM_GXL before r1p06f2, and XGM_GXL up to r1p01
upper32BufRxQAddr	Upper 32 bits of physical address of Rx descriptor queues.
	Used only if DMA configured for 64-bit addressing
txQAddr	start address of Tx descriptor queues (contiguous block for all queues)
txQPhyAddr	physical address of Tx descriptor queues
rxQAddr	start address of Rx descriptor queues (contiguous block for all queues)
rxQPhyAddr	physical address of Rx descriptor queues
rxBufLength	length of rx buffers, as a multiple of 64 bytes, e.g. 2 = 128 bytes
txPktBufSize	transmit packet buffer size select
rxPktBufSize	receive packet buffer size select
dmaDataBurstLen	fixed burst length for DMA data transfers
dmaCfgFlags	DMA config register bits 24, 25 & 26. OR the following bit-flags to set corresponding bits - CEDI_CFG_DMA_DISC_RXP, CEDI_CFG_DMA_FRCE_RX_BRST, CEDI_CFG_DMA_FRCE_TX_BRST
enableMdio	set to 1 to initialise with MDIO enabled, or 0 to disable
mdcPclkDiv	divisor to generate MDC from pclk
ifTypeSel	MII/GMII/SGMII and speed selection.
altSgmiiEn	enable alternative SGMII mode (auto-negotiation ACK driven differently)
fullDuplex	full/half duplex operation
enRxHalfDupTx	enable Rx in half-duplex while Tx
extAddrMatch	enable external address match interface
rxBufOffset	start-of-frame offset for writing Rx data (0 to 3 bytes)
rxLenErrDisc	enable discard of frames with length field error
disCopyPause	disable copying Rx pause frames to memory
uniDirEnable	enable uni-direction Tx operation
chkSumOffEn	enable checksum offload operation - OR bit-flags to enable in Tx and/or Rx: CEDI_CFG_CHK_OFF_TX, CEDI_CFG_CHK_OFF_RX

<code>rx1536ByteEn</code>	enable Rx of frames up to 1536 bytes
<code>rxJumboFrEn</code>	enable Rx of jumbo frames
<code>enRxBadPreamble</code>	enable Rx frames with non-standard preamble
<code>ignoreIpgRxEr</code>	ignore IPG rx_er (NetCfg b30)
<code>storeUdpTcpOffset</code>	replace CRC upper 16 bits with offset to UDP/TCP header for Rx frames
<code>enExtTsuPort</code>	enable external TSU timer port
<code>aw2wMaxPipeline</code>	Maximum number of outstanding AXI write requests, that can be issued by DMA via the AW channel. Must not be = 0 if using AXI as this would disable writes
<code>ar2rMaxPipeline</code>	Maximum number of outstanding AXI read requests, that can be issued by DMA via the AR channel. Must not be = 0 if using AXI as this would disable reads
<code>pfcMultiQuantum</code>	enable pfc multiple quantum (8 different priorities)
<code>statsRegs</code>	address of CEDI_Statistics struct for driver to fill out: ignored if no_stats = 1
<code>incExpressTraffic</code>	if 1 then Interspersing Express Traffic feature will be handled by driver

6.3. CEDI_SysReq

Type: struct

Description

System requirements returned by probe.

Fields

<code>privDataSize</code>	"size of memory required for driver's private data"
<code>txDescListSize</code>	size of contiguous block of Tx descriptor lists
<code>rxDescListSize</code>	size of contiguous block of Rx descriptor lists
<code>statsSize</code>	size of memory for storing statistics register contents: returned as 0 if no_stats was set to 1

6.4. CEDI_TxStatus

Type: struct

Description

struct for returning contents of the Tx Status register

Fields

usedBitRead
collisionOcc
retryLimExc
txActive
txFrameErr
txComplete
txUnderRun
lateCollision
hRespNotOk
txMacLockup
txDmaLockup

6.5. CEDI_RxStatus

Type: struct

Description

Bit-field struct for returning contents of the Rx Status register.

Fields

buffNotAvail
frameRx
rxOverrun
hRespNotOk
rxMacLockup
rxDmaLockup

6.6. CEDI_BuffAddr

Type: struct

Description

Struct for virtual & physical addresses of DMA-addressable data buffer.

Fields

vAddr

pAddr

6.7. CEDI_TimeStampData

Type: struct

Description

Struct describing time stamp data allocated in descriptor.

Fields

tsValid TS valid - set to 1 if valid data stored.

tsNanoSec Timestamp nanoseconds[29:0] - word2/4[29:0].

tsSecs Timestamp seconds[1:0] from word2/4[31:30], seconds[5:2]from word3/5[5:2].

6.8. CEDI_TransmitLaunchTime

Type: struct

Description

Struct describing time launch time data allocated in descriptor.

It is used to specify on a per-frame basis the TSU time (launch time) at which a frame is to be transmitted

Fields

nanoSecs Time launch time nanoseconds[29:0] - word2/4[29:0].

secs Time launch time seconds[1:0] word2/4[31:30].

6.9. CEDI_TxDescData

Type: struct

Description

struct for returning tx descriptor data

Fields

bufAdd physical & virtual addresses of buffer freed up

`txDescStat` Tx descriptor status word - only valid if first buffer of frame.

`status` descriptor queue status, see `freeTxDesc` for description

`txTsData` Tx descriptor timestamp when valid (`txTsData.tsValid` will be set to 1).

6.10. CEDI_TxDescStat

Type: struct

Description

struct for returning tx descriptor status fields

Fields

`chkOffErr` IP/TCP/UDP checksum offload errors - see `CEDI_TXD_CHKOFF_` constants.

`lateColl` late collision Tx error detected

`frameCorr` DMA frame corruption.

`txUnderrun` Tx Underrun error detected.

`retryExc` Tx error, retries limit exceeded.

6.11. CEDI_RxDescData

Type: struct

Description

struct for returning rx descriptor data

Fields

`rxDescStat` Rx descriptor status word.

`status` Rx data status, see `readRxBuf` for description.

`rxTsData` Rx descriptor timestamp when valid (`rxTsData.tsValid` will be set to 1).

6.12. CEDI_RxDescStat

Type: struct

Description

struct for rx descriptor status fields

Fields

bufLen	valid when eoh (length of header) or eof (total frame length) - includes b13 if not ignore fcs enabled
fcsStatus	ignore if jumbo frames enabled
sof	Start Of Frame in this buffer.
eof	End Of Frame in this buffer.
header	Header-data split: header buffer.
eoh	Header-data split: End Of Header in this buffer.
cfi	Canonical Format Indicator.
vlanPri	VLAN Priority.
priTagDet	Priority tag detected.
vlanTagDet	VLAN tag detected.
typeIdMatch	only set if rx chksum offload disabled
typeMatchReg	Type ID match register if typeIdMatch set (RegNumber-1, i.e. 0 => reg. 1)
snapNoVlanCfi	ignore if rx chksum offload disabled
chkOffStat	chksum offload checking status
specAddMatch	specific address register match (b27 OR b28) If packet buffer mode and extra specific address registers are used, this indicates match in one of the first eight registers
specAddReg	specific addr match register if specAddMatch set (RegNumber-1, i.e. 0 => reg. 1)
extAddrMatch	external address match
uniHashMatch	unicast hash match
multiHashMatch	multicast hash match
broadcast	all-1's broadcast address detected
crc	FCS/CRC error.

6.13. CEDI_qTxBufParams

Type: struct

Description

parameters for qTxBuf function

Fields

queueNum	number of Tx queue
bufAdd	pointer to struct containing physical & virtual addresses of buffer - virtual field is for upper layer use only, can contain other reference/status if required
length	length of data in buffer
flags	bit-flags (CEDI_TXB_xx) specifying last buffer/auto CRC and LSO controls
mssMfs	Segment/fragment size (MSS for TCP or MFS for UDP) - used for all data descriptors.
tcpStream	TCP Stream number, determines which auto-sequence counter to use - only for first (header) descriptor used when flags includes CEDI_TXB_TSO_AUTO_SEQ and CEDI_TXB_TCP_ENCAP.
launchTime	Transmit launch time for start frame.

6.14. CEDI_FrameSize**Type:** struct**Description**

Q buffer sizes.

Fields

FrameSize array of sizes per queue

6.15. CEDI_MacAddress**Type:** struct**Description**

struct for MAC address

Fields

byte

6.16. CEDI_Statistics**Type:** struct**Description**

struct containing all statistics register values (144 bytes long)

Fields

octetsTxLo	b31:0 of octets transmitted
octetsTxHi	b47:32 of octets transmitted
framesTx	
broadcastTx	
multicastTx	
pauseFrTx	
fr64byteTx	
fr65_127byteTx	
fr128_255byteTx	
fr256_511byteTx	
fr512_1023byteTx	
fr1024_1518byteTx	
fr1519_byteTx	
underrunFrTx	
singleCollFrTx	
multiCollFrTx	
excessCollFrTx	
lateCollFrTx	
deferredFrTx	
carrSensErrsTx	
octetsRxLo	b31:0 of octets received
octetsRxHi	b47:32 of octets received
framesRx	
broadcastRx	
multicastRx	
pauseFrRx	
fr64byteRx	

```

fr65_127byteRx
fr128_255byteRx
fr256_511byteRx
fr512_1023byteRx
fr1024_1518byteRx
fr1519_byteRx
undersizeFrRx
oversizeFrRx
jabbersRx
fcsErrorsRx
lenChkErrRx
rxSymbolErrs
alignErrsRx
rxResourceErrs
overrunFrRx
ipChksumErrs
tcpChksumErrs
udpChksumErrs
dmaRxBufFlush

```

6.17. CEDI_NumScreeners

Type: struct

Description

struct for returning number of screening registers

Fields

type1ScrRegs	number of Type1 screener registers
type2ScrRegs	number of Type2 screener registers
ethTypeRegs	number of ethType registers
compareRegs	number of compare registers

6.18. CEDI_T1Screen

Type: struct

Description

struct for writing/reading Type1 screening registers

Fields

qNum the priority queue allocated if match found,

udpEnable if =1, enable UDP port matching

udpPort UDP port number to match if udpEnable=1 (ignored otherwise).

dstcEnable if =1, enable DS/TC field matching

dstcMatch DS field (IPv4) or TC field (IPv6) value to match against.

6.19. CEDI_T2Screen

Type: struct

Description

struct for writing/reading Type2 screening registers

Fields

qNum the priority queue allocated if match found,

vlanEnable if =1, enable VLAN priority matching

vlanPriority VLAN priority to match if vlanEnable=1 (ignored otherwise).

eTypeEnable if =1, enable EtherType field matching

ethTypeIndex Index of EtherType match register: when supported up to:
CEDI_DesignCfg.num_scr2_etype_regs.

compAEnable if =1, enable compare A matching

compAIndex Index of compare register for compare A match.

When supported up to:

compBEnable if =1, enable compare B matching

compBIndex Index of compare register for compare B match.

When supported up to:

`compCEnable` if =1, enable compare C matching

`compCIndex` Index of compare register for compare C match.

When supported up to:

6.20. CEDI_T2Compare

Type: struct

Description

struct for writing/reading screener Type 2 compare registers

Fields

`compMask` Mask Value field (or 1st 16 bits of compare when `disableMask` set).

`compValue` Compare Value field (or 2nd 16 bits of compare when `disableMask` set).

`offsetVal` Offset value.

`offsetPosition` Position in frame to apply offset.

`disableMask` Disable mask value function, to extend compare value to 4 bytes.

Set to 1 to disable mask.

6.21. CEDI_1588TimerVal

Type: struct

Description

struct for writing/reading 1588 timer

Fields

`secsUpper` Upper 16 bits of seconds value.

`secsLower` Lower 32 bits of seconds value.

`nanosecs` Nanoseconds value (30 bits).

6.22. CEDI_TsuTimerVal

Type: struct

Description

struct for writing/reading TSU timer

Fields

`secsUpper` Upper 16 bits of seconds value.
`secsLower` Lower 32 bits of seconds value.
`nanosecs` Upper 22 bits of nanoseconds value.

6.23. CEDI_TimerIncrement

Type: struct

Description

struct for writing/reading the 1588 timer increment registers

Fields

`nanoSecsInc` Whole nanoseconds to increment timer each clock cycle.
`subNsInc` Sub-nanoseconds to increment the timer (16 bits).
`lsbSubNsInc` Lower 8 bits of sub-nanoseconds to increment the timer.
`altIncCount` Number of increments before changing to alternative increment.
 If = 0 then never use alternative increment.
`altNanosInc` Alternative nanoseconds increment to apply.

6.24. CEDI_AnAdvPage

Type: struct

Description

struct for returning contents of the auto-negotiation advertisement register

Fields

`fullDuplex` full duplex capability
`halfDuplex` half duplex capability
`pauseCap` pause capability
`remFlt` remote fault condition
`nextPage` next page exchange required

6.25. CEDI_defLpAbility

Type: struct

Description

struct for default (non-SGMII) auto-negotiation link partner abilities

Fields

`fullDuplex` full duplex capability
`halfDuplex` half duplex capability
`pauseCap` pause capability
`remFlt` remote fault condition
`lpAck` link partner acknowledge indication
`lpNextPage` next page exchange required

6.26. CEDI_sgmLpAbility

Type: struct

Description

struct for SGMII mode auto-negotiation link partner abilities

Fields

`speed` speed, = 10/100/1000Mbps
`duplex` duplex capability, = 0 for half duplex, =1 for full duplex
`lpAck` link partner acknowledge indication
`linkStatus` =0 if link down, =1 if link up

6.27. CEDI_LpAbility

Type: struct

Description

struct of two possible link partner ability structs, one for default configuration and one for SGMII configuration

Fields

`defLpAb1`
`sgmLpAb1`

6.28. CEDI_LpAbilityPage

Type: struct

Description

Struct for link partner ability page info.

If sgmi = 0 then read default ability page struct from ablInfo. If sgmi = 1 then read SGMII mode ability page struct from ablInfo.

Fields

ablInfo

sgmi

6.29. CEDI_AnNextPage

Type: struct

Description

Struct for auto-negotiation next page register data.

Fields

message message data as defined by message page indicator (11 bits)

ack2 acknowledge 2

msgPage message page indicator

np set if another next page to transmit

6.30. CEDI_LpNextPage

Type: struct

Description

struct for auto-negotiation link partner next page data

Fields

message message data as defined by message page indicator (11 bits)

toggle toggles for each received page

ack2 acknowledge 2

msgPage message page indicator

ack indicates if link partner successfully received last message

np set if another next page to transmit

6.31. CEDI_LpPage

Type: struct

Description

struct for link partner page data - ability page or next page

Fields

lpBasePage

lpNextPage

6.32. CEDI_LpPageRx

Type: struct

Description

Struct for returning page data from link partner.

If nextPage = 0 then read link partner base page from lpPage.lpBasePage, else read next page from lpPage.lpNextPage

Fields

lpPageDat

nextPage

6.33. CEDI_NetAnStatus

Type: struct

Description

Struct for returning network status related to auto-negotiation.

Fields

linkState if auto-negotiation enabled: = 1 if LINK up, = 0 if LINK down else synchronisation status

duplexRes if = 1 then both devices full duplex

pauseTxRes enable pause Tx

pauseRxRes enable pause Rx

6.34. CEDI_WakeOnLanReg

Type: struct

Description

Struct for writing/reading the Wake On LAN register.

Fields

wolReqAddr Least significant 16 bits of target IP address to match.

magPktEn Magic packet events cause WOL assert (equal 1 means enabled).

arpEn ARP request events cause WOL assert (equal 1 means enabled).

specAd1En Specific address 1 events cause WOL assert (equal 1 means enabled).

multiHashEn Multicast hash events cause WOL assert (equal 1 means enabled).

6.35. CEDI_LpiStats

Type: struct

Description

Struct for returning the LPI Tx and Rx statistics.

Fields

rxLpiTrans Number of transitions to Rx low power idle.

rxLpiTime Counts time in Rx LPI indication.

txLpiTrans Number of transitions to LPI Tx enable.

txLpiTime Counts time in LPI Tx enable.

6.36. CEDI_EnstTimeConfig

Type: struct

Description

time configuration of Enhancement for Scheduled Traffic

Fields

startTimes time at which queue the queue will start (whole seconds part)

`startTimeNs` time at which queue the queue will start (nanoseconds part)
`onTime` time in which the gate for a queue has to be open, expressed in bytes
`offTime` time in which the gate for a queue has to be closed, expressed in bytes

6.37. CEDI_DesignCfg

Type: struct

Description

Struct containing all design configuration fields, plus some other features which are revision-dependent, e.g.

`intrp_mod`

Fields

`moduleId`
`moduleRev`
`fixNumber`
`numQueues`
`no_pcs`
`serdes`
`RDC_50`
`TDC_50`
`int_loopback`
`no_int_loopback`
`ext_fifo_interface`
`apb_rev1`
`apb_rev2`
`user_io`
`user_out_width`
`user_in_width`
`no_scan_pins`
`no_stats`
`no_snapshot`

irq_read_clear
exclude_cbs
exclude_qbv
num_spec_add_filters
dma_bus_width
axi_cache_value
jumbo_max_length
hprot_value
rx_pkt_buffer
tx_pkt_buffer
rx_pbuf_addr
tx_pbuf_addr
axi
rx_fifo_cnt_width
tx_fifo_cnt_width
tsu
phy_ident
dma_bus_width_def
mdc_clock_div
endian_swap_def
rx_pbuf_size_def
tx_pbuf_size_def
rx_buffer_length_def
tsu_clk
axi_prot_value
tx_pbuf_queue_segment_size
ext_tsu_timer
tx_add_fifo_if
host_if_soft_select

tx_pbuf_num_segments_q0
tx_pbuf_num_segments_q1
tx_pbuf_num_segments_q2
tx_pbuf_num_segments_q3
tx_pbuf_num_segments_q4
tx_pbuf_num_segments_q5
tx_pbuf_num_segments_q6
tx_pbuf_num_segments_q7
dma_addr_width
tx_pbuf_num_segments_q8
tx_pbuf_num_segments_q9
tx_pbuf_num_segments_q10
tx_pbuf_num_segments_q11
tx_pbuf_num_segments_q12
tx_pbuf_num_segments_q13
tx_pbuf_num_segments_q14
tx_pbuf_num_segments_q15
num_type1_screeners
num_type2_screeners
num_scr2_ethtype_regs
num_scr2_compare_regs
axi_access_pipeline_bits
pfc_multi_quantum
pbuf_rsc
pbuf_lso
intrpt_mod
hdr_split
num_cb_streams
lockup_supported

`rx_pbuf_data``tx_pbuf_data`

6.38. CEDI_Callbacks

Type: struct

Description

struct containing function pointers for event notification callbacks issued by isr().

Each call passes the driver's privateData (pD) pointer for instance identification if necessary, and may also pass data related to the event.

Fields

`phyManComplete``txEvent``txError``rxFrame``rxError``hrespError``lpPageRx``anComplete``linkChange``lockupEvent``tsuEvent``pauseEvent``ptpPriFrameTx``ptpPeerFrameTx``ptpPriFrameRx``ptpPeerFrameRx``lpiStatus``wolEvent``extInpIntr`

mmslEvent

6.39. CEDI_QueueSegAlloc

Type: struct

Description

Struct contains configuration of SRAM memory distribution across the priority queues.

It defines how many segments will be distributed to each queue.

Fields

segAlloc	Array with segments allocation settings. Element 0 of array defines number of segments allocated to queue 0 element 1 of array defines number of segments allocated to queue 1, etc.
segAllocCount	Number of elements in SegmentsPerQueue size. It should be the same as number of active queues.

6.40. CEDI_FrameEliminationConfig

Type: struct

Description

Frame elimination configuration.

Fields

enVectAlg	if 1 enable vector recovery algorithm
seqNumIdentification	selects what should be used to identify sequence number
offsetValue	Offset value in bytes from the start packet delimiter to the most significant byte of the 802.1CB sequence number. (0-511)
smemberStream1	first screener type 2 register used for member stream identification
smemberStream2	second screener type 2 register used for member stream identification
seqNumLength	number of significant bits of 802.1CB sequence number. (1-15)
seqRecWindow	Vector recovery window, defines window size used by the vector recovery algorithm to determine whether to reject a packet.

6.41. CEDI_FrameEliminationTagConfig

Type: struct

Description

Frame elimination tag configuration.

Fields

redundancyTag Ethertype value used to identify the redundancy tag.

enStripRedTag If 1 the redundancy tag is stripped from receive frames.

6.42. CEDI_FrameEliminationStats

Type: struct

Description

Frame elimination statistics.

Fields

latentErrors Count of sequence numbers seen without a duplicate.

vecRecRogue Count of number of frames dropped by the vector recovery algorithm for being out of range.

outOfOrder Count of out of order sequence numbers received.

seqRstCount count of number of times the sequence recovery reset timer decrements to zero

6.43. CEDI_PreemptionConfig

Type: struct

Description

Configuration of preemption feature.

Fields

routeRxToPmac route all received traffic to the pmac rather than the emac when pre_active is 0 in the MMSL status.

enPreemption enabling/disabling the pre-emption operation.

enVerify enabling/disabling the verification procedure which will determine if the link partner can support 802.3br

addFragSize minimum number of bytes which the pMAC has to send before the pre-emption will be allowed.

$\text{min_frag_size} = (64 * (1 + \text{addFragSize}) - 4)$

6.44. CEDI_MmslStats

Type: struct

Description

mmsl statistics

Fields

assErrorCount A count of MAC frames with reassembly errors.

smdErrorCount A count of received MAC frames / MAC frame fragments rejected due to unknown SMD value or arriving with an SMD-C when no frame is in progress.

assOkCount A count of MAC frames that were successfully reassembled and delivered to MAC.

fragCountRx A count of the number of additional mPackets received due to preemption.

fragCountTx A count of the number of additional mPackets transmitted due to preemption.

6.45. CEDI_MmslStatus

Type: struct

Description

mmsl status

Fields

preActive Set to one if pre-emption capability is active.

respondStatus Return state of the Respond state machine.

verifyStatus Return state of the verification state machine.

events MMSL interrupt flags.

6.46. CEDI_LockupConfig

Type: struct

Description

Lockup detection and recovery configuration.

Fields

<code>prescaler</code>	defines a prescaler value which is the number of <code>tx_clk</code> periods used to scale the <code>dmaLockupTime</code> , <code>txLockupTime</code> and <code>rxLockupTime</code>
<code>dmaLockupTime</code>	Time after which transmit and receive DMA lockup detection.
<code>txLockupTime</code>	The time after which the transmit MAC lockup detection monitor will trigger.
<code>rxLockupTime</code>	The time after which the receive MAC lockup detection monitor will trigger.
<code>enLockupRecovery</code>	Lockup recovery. If this bit is set then the module will go into a soft reset state if a lockup is detected.
<code>enRxMacLockupMon</code>	Enable the RX MAC lockup detector. Enable the monitor that detects lockups in the MAC receive path.
<code>enRxDmaLockupMon</code>	Enable the RX DMA lockup detector. Enable the monitor that detects lockups in the receive DMA.
<code>enTxMacLockupMon</code>	Enable the RX MAC lockup detector. Enable the monitor that detects lockups in the MAC transmit path.
<code>enTxDmaLockupMon</code>	Enable the TX DMA lockup detector. Enable the monitor that detects lockups in the transmit DMA.
<code>enTxDmaTimers</code>	ORed combination of bit flags DMA TX lockup timer for each queue (bit 0 - timer for queue 0, bit 15 - timer for queue 15). Set bit means that timer is enabled.

6.47. CEDI_RxQFlushConfig

Type: struct

Description

Receive queue flushing configuration.

Fields

<code>dropAllFrames</code>	If set to 1, all frames for this queue will be dropped.
<code>dropOnRsrcErr</code>	If set to 1, and free DMA descriptor for queue cannot be obtained, frames will be dropped.
<code>rxQFlushMode</code>	Used to select queue flushing mode using limits.
<code>maxVal</code>	Used for receive queue flushing modes using limits.

6.48. CEDI_Type2ScreenerRateLimit

Type: struct

Description

Used to configure max receive rate for type 2 screening register.

Fields

`intervalTime` Interval time at which number of received bytes is compared against `max_rate_val`.

`maxRateVal` Maximum number of bytes screener is permitted to match in interval time.

6.49. CEDI_AxiQosConfig

Type: struct

Description

AXI quality of service configuration.

Fields

`descriptorQos` Determines what value to drive to the ARQOS and AWQOS outputs (4-bits each) for queue 0 descriptor accesses.

`dataQos` Determines what value to drive to the ARQOS and AWQOS outputs (4-bits each) for queue 0 data accesses.

6.50. CEDI_AsfInfo

Type: struct

Description

Information about ASF in GEM controller.

There could be 0, 1 or two instances ASF componets. 0 - if ASF is not included to current configuration 1 - if ASF is included but `gem_has_802p3_br` hardware configuration option is disabled 2 - if ASF is included and `gem_has_802p3_br` hardware configuration option is enabled

Fields

`asfCount` Number of ASF instances, 0 - means that there is no ASF.

`asfRegBases` ASF register start addresses.

6.51. CEDI_BusWidth

Type: enum

Values

CEDI_DMA_BUS_WIDTH_32 Value: 0

CEDI_DMA_BUS_WIDTH_64 Value: 1

CEDI_DMA_BUS_WIDTH_128 Value: 2

6.52. CEDI_DmaDatBLen**Type:** enum**Values**

CEDI_DMA_DBUR_LEN_1 Value: 0

CEDI_DMA_DBUR_LEN_4 Value: 1

CEDI_DMA_DBUR_LEN_8 Value: 2

CEDI_DMA_DBUR_LEN_16 Value: 3

6.53. CEDI_MdcClkDiv**Type:** enum**Values**

CEDI_MDC_DIV_BY_8 Value: 0

CEDI_MDC_DIV_BY_16 Value: 1

CEDI_MDC_DIV_BY_32 Value: 2

CEDI_MDC_DIV_BY_48 Value: 3

CEDI_MDC_DIV_BY_64 Value: 4

CEDI_MDC_DIV_BY_96 Value: 5

CEDI_MDC_DIV_BY_128 Value: 6

CEDI_MDC_DIV_BY_224 Value: 7

6.54. CEDI_MdioDevType**Type:** enum**Values**

CEDI_MDIO_DEV_PMD_PMA Value: 1

CEDI_MDIO_DEV_WIS Value: 2

CEDI_MDIO_DEV_PCS Value: 3

CEDI_MDIO_DEV_PHY_XS Value: 4

CEDI_MDIO_DEV_DTE_XS Value: 5

6.55. CEDI_Interface

Type: enum

Values

CEDI_IFSP_10M_MII Value: 0

CEDI_IFSP_100M_MII Value: 1

CEDI_IFSP_1000M_GMII Value: 2

CEDI_IFSP_10M_SGMII Value: 3

CEDI_IFSP_100M_SGMII Value: 4

CEDI_IFSP_1000M_SGMII Value: 5

CEDI_IFSP_1000BASE_X Value: 6

CEDI_IFSP_2500M_SGMII Value: 7

CEDI_IFSP_2500BASE_X Value: 8

6.56. CEDI_TxTsMode

Type: enum

Values

CEDI_TX_TS_DISABLED Value: 0

CEDI_TX_TS_PTP_EVENT_ONLY
Value: 1

CEDI_TX_TS_PTP_ALL Value: 2

CEDI_TX_TS_ALL Value: 3

6.57. CEDI_RxTsMode

Type: enum

Values

CEDI_RX_TS_DISABLED Value: 0

CEDI_RX_TS_PTP_EVENT_ONLY

Value: 1

CEDI_RX_TS_PTP_ALL Value: 2

CEDI_RX_TS_ALL Value: 3

6.58. CEDI_LinkFaultIndication

Type: enum

Values

CEDI_LINK_FAULT_INDICATION_OK

Value: 0

CEDI_LINK_FAULT_INDICATION_LOCAL_FAULT

Value: 1

CEDI_LINK_FAULT_INDICATION_REMOTE_FAULT

Value: 2

CEDI_LINK_FAULT_INDICATION_LINK_INTERRUPT

Value: 3

6.59. CEDI_RedundancyTagSize

Type: enum

Values

CEDI_REDUNDANCY_TAG_SIZE_4

Value: 0

CEDI_REDUNDANCY_TAG_SIZE_6

Value: 1

6.60. CEDI_IfSpeed

Type: enum

Description

Enum defining speed selection or indication.

Values

CEDI_SPEED_10M Value: 0

CEDI_SPEED_100M Value: 1

CEDI_SPEED_1000M Value: 2

CEDI_SPEED_2500M Value: 3

6.61. CEDI_TxFreeDStat

Type: enum

Description

Enum defining Tx free descriptor call status.

Values

CEDI_TXDATA_1ST_NOT_LAST Value: 0

CEDI_TXDATA_1ST_AND_LAST Value: 1

CEDI_TXDATA_MID_BUFFER Value: 2

CEDI_TXDATA_LAST_BUFFER Value: 3

CEDI_TXDATA_NONE_FREED Value: 4

6.62. CEDI_RxRdStat

Type: enum

Description

Enum defining Rx data buffer read status.

Values

CEDI_RXDATA_SOF_EOF Value: 0

CEDI_RXDATA_SOF_ONLY Value: 1

CEDI_RXDATA_NO_FLAG Value: 2

CEDI_RXDATA_EOF_ONLY Value: 3

CEDI_RXDATA_NODATA Value: 4

6.63. CEDI_T2Offset

Type: enum

Description

enum defining offset for screener type 2 compare values.

Note the offset is applied after the specified point, e.g. T2COMP_OFF_ETYPE denotes the ethertype field, so an offset of 12 bytes from this would be the source IP address in an IP header.

Values

CEDI_T2COMP_OFF_SOF	Value: 0
CEDI_T2COMP_OFF_ETYPE	Value: 1
CEDI_T2COMP_OFF_IPHDR	Value: 2
CEDI_T2COMP_OFF_TCPUDP	Value: 3
CEDI_T2COMP_OFF_VID_C_TAG	Value: 4
CEDI_T2COMP_OFF_VID_S_TAG	Value: 5

6.64. CEDI_SegmentsPerQueue

Type: enum

Description

enum defines number of segments of SRAM memory per TX queue

Values

CEDI_SEGMENTS_PER_QUEUE_1	Value: 0
CEDI_SEGMENTS_PER_QUEUE_2	Value: 1
CEDI_SEGMENTS_PER_QUEUE_4	Value: 2
CEDI_SEGMENTS_PER_QUEUE_8	Value: 3
CEDI_SEGMENTS_PER_QUEUE_16	Value: 4
CEDI_SEGMENTS_PER_QUEUE_32	Value: 5
CEDI_SEGMENTS_PER_QUEUE_64	

Value: 6

CEDI_SEGMENTS_PER_QUEUE_128

Value: 7

6.65. CEDI_TxSchedType

Type: enum

Description

Type of scheduling algorithm of transmit queue.

Values

CEDI_TX_SCHED_TYPE_FIXED Value: 0

CEDI_TX_SCHED_TYPE_CBS Value: 1

CEDI_TX_SCHED_TYPE_DWRR Value: 2

CEDI_TX_SCHED_TYPE_ETS Value: 3

6.66. CEDI_PauseCap

Type: enum

Description

enum defining pause capability in auto-negotiation

Values

CEDI_AN_PAUSE_CAP_NONE Value: 0

CEDI_AN_PAUSE_CAP_SYM Value: 1

CEDI_AN_PAUSE_CAP_ASYM Value: 2

CEDI_AN_PAUSE_CAP_BOTH Value: 3

6.67. CEDI_LoopbackMode

Type: enum

Description

enum defining loopback mode selection

Values

CEDI_NO_LOOPBACK Value: 0

CEDI_LOCAL_LOOPBACK Value: 1

CEDI_SERDES_LOOPBACK Value: 2

6.68. CEDI_RemoteFault

Type: enum

Description

enum defining remote fault indication by link partner in auto-negotiation

Values

CEDI_AN_REM_FLT_NONE Value: 0

CEDI_AN_REM_FLT_LNK_FAIL Value: 1

CEDI_AN_REM_FLT_OFFLINE Value: 2

CEDI_AN_REM_FLT_AN_ERR Value: 3

6.69. CEDI_MacType

Type: enum

Description

MAC object type.

Values

CEDI_MAC_TYPE_EMAC express MAC

Value: 0

CEDI_MAC_TYPE_PMAC preemptive MAC

Value: 1

CEDI_MAC_TYPE_MAC normal MAC(mac (when Express traffic is not supported or unwanted)

Value: 2

6.70. CEDI_SeqNumIdentification

Type: enum

Description

defines what should be used to identify sequence number, redundancy tag or value at specific offset

Values

CEDI_SEQ_NUM_IDEN_OFFSET Value: 0

CEDI_SEQ_NUM_IDEN_TAG Value: 1

6.71. CEDI_FragSize

Type: enum

Description

defines possible values of add frag size which is number of bytes which the pMAC has to send before the pre-emption will be allowed

Values

CEDI_FRAG_SIZE_64 64 bytes

Value: 0

CEDI_FRAG_SIZE_128 128 bytes

Value: 1

CEDI_FRAG_SIZE_192 192 bytes

Value: 2

CEDI_FRAG_SIZE_256 256 bytes

Value: 3

6.72. CEDI_RespondStatus

Type: enum

Description

defines possible values of return states of the respond state machine.

Values

CEDI_R_IDLE Value: 0

CEDI_SEND_RESPOND Value: 1

6.73. CEDI_VerifyStatus

Type: enum

Description

defines possible values of return states of the verification state machine.

Values

CEDI_INIT_VERIFICATION Value: 0

CEDI_VERIFICATION_IDLE Value: 1

CEDI_SEND_VERIFY Value: 2

CEDI_WAIT_FOR_RESPONSE Value: 3

CEDI_VERIFIED Value: 4

CEDI_VERIFY_FAIL Value: 5

6.74. CEDI_RxQFlushMode

Type: enum

Description

Defines possible modes of receive queue flushing.

Values

CEDI_FLUSH_MODE_OFF Disables Rx flushing based on limits.
Value: 0

CEDI_FLUSH_MODE_LIMIT_NUM_BYTES
Mode depending on received bytes.
Value: 1

CEDI_FLUSH_MODE_LIMIT_FRAME_SIZE
Mode depending on received frames size.
Value: 2

6.75. CEDI_CbPhyManComplete

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CEDI_CbPhyManComplete )(struct CEDI_PrivateData *pD, uint8_t read,
uint16_t readData)
```

Description

Reports PHY Management frame tx complete.

Params: read = 1 if a read operation, = 0 if a write operation, and readData - data from the PHY, if read operation.

6.76. CEDI_CbTxEvent

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CEDI_CbTxEvent )(struct CEDI_PrivateData *pD, uint32_t event, uint8_t
queueNum)
```

Description

Reports Tx completed / Tx used bit read events.

Params: event - indicates two types of event, which are OR'd together if both occurred (CEDI_EV_TX_COMPLETE: frame has been transmitted successfully, and CEDI_EV_TX_USED_READ: used bit set has been read in Tx descriptor list), and queueNum- the priority queue number (only relevant to TX_COMPLETE) range 0 to txQs-1.

6.77. CEDI_CbTxError

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CEDI_CbTxError )(struct CEDI_PrivateData *pD, uint32_t error, uint8_t
queueNum)
```

Description

Reports Tx errors.

Params: error - indicates one or more of three possible events, OR'd together (CEDI_EV_TX_UNDERRUN: Tx underrun, CEDI_EV_TX_RETRY_EX_LATE_COLL: retry limit exceeded or late collision, CEDI_EV_TX_FR_CORRUPT: Tx frame corruption), and queueNum - the priority queue number, range 0 to txQs-1 (not relevant to TX_UNDERRUN).

6.78. CEDI_CbRxFrame

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CEDI_CbRxFrame )(struct CEDI_PrivateData *pD, uint8_t queueNum)
```

Description

Reports Frame Rx completed successfully.

Param: queueNum - the priority queue number, range 0 to rxQs-1.

6.79. CEDI_CbRxError

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CEDI_CbRxError )(struct CEDI_PrivateData *pD, uint32_t error, uint8_t queueNum)
```

Description

Reports Rx errors.

Params: error - one or more of two possible errors, together (CEDI_EV_RX_USED_READ: used bit set has been read in Rx descriptor list; CEDI_EV_RX_OVERRUN: Rx overrun error), and queueNum - the priority queue number, range 0 to rxQs-1.

6.80. CEDI_CbHrespError

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CEDI_CbHrespError )(struct CEDI_PrivateData *pD, uint8_t queueNum)
```

Description

Reports Hresp not OK error.

Param: queueNum - number of the Tx or Rx queue being accessed (range 0 to rxQs-1).

6.81. CEDI_CbLpPageRx

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CEDI_CbLpPageRx )(struct CEDI_PrivateData *pD, struct CEDI_LpPageRx
*pageRx)
```

Description

Reports PCS auto-negotiation page received.

Param: pageRx - pointer to a struct containing the link partner base or next page data.

6.82. CEDI_CbAnComplete

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CEDI_CbAnComplete )(struct CEDI_PrivateData *pD, struct CEDI_NetAnStatus
*netStat)
```

Description

Reports PCS auto-negotiation completed.

Param: netStat - pointer to a struct with the network resolution status.

6.83. CEDI_CbLinkChange

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CEDI_CbLinkChange )(struct CEDI_PrivateData *pD, uint8_t linkState)
```

Description

Reports Link Status Changed.

Param: linkState - link status: if auto-negotiation enabled, when =1 link is up, else link is down; if auto-negotiation not enabled, this will indicate the synchronisation status. If link status has gone down, the value will not return to 1 until after it has been read, therefore to see current state must re-read with a `getLinkStatus` call.

6.84. CEDI_CbLockupEvent

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CEDI_CbLockupEvent )(struct CEDI_PrivateData *pD, uint32_t event)
```

Description

Reports Lockup Event occurred.

Param: event - indicates one of two events, OR'd together (CEDI_EV_RX_LOCKUP: RX lockup detected; CEDI_EV_TX_LOCKUP: TX lockup detected).

6.85. CEDI_CbTsuEvent

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CEDI_CbTsuEvent )(struct CEDI_PrivateData *pD, uint32_t event)
```

Description

Reports TSU Event occurred.

Param: event - indicates one of two events, OR'd together (CEDI_EV_TSU_SEC_INC: TSU seconds register increment; CEDI_EV_TSU_TIME_MATCH: TSU timer count match).

6.86. CEDI_CbPauseEvent

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CEDI_CbPauseEvent )(struct CEDI_PrivateData *pD, uint32_t event)
```

Description

Reports Pause Event occurred.

Param: event - indicates one or more of three event types, OR'd together (CEDI_EV_PAUSE_FRAME_TX: Pause frame transmitted; CEDI_EV_PAUSE_TIME_ZERO: Pause time zero; CEDI_EV_PAUSE_NZ_QU_RX: Pause frame with non-zero quantum received).

6.87. CEDI_CbPtpPriFrameTx

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CEDI_CbPtpPriFrameTx )(struct CEDI_PrivateData *pD, uint32_t type, struct CEDI_1588TimerVal *timeVal)
```

Description

Reports PTP Primary Frame transmitted.

Params: type - indicates one of two frame types, OR'd together (CEDI_EV_PTP_TX_DLY_REQ: delay_req; CEDI_EV_PTP_TX_SYNC: sync) , and time: pointer to a struct containing the time latched when the frame was transmitted. This is not set if IP config param tsu=0.

6.88. CEDI_CbPtpPeerFrameTx

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CEDI_CbPtpPeerFrameTx )(struct CEDI_PrivateData *pD, uint32_t type, struct CEDI_1588TimerVal *timeVal)
```

Description

Reports PTP Peer Frame transmitted.

Params: type - indicates one of two frame types, OR'd together (CEDI_EV_PTP_TX_PDLY_REQ: pdelay_req; CEDI_EV_PTP_TX_PDLY_RSP: pdelay_rsp) , and time: pointer to a struct containing the time latched when the frame was transmitted. This is not set if IP config param tsu=0.

6.89. CEDI_CbPtpPriFrameRx

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CEDI_CbPtpPriFrameRx )(struct CEDI_PrivateData *pD, uint32_t type, struct CEDI_1588TimerVal *timeVal)
```

Description

Reports PTP Primary Frame received.

Params: type - indicates one of two frame types, OR'd together (CEDI_EV_PTP_RX_DLY_REQ: delay_req; CEDI_EV_PTP_RX_SYNC: sync) , and time: pointer to a struct containing the time latched when the frame was received. This is not set if IP config param tsu=0.

6.90. CEDI_CbPtpPeerFrameRx

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CEDI_CbPtpPeerFrameRx )(struct CEDI_PrivateData *pD, uint32_t type,
struct CEDI_1588TimerVal *timeVal)
```

Description

Reports PTP Peer Frame received.

Params: type - indicates one of two frame types, OR together (CEDI_EV_PTP_RX_PDLY_REQ: pdelay_req; CEDI_EV_PTP_RX_PDLY_RSP: pdelay_rsp) , and time: pointer to a struct containing the time latched when the frame was received. This is not set if IP config param tsu=0.

6.91. CEDI_CbLpiStatus

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CEDI_CbLpiStatus )(struct CEDI_PrivateData *pD)
```

Description

Reports LPI Status changed.

6.92. CEDI_CbWoLEvent

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CEDI_CbWoLEvent )(struct CEDI_PrivateData *pD)
```

Description

Reports Wake On LAN event received.

6.93. CEDI_CbExtInpIntr

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CEDI_CbExtInpIntr )(struct CEDI_PrivateData *pD)
```

Description

Reports External input interrupt occurred.

6.94. CEDI_CbMmslEvent

Type: typedef

This is a callback event type. A function with matching prototype needs to be registered with the `callbacks` structure at `init` to enable event signaling.

Function Declaration

```
void(* CEDI_CbMmslEvent )(struct CEDI_PrivateData *pD, uint32_t events)
```

Description

Callback function for MMSL-related interrupts.

Chapter 7. Driver API Object

7.1. Introduction

API listing for the driver.

The API is contained in the object as function pointers in the object structure. As the actual functions resides in the Driver Object, the client software must first use the global `GetInstance` function to obtain the Driver Object Pointer. The actual APIs then can be invoked using `obj->(api_name)()` syntax. These functions are defined in the header file of the core driver and utilized by the API.

7.2. CEDI_GetInstance

Function Declaration

```
CEDI_OBJ* CEDI_GetInstance();
```

```
void
```

Description

In order to access the CEDI APIs, the upper layer software must call this global function to obtain the pointer to the driver object.

Return Value

CEDI_OBJ* Driver Object Pointer

CEDI_OBJ

7.3. probe

Function Declaration

```
uint32_t probe(config, memReq);
```

```
CEDI_Config* config;
```

```
CEDI_SysReq* memReq;
```

Parameter List

`memReq` [out]

returns the size of memory allocations required

`config` [in]

driver/hardware configuration required

Description

Get the driver's memory requirements to support the given configuration.

If config->txQs or config->rxQs specify more than the number of queues available in the h/w configuration, they will be reduced to match the latter value, and the memReq values will be based on this.

Return Value

EOK on success (requirements struct filled)

ENOTSUP if configuration cannot be supported due to driver/hardware constraints

EINVAL if config is NULL, or hardware not present

7.4. init

Function Declaration

```
uint32_t init(pD, config, callbacks);

CEDI_PrivateData* pD;
const CEDI_Config* config;
const CEDI_Callbacks* callbacks;
```

Parameter List

callbacks	[in]	client-supplied callback functions
config	[inout]	specifies driver/hardware configuration
pD	[in]	driver state info specific to this instance

Description

Initialise the driver instance and state, configure the EMAC as specified in the 'config' settings, set up Tx & Rx descriptor lists.

Return Value

EOK on success

ENOTSUP if hardware has an inconsistent configuration or doesn't support feature(s) required by 'config' parameters

EIO if driver encountered an error accessing hardware

EINVAL if illegal/inconsistent values in 'config'

7.5. destroy

Function Declaration

```
void destroy(pD);
```



```
CEDI_PrivateData* pD;
```

Parameter List

pD [in]

driver state info specific to this instance

Description

Destroy the driver (automatically performs a stop).

7.6. start

Function Declaration

```
void start(pD);
```

```
CEDI_PrivateData* pD;
```

Parameter List

pD [in]

driver state info specific to this instance

Description

Start the EMAC driver, enabling interrupts and PCS auto- negotiation.

7.7. stop

Function Declaration

```
void stop(pD);
```

```
CEDI_PrivateData* pD;
```

Parameter List

pD [in]

driver state info specific to this instance

Description

Stop the driver.

This should disable the hardware, including its interrupt at the source, and make a best-effort to cancel any pending transactions.

7.8. isr

Function Declaration

```
uint32_t isr(pD);
CEDI_PrivateData* pD;
```

Parameter List

pD [in]
driver state info specific to this instance

Description

Driver ISR.

Platform-specific code is responsible for ensuring this gets called when the corresponding hardware's interrupt is asserted. Registering the ISR should be done after calling init, and before calling start. The driver's ISR will not attempt to lock any locks, but will perform client callbacks. If the client wishes to defer processing to non-interrupt time, it is responsible for doing so.

Return Value

EOK if any interrupt detected (and callback)

ECANCELED if no interrupt bit detected

EINVAL for invalid *pD* pointer

7.9. setEventEnable

Function Declaration

```
uint32_t setEventEnable(pD, events, enable, queueNum);
CEDI_PrivateData* pD;
uint32_t events;
uint8_t enable;
uint8_t queueNum;
```

Parameter List

enable [in]
if =1 enable the events, if =0 then disable

events [in]
OR'd combination of bit-flags selecting the events to be enabled or disabled

pD [in]

driver private state info specific to this instance

queueNum [in]

between 0 and config->rxQs-1, or = CEDI_ALL_QUEUES number of Tx or Rx priority queue, relevant to some of Tx and Rx events: CEDI_EV_TX_COMPLETE, CEDI_EV_TX_RETRY_EX_LATE_COLL, CEDI_EV_TX_FR_CORRUPT, CEDI_EV_RX_COMPLETE, CEDI_EV_RX_USED_READ, CEDI_EV_RX_OVERRUN, CEDI_EV_HRESP_NOT_OK. Must be = 0 or CEDI_ALL_QUEUES for other events. To dis/enable on all available Qs, set queueNum to CEDI_ALL_QUEUES and set events to CEDI_EVSET_ALL_Q0_EVENTS.

Description

Enable or disable the specified interrupts.

mechanical test will always fail as no callback pointers are given

Return Value

EOK for success

EINVAL for invalid queueNum

7.10. getEventEnable

Function Declaration

```
uint32_t getEventEnable(pD, queueNum, event);
```

```
CEDI_PrivateData* pD;
uint8_t queueNum;
uint32_t* event;
```

Parameter List

event [out]

returned enabled events

pD [in]

driver private state info specific to this instance

queueNum [in]

Tx or Rx priority queue to report events for

Description

Read the enabled state of the specified interrupts.

OR the returned event value with the CEDI_EV_ event bit-flags to determine if event(s) are enabled.

Return Value

EOK for success

EINVAL for invalid pointer or queueNum

7.11. setIntrptModerate

Function Declaration

```
uint32_t setIntrptModerate(pD, txIntDelay, rxIntDelay);
```

```
CEDI_PrivateData* pD;  
uint8_t txIntDelay;  
uint8_t rxIntDelay;
```

Parameter List

txIntDelay [in]

Interrupt delay to apply to Tx frame complete

rxIntDelay [in]

Interrupt delay to apply to Rx frame complete

pD [in]

driver private state info specific to this instance

Description

Set the value of the Tx & Rx interrupt moderation register fields.

A non-zero value in either field introduces an interrupt stand-off delay of the 8-bit value in units of 800ns before the corresponding frame complete event causes an interrupt, limiting the interrupt rate.

Return Value

EOK for success

EINVAL for invalid pointer

7.12. getIntrptModerate

Function Declaration

```
uint32_t getIntrptModerate(pD, txIntDelay, rxIntDelay);
```

```
CEDI_PrivateData* pD;  
uint8_t* txIntDelay;  
uint8_t* rxIntDelay;
```

Parameter List

<code>txIntDelay</code>	[out]	Interrupt delay to apply to Tx frame complete
<code>rxIntDelay</code>	[out]	Interrupt delay to apply to Rx frame complete
<code>pD</code>	[in]	driver private state info specific to this instance

Description

Read the values of the interrupt moderation register fields.

Return Value

EOK for success

EINVAL for invalid pointers

7.13. setIntrptModerateThreshold

Function Declaration

```
uint32_t      setIntrptModerateThreshold(pD,          txIntFrameThreshold,
rxIntFrameThreshold);

CEDI_PrivateData* pD;
uint8_t txIntFrameThreshold;
uint8_t rxIntFrameThreshold;
```

Parameter List

<code>txIntFrameThreshold</code>	[in]	Count of frames to transmit before asserting interrupt.
<code>rxIntFrameThreshold</code>	[in]	Count of frames to receive before asserting interrupt.
<code>pD</code>	[in]	driver private state info specific to this instance

Description

Set the value of the Tx & Rx frame threshold interrupt moderation register fields.

A non-zero value in either field introduces an interrupt stand-off delay until programmed number of frames is transmitted/received.

Return Value

EOK for success

ENOTSUP if feature is not available

EINVAL for invalid pointer

7.14. getIntrptModerateThreshold

Function Declaration

```
uint32_t      getIntrptModerateThreshold(pD,          txIntFrameThreshold,
rxIntFrameThreshold);
```

```
CEDI_PrivateData* pD;
uint8_t* txIntFrameThreshold;
uint8_t* rxIntFrameThreshold;
```

Parameter List

<i>txIntFrameThreshold</i>	[out]	Count of frames to transmit before asserting interrupt.
<i>rxIntFrameThreshold</i>	[out]	Count of frames to receive before asserting interrupt.
<i>pD</i>	[in]	driver private state info specific to this instance

Description

Read the value of the Tx & Rx frame threshold interrupt moderation register fields.

Return Value

EOK for success

ENOTSUP if feature is not available

EINVAL for invalid pointers

7.15. setIfSpeed

Function Declaration

```
uint32_t setIfSpeed(pD, speedSel);
```

```
CEDI_PrivateData* pD;  
CEDI_IfSpeed speedSel;
```

Parameter List

speedSel [in]
a CEDI_IfSpeed enum indicating Tx/Rx speed

pD [in]
driver private state info specific to this instance

Description

Select 10/100/1000/2500Mbps operation speed.

Return Value

EOK if successful

EINVAL if invalid parameter

7.16. getIfSpeed

Function Declaration

```
uint32_t getIfSpeed(pD, speedSel);  
  
CEDI_PrivateData* pD;  
CEDI_IfSpeed* speedSel;
```

Parameter List

speedSel [out]
returns CEDI_IfSpeed enum indicating Tx/Rx speed

pD [in]
driver private state info specific to this instance

Description

Read selected operation speed.

Return Value

EOK for success

EINVAL if invalid parameter

7.17. setJumboFramesRx

Function Declaration

```
void setJumboFramesRx(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t enable;
```

Parameter List

enable [in]

if =1 then enable jumbo frames accept, if =0 disable.

pD [in]

driver private state info specific to this instance

Description

Enable/disable reception of jumbo frames.

7.18. getJumboFramesRx

Function Declaration

```
uint32_t getJumboFramesRx(pD, enable);
```

```
const CEDI_PrivateData* pD;  
uint8_t* enable;
```

Parameter List

enable [out]

equal 1 if jumbo frames accept enabled; equal 0 if disabled.

pD [in]

driver private state info specific to this instance

Description

Read jumbo frames enable status.

Return Value

EOK for success

EINVAL for invalid pointer

7.19. setJumboFrameRxMaxLen

Function Declaration

```
uint32_t setJumboFrameRxMaxLen(pD, length);
```

```
CEDI_PrivateData* pD;  
uint16_t length;
```

Parameter List

length [in]
 max. length to receive, in bytes. Maximum is 16383

pD [in]
 driver private state info specific to this instance

Description

Set maximum length of jumbo frames to be received.

(This is initialised to gem_jumbo_max_length bytes.) Disables jumbo frame reception temporarily while changing max length.

Return Value

EOK if successful

EINVAL if length greater than 16383

7.20. getJumboFrameRxMaxLen

Function Declaration

```
uint32_t getJumboFrameRxMaxLen(pD, length);
```

```
CEDI_PrivateData* pD;  
uint16_t* length;
```

Parameter List

length [out]
 returns max. length to receive, in bytes

pD [in]
 driver private state info specific to this instance

Description

Read maximum length of jumbo frames to be received.

Return Value

EOK for success

EINVAL for invalid pointer

7.21. setUniDirEnable

Function Declaration

```
uint32_t setUniDirEnable(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t enable;
```

Parameter List

enable [in]

if =1 enable, if =0 then disable

pD [in]

driver private state info specific to this instance

Description

Enable/disable uni-direction transmit operation.

Return Value

EOK for success

EINVAL if invalid parameter

7.22. getUniDirEnable

Function Declaration

```
uint32_t getUniDirEnable(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t* enable;
```

Parameter List

enable [out]

equal 1 if enabled, equal 0 if disabled

pD [in]

driver private state info specific to this instance

Description

Read enable status for uni-direction transmit operation.

Return Value

EOK for success

EINVAL for invalid pointer

7.23. setTxChecksumOffload

Function Declaration

```
uint32_t setTxChecksumOffload(pD, enable);
```

CEDI_PrivateData* pD;

uint8_t enable;

Parameter List

enable [in]

if =1 then enable, if =0 then disable

pD [in]

driver private state info specific to this instance

Description

Enable/disable Tx IP, TCP and UDP checksum generation offload.

Only valid if using DMA packet buffering mode.

Return Value

EOK for success

EINVAL if packet buffering not enabled or null pointer

7.24. getTxChecksumOffload

Function Declaration

```
uint32_t getTxChecksumOffload(pD, enable);
```

CEDI_PrivateData* pD;

uint8_t* enable;

Parameter List

enable [out]

equal 1 if enabled, equal 0 if disabled

pD [in]

driver private state info specific to this instance

Description

Read enable/disable of Tx IP, TCP and UDP checksum generation offload.

Return Value

EOK for success

EINVAL for invalid pointer

7.25. setRxBufOffset

Function Declaration

```
uint32_t setRxBufOffset(pD, offset);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t offset;
```

Parameter List

pD [in]

driver private state info specific to this instance

offset [in]

offset to use, range 0 to 3 bytes

Description

Set Rx buffer offset for writing start of frame.

Return Value

EOK for success

EINVAL if offset invalid or null pointer

7.26. getRxBufOffset

Function Declaration

```
uint32_t getRxBufOffset(pD, offset);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t* offset;
```

Parameter List

pD [in]
driver private state info specific to this instance

offset [out]
offset in use, range 0 to 3 bytes

Description

Read Rx buffer offset for writing start of frame.

Return Value

EOK for success

EINVAL for invalid pointer

7.27. set1536ByteFramesRx

Function Declaration

```
void set1536ByteFramesRx(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t enable;
```

Parameter List

enable [in]
if =1 then enable 1536-byte frames accept, if =0 disable

pD [in]
driver private state info specific to this instance

Description

Enable/disable reception of frames up to 1536 bytes, instead of normal 1518 bytes limit.

7.28. get1536ByteFramesRx

Function Declaration

```
uint32_t get1536ByteFramesRx(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t* enable;
```

Parameter List

`enable` [out]

equal 1 if 1536-byte frames accept enabled equal 0 if disabled.

`pD` [in]

driver private state info specific to this instance

Description

Read 1536-byte frames Rx enable status.

Return Value

EOK for success

EINVAL for invalid pointer

7.29. setRxChecksumOffload

Function Declaration

```
void setRxChecksumOffload(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t enable;
```

Parameter List

`enable` [in]

if =1 then enable, if =0 then disable

`pD` [in]

driver private state info specific to this instance

Description

Enable/disable Rx IP, TCP and UDP checksum offload.

When enabled, frames with bad IP, TCP or UDP checksums will be discarded.

7.30. getRxChecksumOffload

Function Declaration

```
uint32_t getRxChecksumOffload(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t* enable;
```

Parameter List

enable [out]
 equal 1 if enabled, equal 0 if disabled

pD [in]
 driver private state info specific to this instance

Description

Read enable/disable of Rx IP, TCP and UDP checksum offload.

Return Value

EOK for success

EINVAL for invalid pointer

7.31. setFcsRemove

Function Declaration

```
void setFcsRemove(pD, enable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t enable;
```

Parameter List

enable [in]
 if =1 then enable, if =0 then disable

pD [in]
 driver private state info specific to this instance

Description

Enable/disable FCS remove feature.

When enabled, received frames will be written without frame check sequence (last four bytes).

7.32. getFcsRemove

Function Declaration

```
uint32_t getFcsRemove(pD, enable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t* enable;
```

Parameter List

`enable` [out]
 equal 1 if enabled, equal 0 if disabled

`pD` [in]
 driver private state info specific to this instance

Description

Read enable/disable status for FCS remove feature.

Return Value

EOK for success

EINVAL for invalid pointer

7.33. setTxPartialStFwd

Function Declaration

```
uint32_t setTxPartialStFwd(pD, watermark, enable);

CEDI_PrivateData* pD;
uint32_t watermark;
uint8_t enable;
```

Parameter List

`enable` [in]
 if =1 then enables partial store and forward Tx mode, if =0 then disables

`pD` [in]
 driver private state info specific to this instance

`watermark` [in]
 value to control data forwarding - ignored if enable==0

Description

Enable/disable partial store and forward Tx mode if possible.

Return Value

EOK if successful

ENOTSUP if not in tx packet buffer mode

EINVAL if parameter invalid

7.34. getTxPartialStFwd

Function Declaration

```
uint32_t getTxPartialStFwd(pD, watermark, enable);
```

```
CEDI_PrivateData* pD;  
uint32_t* watermark;  
uint8_t* enable;
```

Parameter List

<i>enable</i>	[out]
	=1 if partial store and forward Tx mode enabled
<i>pD</i>	[in]
	driver private state info specific to this instance
<i>watermark</i>	[out]
	returns value to control data forwarding if partial store and forward Tx mode enabled

Description

Get watermark value for partial store and forward Tx mode.

Return Value

EOK for success

ENOTSUP if not in tx packet buffer mode

EINVAL if invalid parameter

7.35. setRxPartialStFwd

Function Declaration

```
uint32_t setRxPartialStFwd(pD, watermark, enable);
```

```
CEDI_PrivateData* pD;  
uint32_t watermark;  
uint8_t enable;
```

Parameter List

<i>enable</i>	[in]
	if =1 then enables partial store and forward Rx mode, if =0 then disables

pD [in]
driver private state info specific to this instance

watermark [in]
value to control data forwarding

Description

Enable/disable partial store and forward Rx mode if possible.

Return Value

EOK if successful

ENOTSUP if not in rx packet buffer mode

EINVAL if invalid parameter

7.36. getRxPartialStFwd

Function Declaration

```
uint32_t getRxPartialStFwd(pD, watermark, enable);
```

```
CEDI_PrivateData* pD;  
uint32_t* watermark;  
uint8_t* enable;
```

Parameter List

enable [out]
=1 if partial store and forward Rx mode enabled

pD [in]
driver private state info specific to this instance

watermark [out]
pointer for returning watermark value set (undefined if partial store & forward not enabled)

Description

Get watermark value for partial store and forward Rx mode.

Return Value

EOK for success

ENOTSUP if not in rx packet buffer mode

EINVAL if invalid parameter

7.37. setRxDmaDataAddrMask

Function Declaration

```
uint32_t setRxDmaDataAddrMask(pD, enableBit, bitValues);
```

```
CEDI_PrivateData* pD;  
uint8_t enableBit;  
uint8_t bitValues;
```

Parameter List

bitValues [in]

4-bit field specifying values to force address bits 31:28 to, if corresponding bit in *enableBit* is set to 1

pD [in]

driver private state info specific to this instance

enableBit [in]

4-bit field selecting bits to force value on

Description

Set the fields of the Rx DMA Data Buffer Address Mask register, which allows any/all of bits 31:28 of rx data buffer address to be forced to a particular value.

Return Value

EOK if successful

EINVAL if parameter value invalid

7.38. getRxDmaDataAddrMask

Function Declaration

```
uint32_t getRxDmaDataAddrMask(pD, enableBit, bitValues);
```

```
CEDI_PrivateData* pD;  
uint8_t* enableBit;  
uint8_t* bitValues;
```

Parameter List

bitValues [out]

pointer for returning 4-bit field specifying values to force address bits to, if corresponding bit in enableBit is set to 1

pD [in]

driver private state info specific to this instance

enableBit [out]

pointer for returning 4-bit field selecting bits to force value on

Description

Read the fields of the Rx DMA Data Buffer Address Mask register, which allows any/all of bits 31:28 of rx data buffer address to be forced to a particular value.

Return Value

EOK if successful

EINVAL if any parameter equal NULL

7.39. setRxBadPreamble

Function Declaration

```
void setRxBadPreamble(pD, enable);
```

```
CEDI_PrivateData* pD;
uint8_t enable;
```

Parameter List

enable [in]

if =1 then enable, if =0 then disable

pD [in]

driver private state info specific to this instance

Description

Enable/disable receive bad preamble feature.

When enabled, frames with non-standard preamble will not be rejected.

7.40. getRxBadPreamble

Function Declaration

```
uint32_t getRxBadPreamble(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t* enable;
```

Parameter List

enable [out]

equal 1 if enabled, equal 0 if disabled

pD [in]

driver private state info specific to this instance

Description

Read enable/disable status for receive bad preamble feature.

Return Value

EOK for success

EINVAL for invalid pointer

7.41. setFullDuplex

Function Declaration

```
uint32_t setFullDuplex(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t enable;
```

Parameter List

enable [in]

if =1 select full duplex operation if =0 select half duplex.

pD [in]

driver private state info specific to this instance

Description

Select full/half duplex operation.

Return Value

EOK for success

EINVAL if invalid parameter

7.42. getFullDuplex

Function Declaration

```
uint32_t getFullDuplex(pD, enable);

const CEDI_PrivateData* pD;
uint8_t* enable;
```

Parameter List

enable [out]

equal 1 if full duplex operation enabled equal 0 if half duplex.

pD [in]

driver private state info specific to this instance

Description

Read full/half duplex operation.

Return Value

EOK for success

EINVAL for invalid pointer

7.43. setIgnoreFcsRx

Function Declaration

```
void setIgnoreFcsRx(pD, enable);

CEDI_PrivateData* pD;
uint8_t enable;
```

Parameter List

enable [in]

if =1 then enable, if =0 then disable

pD [in]

driver private state info specific to this instance

Description

Enable/disable ignore FCS feature.

When enabled, frames with FCS/CRC errors will not be rejected.

7.44. getIgnoreFcsRx

Function Declaration

```
uint32_t getIgnoreFcsRx(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t* enable;
```

Parameter List

enable [out]

equal 1 if enabled, equal 0 if disabled

pD [in]

driver private state info specific to this instance

Description

Read enable/disable status for FCS ignore feature.

Return Value

EOK for success

EINVAL for invalid pointer

7.45. setRxHalfDuplexInTx

Function Declaration

```
void setRxHalfDuplexInTx(pD, enable);
```

```
const CEDI_PrivateData* pD;  
uint8_t enable;
```

Parameter List

enable [in]

if =1 then enable, if =0 then disable

pD [in]

driver private state info specific to this instance

Description

Enable/disable frame Rx in half-duplex mode while transmitting.

7.46. getRxHalfDuplexInTx

Function Declaration

```
uint32_t getRxHalfDuplexInTx(pD, enable);

CEDI_PrivateData* pD;
uint8_t* enable;
```

Parameter List

enable [out]
equal 1 if enabled, equal 0 if disabled

pD [in]
driver private state info specific to this instance

Description

Read enable status for frame Rx in half-duplex mode while transmitting.

Return Value

EOK for success

EINVAL for invalid pointer

7.47. getIfCapabilities

Function Declaration

```
uint32_t getIfCapabilities(pD, cap);

const CEDI_PrivateData* pD;
uint32_t* cap;
```

Parameter List

cap [out]
pointer for returning supported capabilities, OR combination of CEDI_CAP_XXX flags

pD [in]
driver private state info specific to this instance

Description

Return capabilities supported by the driver/EMAC hardware.

Return Value

EOK for success

EINVAL for invalid pointer

7.48. setLoopback

Function Declaration

```
uint32_t setLoopback(pD, mode);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t mode;
```

Parameter List

mode [in]

enum selecting mode enable/disable. =CEDI_SERDES_LOOPBACK: select loopback mode in PHY transceiver, if available; =CEDI_LOCAL_LOOPBACK: select internal loopback mode. Tx and Rx should be disabled when enabling or disabling this mode. Only available if int_loopback defined; =CEDI_NO_LOOPBACK: disable loopback mode

pD [in]

driver private state info specific to this instance

Description

Enable or disable loop back mode in the EMAC.

Return Value

EOK for success

ENOTSUP if CEDI_SERDES_LOOPBACK selected and no_pcs defined, or if CEDI_LOCAL_LOOPBACK selected and either (no_int_loopback defined or PCS mode is selected)

7.49. getLoopback

Function Declaration

```
uint32_t getLoopback(pD, mode);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t* mode;
```

Parameter List

mode [out]

returns enum for mode enable/disable. =CEDI_SERDES_LOOPBACK: selected loopback mode in PHY transceiver, if available; =CEDI_LOCAL_LOOPBACK: selected internal loopback mode. Only available if int_loopback defined; =CEDI_NO_LOOPBACK: disabled loopback mode

pD [in]

driver private state info specific to this instance

Description

Return loopback mode, same values as for setLoopbackMode.

7.50. calcMaxTxFrameSize

Function Declaration

```
uint32_t calcMaxTxFrameSize(pD, maxTxSize);
```

```
CEDI_PrivateData* pD;  
CEDI_FrameSize* maxTxSize;
```

Parameter List

maxTxSize [out]

pointer for returning array of sizes for queues

pD [in]

driver private state info specific to this instance

Description

Identify max Tx pkt size for queues.

When using full store & forward packet buffering, this is based on the sram size for each queue, otherwise it is limited by an internal counter to 16kB.

Return Value

EOK if successful

EINVAL if any parameter equal NULL

7.51. queueTxBuf

Function Declaration

```
uint32_t queueTxBuf(pD, queueNum, bufAdd, length, flags);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t queueNum;
CEDI_BuffAddr* bufAdd;
uint32_t length;
uint8_t flags;
```

Parameter List

bufAdd	[in]	pointer to address of buffer
flags	[in]	bit-flags (CEDI_TXB_xx) specifying last buffer/auto CRC
length	[in]	length of data in buffer
pD	[in]	driver private state info specific to this instance
queueNum	[in]	number of Tx queue

Description

Add a buffer containing Tx data to the end of the transmit queue.

Use repeated calls for multi-buffer frames, setting lastBuffer on the last call, to indicate the end of the frame.

Return Value

EOK if successful

ENOENT if no available descriptors

EINVAL if invalid queueNum, length or buffer alignment, NULL pointers or buffer address

7.52. qTxBuf

Function Declaration

```
uint32_t qTxBuf(pD, params);

CEDI_PrivateData* pD;
CEDI_qTxBufParams* params;
```

Parameter List

params	[in]
--------	--------

pointer to struct for all parameters required by the function

pD [in]

driver private state info specific to this instance

Description

Add a buffer containing Tx data to the end of the transmit queue.

Use repeated calls for multi-buffer frames, setting CEDI_TXB_LAST_BUFF in the flags on the last call, to indicate the end of the frame. This function is required for utilising TSO/UFO.

Return Value

EOK if successful

ENOTSUP if CEDI_TXB_ENABLE_TLT is selected and Time Base Scheduling is not supported

ENOENT if no available descriptors

EINVAL if invalid queueNum, length or buffer alignment, NULL pointers or buffer address or prm->flags specifies CEDI_TXB_LAST_BUFF at same time as CEDI_TXB_TCP_ENCAP or CEDI_TXB_UDP_ENCAP

7.53. deQTxBuf

Function Declaration

```
uint32_t deQTxBuf(pD, params);
```

```
CEDI_PrivateData* pD;
```

```
CEDI_qTxBufParams* params;
```

Parameter List

params [inout]

pointer to struct for parameters to be returned. Not all fields are used - queueNum must be specified, and bufAdd and length are returned

pD [in]

driver private state info specific to this instance

Description

Remove buffer from head of transmit queue in case of error during queueing and free the corresponding descriptor.

Caller must have knowledge of queueing status, i.e. latest frame has not been completed for transmission (first used bit still set) and how many descriptors have been queued for untransmitted frame.

Return Value

EOK if successful

ENOENT if no unfree descriptors

EINVAL if invalid queueNum or NULL parameters

7.54. txDescFree

Function Declaration

```
uint32_t txDescFree(pD, queueNum, numFree);
```

```
const CEDI_PrivateData* pD;  
uint8_t queueNum;  
uint16_t* numFree;
```

Parameter List

<i>numFree</i>	[out]	pointer for returning number of free descriptors
<i>pD</i>	[in]	driver private state info specific to this instance
<i>queueNum</i>	[in]	number of Tx queue

Description

Get number of free descriptors in specified Tx queue.

Return Value

EOK if successful

EINVAL if invalid parameter

7.55. freeTxDesc

Function Declaration

```
uint32_t freeTxDesc(pD, queueNum, descData);
```

```
CEDI_PrivateData* pD;  
uint8_t queueNum;  
CEDI_TxDescData* descData;
```

Parameter List

<i>descData</i>	[out]
-----------------	---------

pointer for returning status & descriptor data

pD [in]

driver private state info specific to this instance

queueNum [in]

number of Tx queue

Description

Read Tx descriptor queue and free used descriptor.

Struct fields:

Return Value

EOK if successful (and status is set)

ENOENT if the queue is empty (status equal CEDI_TXDATA_NONE_FREED)

EIO if an incomplete frame was detected (no lastBuffer flag in queue)

EINVAL if any parameter invalid

7.56. getTxDescStat

Function Declaration

```
void getTxDescStat(pD, txDStatWord, txDStat);

const CEDI_PrivateData* pD;
uint32_t txDStatWord;
CEDI_TxDescStat* txDStat;
```

Parameter List

txDStatWord [in]

- Tx descriptor status word

pD [in]

driver private state info specific to this instance

txDStat [out]

- pointer to bit-field struct for decoded status fields

Description

Decode the Tx descriptor status into a bit-field struct.

7.57. getTxDescSize

Function Declaration

```
void getTxDescSize(pD, txDescSize);

const CEDI_PrivateData* pD;
uint32_t* txDescSize;
```

Parameter List

<i>txDescSize</i>	[out]	
		- pointer to Tx descriptor size
<i>pD</i>	[in]	
		driver private state info specific to this instance

Description

Provides the size of Tx descriptor calculated for current configuration.

7.58. resetTxQ

Function Declaration

```
uint32_t resetTxQ(pD, queueNum);

CEDI_PrivateData* pD;
uint8_t queueNum;
```

Parameter List

<i>pD</i>	[in]	
		driver private state info specific to this instance
<i>queueNum</i>	[in]	
		number of Tx queue

Description

Reset transmit buffer queue.

Any untransmitted buffer data will be discarded and must be re-queued. Transmission must be disabled before calling this function.

Return Value

EOK if successful

EINVAL if invalid parameter

7.59. startTx

Function Declaration

```
uint32_t startTx(pD);
CEDI_PrivateData* pD;
```

Parameter List

pD [in]

driver private state info specific to this instance

Description

Enable & start the transmit circuit.

Not required during normal operation, as queueTxBuf will automatically start Tx when complete frame has been queued, but may be used to restart after a Tx error.

Return Value

EOK for success

ECANCELED if no entries in buffer

7.60. stopTx

Function Declaration

```
void stopTx(pD);
CEDI_PrivateData* pD;
```

Parameter List

pD [in]

driver private state info specific to this instance

Description

Halt transmission as soon as current frame Tx has finished.

7.61. abortTx

Function Declaration

```
void abortTx(pD);
```



```
CEDI_PrivateData* pD;
```

Parameter List

pD [in]

driver private state info specific to this instance

Description

Immediately disable transmission without waiting for completion.

Since the EMAC will reset to point to the start of transmit descriptor list, the buffer queues may have to be reset after this call.

7.62. transmitting

Function Declaration

```
uint32_t transmitting(pD);
```

```
CEDI_PrivateData* pD;
```

Parameter List

pD [in]

driver private state info specific to this instance

Description

Get state of transmitter.

Return Value

1 if active

0 if idle or *pD* equal NULL

7.63. enableTx

Function Declaration

```
void enableTx(pD);
```

```
CEDI_PrivateData* pD;
```

Parameter List

pD [in]

driver private state info specific to this instance

Description

Enable the transmit circuit.

This will be done automatically when call startTx, but it may be desirable to call this earlier, since some functionality depends on transmit being enabled.

7.64. getTxEnabled

Function Declaration

```
uint32_t getTxEnabled(pD);
```

```
CEDI_PrivateData* pD;
```

Parameter List

pD [in]

driver private state info specific to this instance

Description

Get state of transmission enabled.

Return Value

1 if transmission enabled

0 if transmission disabled or *pD* equal NULL

7.65. getTxStatus

Function Declaration

```
uint32_t getTxStatus(pD, status);
```

```
CEDI_PrivateData* pD;
```

```
CEDI_TxStatus* status;
```

Parameter List

status [out]

pointer to struct with fields for each flag

pD [in]

driver private state info specific to this instance

Description

Get the content of EMAC transmit status register.

Return Value

0 if no status bits set or status equal NULL

number raw Tx status value read

7.66. clearTxStatus

Function Declaration

```
void clearTxStatus(pD, resetStatus);
```

```
CEDI_PrivateData* pD;  
uint32_t resetStatus;
```

Parameter List

<i>resetStatus</i>	[in]
	OR'd combination of CEDI_TXS_ bit-flags
<i>pD</i>	[in]
	driver private state info specific to this instance

Description

Reset the bits of EMAC transmit status register as selected in *resetStatus*.

7.67. enableCbs

Function Declaration

```
uint32_t enableCbs(pD, qSel, idleSlope);
```

```
CEDI_PrivateData* pD;  
uint8_t qSel;  
uint32_t idleSlope;
```

Parameter List

<i>idleSlope</i>	[in]
	new idle slope value (in bytes/sec)
<i>pD</i>	[in]
	driver private state info specific to this instance
<i>qSel</i>	[in]
	if =0 selects highest priority queue (queue A), if =1 selects next-highest priority queue (queue B)

Description

Enable credit-based shaping (CBS) on the specified queue.

If already enabled then first disables, sets a new idle slope value for the queue, and re-enables CBS

Return Value

EOK if successful

ENOTSUP if CBS not present in h/w config

EINVAL if priority queueing not enabled (i.e. only one Tx queue) or invalid parameter

7.68. disableCbs

Function Declaration

```
void disableCbs(pD, qSel);
```

```
CEDI_PrivateData* pD;  
uint8_t qSel;
```

Parameter List

pD [in]

driver private state info specific to this instance

qSel [in]

if =0 selects highest priority queue (queue A), if =1 selects next-highest priority queue (queue B)

Description

Disable CBS on the specified queue.

7.69. getCbsQSetting

Function Declaration

```
uint32_t getCbsQSetting(pD, qSel, enable, idleSlope);
```

```
CEDI_PrivateData* pD;  
uint8_t qSel;  
uint8_t* enable;  
uint32_t* idleSlope;
```

Parameter List

enable [out]

returns: 1 if CBS enabled for the specified queue 0 if not enabled

`idleSlope` [out]

pointer for returning the idleSlope value for selected queue.

`pD` [in]

driver private state info specific to this instance

`qSel` [in]

if =0 selects highest priority queue (queue A), if =1 selects next-highest priority queue (queue B)

Description

Read CBS setting for the specified queue.

Return Value

EOK for success

ENOTSUP if CBS not present in h/w config

EINVAL for invalid pointer

7.70. setIpgStretch

Function Declaration

```
uint32_t setIpgStretch(pD, enable, multiplier, divisor);
```

`CEDI_PrivateData* pD;`

`uint8_t enable;`

`uint8_t multiplier;`

`uint8_t divisor;`

Parameter List

`enable` [in]

if =1 then enable IPG stretch, if =0 then disable

`divisor` [in]

after multiplying previous frame length, divide by (divisor+1) - if result>96 bits, this is used for the Tx IPG. Ignored if enable equal 0.

`pD` [in]

driver private state info specific to this instance

`multiplier` [in]

multiplying factor applied to previous Tx frame length. Ignored if enable equal 0.

Description

Enable/disable the inter-packet gap (IPG) stretch function.

Return Value

EOK if successful

EINVAL if pD equal NULL

7.71. getIpgStretch

Function Declaration

```
uint32_t getIpgStretch(pD, enable, multiplier, divisor);
```

```
CEDI_PrivateData* pD;  
uint8_t* enable;  
uint8_t* multiplier;  
uint8_t* divisor;
```

Parameter List

<code>enable</code>	[out]	
		pointer for returning enabled state: returns 1 if IPG stretch enabled, 0 if disabled.
<code>divisor</code>	[out]	
		pointer for returning IPG divisor
<code>pD</code>	[in]	
		driver private state info specific to this instance
<code>multiplier</code>	[out]	
		pointer for returning IPG multiplying factor

Description

Read the inter-packet gap (IPG) stretch settings.

Return Value

EOK if successful

EINVAL if any parameter equal NULL

7.72. calcMaxRxFrameSize

Function Declaration

```
uint32_t calcMaxRxFrameSize(pD, maxSize);

CEDI_PrivateData* pD;
uint32_t* maxSize;
```

Parameter List

<i>maxSize</i>	[out]	pointer for returning max frame size same for each Rx queue
<i>pD</i>	[in]	driver private state info specific to this instance

Description

Identify max Rx pkt size for queues - determined by size of Rx packet buffer (if using full store & forward mode), and the current maximum frame size, e.g.

1518, 1536 or jumbo frame.

Return Value

EOK if successful

EINVAL if NULL parameters

7.73. addRxBuf

Function Declaration

```
uint32_t addRxBuf(pD, queueNum, buf, init);

CEDI_PrivateData* pD;
uint8_t queueNum;
CEDI_BuffAddr* buf;
uint8_t init;
```

Parameter List

<i>init</i>	[in]	if >0 then initialise the buffer data to all zeros
<i>buf</i>	[in]	pointer to address of buffer. Checked for word-alignment in 64/128-bit width cases.
<i>pD</i>	[in]	

driver private state info specific to this instance

queueNum [in]

number of the Rx queue

Description

Add a buffer (size determined by rxBufLength in CEDI_Config) to the end of the receive buffer queue.

This function is intended to be used during setting up the receive buffers, and should not be called while Rx is enabled or unread data remains in the queue. Note that one extra descriptor is reserved, to provide wrap-around protection (must always have one used bit set).

Return Value

EOK if successful

EINVAL if invalid queueNum, buffer alignment, or bufStart pointer/addresses

7.74. numRxBufs

Function Declaration

```
uint32_t numRxBufs(pD, queueNum, numBufs);
```

```
const CEDI_PrivateData* pD;
```

```
uint8_t queueNum;
```

```
uint16_t* numBufs;
```

Parameter List

numBufs [out]

pointer for returning number of descriptors

pD [in]

driver private state info specific to this instance

queueNum [in]

number of the Rx queue

Description

Get the total number of buffers/descriptors present in the specified Rx queue.

Return Value

EOK if successful

EINVAL if invalid parameter

7.75. numRxUsed

Function Declaration

```
uint32_t numRxUsed(pD, queueNum);
```

```
CEDI_PrivateData* pD;  
uint8_t queueNum;
```

Parameter List

<i>pD</i>	[in]
driver private state info specific to this instance	
<i>queueNum</i>	[in]
number of the Rx queue	

Description

Get the number of buffers/descriptors marked "used" in the specified Rx queue, i.e. those holding unread data.

Return Value

0 if invalid parameter or NULL pointer
number of used buffers

7.76. readRxBuf

Function Declaration

```
uint32_t readRxBuf(pD, queueNum, buf, init, descData);
```

```
CEDI_PrivateData* pD;  
uint8_t queueNum;  
CEDI_BuffAddr* buf;  
uint8_t init;  
CEDI_RxDescData* descData;
```

Parameter List

<i>init</i>	[in]
if >0 then initialise the (new) buffer data to all zeros. Ignored if no data available.	
<i>descData</i>	[out]
pointer for returning status & descriptor data	
<i>buf</i>	[inout]

pointer to address of memory for new buffer to add to Rx descriptor queue; if data is available the buffer addresses for this are returned in buf, else if no data available then the new buffer can be re-used. Physical address of buffer is checked for word-alignment in 64/128-bit width cases.

pD [in]
driver private state info specific to this instance

queueNum [in]
number of the Rx queue

Description

Read first unread descriptor (at tail of queue): if new data is available it swaps out the buffer and replaces it with a new one, clears the descriptor for re-use, then updates the driver queue- pointer.

Checks for Start Of Frame (SOF) and End Of Frame (EOF) flags in the descriptors, passing back in status parameter. If EOF set, the descriptor status is returned via rxDescStat. Struct fields: uint32_t rxDescStat - Rx descriptor status word uint8_t status - Rx data status, one of the following values: CEDI_RXDATA_SOF_EOF :data available, single-buffer frame (SOF & EOF set) CEDI_RXDATA_SOF_ONLY :data available, start of multi-buffer frame CEDI_RXDATA_NO_FLAG :data available, intermediate buffer of multi- buffer frame CEDI_RXDATA_EOF_ONLY :data available, end of multi-buffer frame CEDI_RXDATA_NODATA :no data available

Return Value

EOK if successful

EINVAL if invalid queueNum, buf, rxDescStat or status parameters

7.77. getRxDescStat

Function Declaration

```
void getRxDescStat(pD, rxDStatWord, rxDStat);

CEDI_PrivateData* pD;
uint32_t rxDStatWord;
CEDI_RxDescStat* rxDStat;
```

Parameter List

rxDStatWord [in]
- Rx descriptor status word

pD [in]
driver private state info specific to this instance

rxDStat [out]
pointer to bit-field struct for decoded status fields

Description

Decode the Rx descriptor status into a bit-field struct.

Note that most of these fields are only valid when End Of Frame bit15 is set - see hardware user guide

7.78. getRxDescSize

Function Declaration

```
void getRxDescSize(pD, rxDescSize);
```

```
const CEDI_PrivateData* pD;  
uint32_t* rxDescSize;
```

Parameter List

rxDescSize [out]

- pointer to Rx descriptor size

pD [in]

driver private state info specific to this instance

Description

Provides the size of Rx descriptor calculated for current configuration.

7.79. rxEnabled

Function Declaration

```
uint32_t rxEnabled(pD);
```

```
CEDI_PrivateData* pD;
```

Parameter List

pD [in]

driver private state info specific to this instance

Description

Get state of receiver.

Return Value

1 if enabled

0 if disabled or pD equal NULL

7.80. enableRx

Function Declaration

```
void enableRx(pD);
CEDI_PrivateData* pD;
```

Parameter List

pD [in]
driver private state info specific to this instance

Description

Enable the receive circuit.

7.81. disableRx

Function Declaration

```
void disableRx(pD);
CEDI_PrivateData* pD;
```

Parameter List

pD [in]
driver private state info specific to this instance

Description

Disable the receive circuit.

7.82. removeRxBuf

Function Declaration

```
uint32_t removeRxBuf(pD, queueNum, buf);
CEDI_PrivateData* pD;
uint8_t queueNum;
CEDI_BuffAddr* buf;
```

Parameter List

buf [out]

pointer to struct for returning virtual and physical addresses of buffer.

pD [in]

driver private state info specific to this instance

queueNum [in]

number of the Rx queue

Description

Remove a buffer from the end of the receive buffer queue.

This function is intended to be used when shutting down the driver, prior to deallocating the receive buffers, and should not be called while Rx is enabled or unread data remains in the queue.

Return Value

EOK if successful

ENOENT if no buffers left to free (mechanical success)

EINVAL if invalid queueNum

7.83. resetRxQ

Function Declaration

```
uint32_t resetRxQ(pD, queueNum, ptrsOnly);
```

```
CEDI_PrivateData* pD;
uint8_t queueNum;
uint8_t ptrsOnly;
```

Parameter List

ptrsOnly [in]

"flag to allow full queue to be reset after link down/up (enableRx). if =1 then only reset tail & stop pointers and clear used bits"

pD [in]

driver private state info specific to this instance

queueNum [in]

number of the Rx queue

Description

Reset Rx buffer descriptor list to initial empty state (if ptrsOnly=0), clearing all descriptors.

For use after a fatal error. Disables receive circuit.

Return Value

EOK if successful

EINVAL if invalid parameter

7.84. getRxStatus

Function Declaration

```
uint32_t getRxStatus(pD, status);
```

```
CEDI_PrivateData* pD;  
CEDI_RxStatus* status;
```

Parameter List

status [out]

pointer to struct with fields for each flag

pD [in]

driver private state info specific to this instance

Description

Return the content of EMAC receive status register.

Return Value

1 if any flags set

0 if not or status equal NULL

7.85. clearRxStatus

Function Declaration

```
void clearRxStatus(pD, resetStatus);
```

```
CEDI_PrivateData* pD;  
uint32_t resetStatus;
```

Parameter List

resetStatus [in]

OR'd combination of CEDI_RXS_ bit-flags

`pD` [in]

driver private state info specific to this instance

Description

Reset the bits of EMAC receive status register as selected in `resetStatus`.

7.86. setHdrDataSplit

Function Declaration

```
uint32_t setHdrDataSplit(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t enable;
```

Parameter List

`enable` [in]

if =1 then enable; if =0 then disable

`pD` [in]

driver private state info specific to this instance

Description

Enable/disable header-data split feature.

When enabled, frame L2/L3/L4 headers will be written to separate buffer, before data starts in a second buffer (if not zero payload)

Return Value

EOK if success

EINVAL if invalid parameter

7.87. getHdrDataSplit

Function Declaration

```
uint32_t getHdrDataSplit(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t* enable;
```

Parameter List

`enable` [out]

pointer for returning enabled status

pD [in]

driver private state info specific to this instance

Description

Read enable/disable status for header-data split feature.

Return Value

EOK successful

EINVAL if invalid parameter

7.88. setRscEnable

Function Declaration

```
uint32_t setRscEnable(pD, queue, enable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t queue;
```

```
uint8_t enable;
```

Parameter List

queue [in]

priority queue to enable or disable RSC on

enable [in]

if =1 enable RSC on selected priority queue(s), else disable it

pD [in]

driver private state info specific to this instance

Description

Enable/disable Receive Segment Coalescing function.

When enabled, consecutive TCP/IP frames on a priority queue will be combined to form a single large frame

Return Value

EOK for success

ENOTSUP if feature not available

EINVAL for invalid parameter

7.89. getRscEnable

Function Declaration

```
uint32_t getRscEnable(pD, queue, enable);

CEDI_PrivateData* pD;
uint8_t queue;
uint8_t* enable;
```

Parameter List

queue [in]
 priority queue to read RSC enabled status for

enable [out]
 pointer for returning enabled status

pD [in]
 driver private state info specific to this instance

Description

Read enabled status of RSC on a specified priority queue.

Return Value

EOK successful

ENOTSUP if feature not available

EINVAL if invalid parameter

7.90. setRscClearMask

Function Declaration

```
uint32_t setRscClearMask(pD, setMask);

CEDI_PrivateData* pD;
uint8_t setMask;
```

Parameter List

pD [in]
 driver private state info specific to this instance

setMask [in]
 if =1 prevents RSC disabling by end-coalesce flags (SYN/FIN/RST/URG) - applies to all queues

Description

Set/Clear Mask of Receive Segment Coalescing disabling.

When mask is set and RSC is enabled, the RSC operation is not disabled by receipt of frame with an end-coalesce flag set (SYN/FIN/RST/URG)

Return Value

EOK for success

ENOTSUP if feature not available

EINVAL for invalid pointer

7.91. setSpecificAddr

Function Declaration

```
uint32_t setSpecificAddr(pD, addrNum, addr, specFilterType, byteMask);

CEDI_PrivateData* pD;
uint8_t addrNum;
const CEDI_MacAddress* addr;
uint8_t specFilterType;
uint8_t byteMask;
```

Parameter List

specFilterType	[in]	flag specifying whether to use MAC source or destination address to be compared for filtering. Source filter when =1.
addrNum	[in]	number of specific address filter, in range 1 - num_spec_add_filters
addr	[in]	pointer to the 6-byte MAC address value to write
pD	[in]	driver private state info specific to this instance
byteMask	[in]	Bits masking out bytes of specific address from comparison.

Description

Set specific address register to the given address value.

Return Value

EOK if success

ENOTSUP if no specific address registers available

EINVAL if invalid parameter

7.92. getSpecificAddr

Function Declaration

```
uint32_t getSpecificAddr(pD, addrNum, addr, specFilterType, byteMask);
```

```
CEDI_PrivateData* pD;
uint8_t addrNum;
CEDI_MacAddress* addr;
uint8_t* specFilterType;
uint8_t* byteMask;
```

Parameter List

specFilterType	[out]	flag specifying whether to use MAC source or destination address to be compared for filtering. =1 for source address
addrNum	[in]	number of specific address filter, in range 1 - num_spec_add_filters
addr	[out]	pointer to a 6-byte MAC address struct for returning the address value
pD	[in]	driver private state info specific to this instance
byteMask	[out]	Bits masking out bytes of specific address from comparison. When high, the associated address byte will be ignored. e.g. LSB of byteMask=1 implies first byte received should not be compared.

Description

Get the value of a specific address register destination address for filtering.

When set to 1 use source address.

Return Value

EOK if success

ENOTSUP if no specific address registers available

EINVAL if invalid parameter

7.93. setSpecificAddr1Mask

Function Declaration

```
uint32_t setSpecificAddr1Mask(pD, mask);
```

```
CEDI_PrivateData* pD;  
const CEDI_MacAddress* mask;
```

Parameter List

mask [in]

pointer to the address mask value to write

pD [in]

driver private state info specific to this instance

Description

Set the specific address 1 mask register to the given value, allowing address matching against a portion of the specific address 1 register.

Return Value

EOK if successful

ENOTSUP if CEDI_DesignCfg.num_spec_add_filters==0

EINVAL if mask equal NULL

7.94. getSpecificAddr1Mask

Function Declaration

```
uint32_t getSpecificAddr1Mask(pD, mask);
```

```
CEDI_PrivateData* pD;  
CEDI_MacAddress* mask;
```

Parameter List

mask [out]

pointer to a 6-byte MAC address struct for returning the mask value

pD [in]

driver private state info specific to this instance

Description

Get the value of specific address 1 mask register.

Return Value

EOK if successful

ENOTSUP if CEDI_DesignCfg.num_spec_add_filters==0

EINVAL if mask equal NULL

7.95. disableSpecAddr

Function Declaration

```
uint32_t disableSpecAddr(pD, addrNum);
```

CEDI_PrivateData* *pD*;

uint8_t *addrNum*;

Parameter List

addrNum [in]

- number of specific address filters, in range 1 - num_spec_add_filters

pD [in]

- driver private state info specific to this instance

Description

Disable the specific address match stored at given register, by writing 0 to lower address register.

Return Value

EOK if successful

ENOTSUP if (CEDI_DesignCfg.num_spec_add_filters==0)

EINVAL if ((*addrNum*==0) || (*addrNum*>CEDI_DesignCfg.num_spec_add_filters))

7.96. setTypeIdMatch

Function Declaration

```
uint32_t setTypeIdMatch(pD, matchSel, typeId, enable);
```

CEDI_PrivateData* *pD*;

uint8_t *matchSel*;

uint16_t *typeId*;

```
uint8_t enable;
```

Parameter List

<code>typeId</code>	[in]
the Type ID match value to write ignored if enable equal 0	
<code>enable</code>	[in]
if =1 enables the type matching for this ID, if =0 then disables type matching for this ID	
<code>matchSel</code>	[in]
number of TypeID Match register, range 1 - 4	
<code>pD</code>	[in]
driver private state info specific to this instance	

Description

En/Disable Type ID match field of the specified register, and set type Id value if enabling.

Return Value

EOK if successful

EINVAL if matchSel invalid

7.97. getTypeIdMatch

Function Declaration

```
uint32_t getTypeIdMatch(pD, matchSel, typeId, enable);
```

```
CEDI_PrivateData* pD;
uint8_t matchSel;
uint16_t* typeId;
uint8_t* enable;
```

Parameter List

<code>typeId</code>	[out]
pointer for returning the Type ID match value read ignored if disabled	
<code>enable</code>	[out]
"pointer for returning enabled status: equal 1 if typeId matching is enabled for this register, equal 0 if disabled"	
<code>matchSel</code>	[in]

number of TypeID Match register, range 1 - 4

pD [in]

driver private state info specific to this instance

Description

Read the specified Type ID match register settings.

Return Value

EOK if successful

EINVAL if matchSel invalid

7.98. setUnicastEnable

Function Declaration

```
void setUnicastEnable(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t enable;
```

Parameter List

enable [in]

if =1 enables reception, if =0 then disables

pD [in]

driver private state info specific to this instance

Description

En/disable reception of unicast frames when hash register matched.

7.99. getUnicastEnable

Function Declaration

```
uint32_t getUnicastEnable(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t* enable;
```

Parameter List

enable [out]

equal 1 if reception enabled, equal 0 if disabled

pD [in]

driver private state info specific to this instance

Description

Return state of unicast frame matching.

Return Value

EOK for success

EINVAL for invalid pointer

7.100. setMulticastEnable

Function Declaration

```
void setMulticastEnable(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t enable;
```

Parameter List

enable [in]

if =1 enables, if =0 then disables

pD [in]

driver private state info specific to this instance

Description

En/disable reception of multicast frames when hash register matched.

7.101. getMulticastEnable

Function Declaration

```
uint32_t getMulticastEnable(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t* enable;
```

Parameter List

enable [out]

equal 1 if enabled, equal 0 if disabled

pD [in]

driver private state info specific to this instance

Description

Return state of multicast frame matching.

Return Value

EOK for success

EINVAL for invalid pointer

7.102. setNoBroadcast

Function Declaration

```
void setNoBroadcast(pD, reject);
```

CEDI_PrivateData* *pD*;

uint8_t *reject*;

Parameter List

pD [in]

driver private state info specific to this instance

reject [in]

if =0 broadcasts are accepted, if =1 they are rejected.

Description

Dis/Enable receipt of broadcast frames.

7.103. getNoBroadcast

Function Declaration

```
uint32_t getNoBroadcast(pD, reject);
```

CEDI_PrivateData* *pD*;

uint8_t* *reject*;

Parameter List

pD [in]

driver private state info specific to this instance

reject [out]

returns 0 if broadcasts are accepted, 1 if they are rejected

Description

Return broadcast rejection setting.

Return Value

EOK for success

EINVAL for invalid pointer

7.104. setVlanOnly

Function Declaration

```
void setVlanOnly(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t enable;
```

Parameter List

enable [in]

if =1 reject non-VLAN-tagged frames, if =0 then accept

pD [in]

driver private state info specific to this instance

Description

En/Disable receipt of only frames which have been VLAN tagged.

7.105. getVlanOnly

Function Declaration

```
uint32_t getVlanOnly(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t* enable;
```

Parameter List

enable [out]

returns 1 if non-VLAN-tagged frames rejected

pD [in]

driver private state info specific to this instance

Description

Return VLAN-tagged filter setting.

Return Value

EOK for success

EINVAL for invalid pointer

7.106. setStackedVlanReg

Function Declaration

```
void setStackedVlanReg(pD, enable, vlanType);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t enable;
```

```
uint16_t vlanType;
```

Parameter List

enable [in]

if =1 enables stacked VLAN processing, if =0 disables it

pD [in]

driver private state info specific to this instance

vlanType [in]

sets user defined VLAN type for matching first VLAN tag. Ignored if *enable* equal 0.

Description

En/Disable stacked VLAN processing mode.

7.107. getStackedVlanReg

Function Declaration

```
void getStackedVlanReg(pD, enable, vlanType);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t* enable;
uint16_t* vlanType;
```

Parameter List

enable [out]
pointer for returning Enabled field equal 1 if enabled equal 0 if disabled.

pD [in]
driver private state info specific to this instance

vlanType [out]
pointer for returning VLAN type field

Description

Reads stacked VLAN register settings.

7.108. setCopyAllFrames

Function Declaration

```
void setCopyAllFrames(pD, enable);
```

```
CEDI_PrivateData* pD;
uint8_t enable;
```

Parameter List

enable [in]
if =1 enables copy all frames mode, if =0 then this is disabled

pD [in]
driver private state info specific to this instance

Description

En/Disable copy all frames mode.

7.109. getCopyAllFrames

Function Declaration

```
uint32_t getCopyAllFrames(pD, enable);
```

```
CEDI_PrivateData* pD;
uint8_t* enable;
```

Parameter List

`enable` [out]

equal 1 if copy all frames mode enabled equal 0 if this is disabled

`pD` [in]

driver private state info specific to this instance

Description

Get "copy all" setting.

Return Value

EOK for success

EINVAL for invalid pointer

7.110. setHashAddr

Function Declaration

```
uint32_t setHashAddr(pD, hAddrTop, hAddrBot);
```

```
CEDI_PrivateData* pD;
```

```
uint32_t hAddrTop;
```

```
uint32_t hAddrBot;
```

Parameter List

`hAddrBot` [in]

least significant 32 bits of hash register

`hAddrTop` [in]

most significant 32 bits of hash register

`pD` [in]

driver private state info specific to this instance

Description

Set the hash address register.

Return Value

EOK for success

EINVAL if `pD` equal NULL

7.111. getHashAddr

Function Declaration

```
uint32_t getHashAddr(pD, hAddrTop, hAddrBot);
```

```
CEDI_PrivateData* pD;  
uint32_t* hAddrTop;  
uint32_t* hAddrBot;
```

Parameter List

hAddrBot [out]

pointer for returning least significant 32 bits of hash register

hAddrTop [out]

pointer for returning most significant 32 bits of hash register

pD [in]

driver private state info specific to this instance

Description

Read the hash address register.

Return Value

EOK for success

EINVAL if any parameter equal NULL

7.112. setLenErrDiscard

Function Declaration

```
void setLenErrDiscard(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t enable;
```

Parameter List

enable [in]

if =1 then enable, if =0 then disable

pD [in]

driver private state info specific to this instance

Description

Enable/disable discard of frames with length shorter than given in length field.

7.113. getLenErrDiscard

Function Declaration

```
uint32_t getLenErrDiscard(pD, enable);

CEDI_PrivateData* pD;
uint8_t* enable;
```

Parameter List

enable [out]
 equal 1 if enabled, equal 0 if disabled

pD [in]
 driver private state info specific to this instance

Description

Read enable/disable status for discard of frames with length shorter than given in length field.

Return Value

EOK for success

EINVAL for invalid pointer

7.114. getNumScreenRegs

Function Declaration

```
uint32_t getNumScreenRegs(pD, regNums);

const CEDI_PrivateData* pD;
CEDI_NumScreeners* regNums;
```

Parameter List

regNums [out]
 points to a CEDI_NumScreeners struct for returning the numbers of registers

pD [in]
 driver private state info specific to this instance

Description

Read numbers of screener, ethtype & compare registers.

Return Value

EOK if successful

EINVAL if parameter invalid

7.115. setType1ScreenReg

Function Declaration

```
uint32_t setType1ScreenReg(pD, regNum, regVals);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t regNum;
```

```
const CEDI_T1Screen* regVals;
```

Parameter List

regNum [in]

Type 1 register number, range 0 to CEDI_DesignCfg.num_type1_screeners-1

regVals [in]

points to a CEDI_T1Screen struct with the match parameters to be written

pD [in]

driver private state info specific to this instance

Description

Write Rx frame matching values to a Type 1 screening register, for allocating to a priority queue.

Return Value

EOK if successful

EINVAL if parameter invalid

7.116. getType1ScreenReg

Function Declaration

```
uint32_t getType1ScreenReg(pD, regNum, regVals);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t regNum;
```



```
CEDI_T1Screen* regVals;
```

Parameter List

regNum [in]
 Type 1 register number, range 0 to CEDI_DesignCfg.num_type1_screeners-1

regVals [out]
 points to a CEDI_T1Screen struct for returning the match parameters

pD [in]
 driver private state info specific to this instance

Description

Read Rx frame matching values from a Type1 screening register.

Return Value

EOK if successful

EINVAL if parameter invalid

7.117. setType2ScreenReg

Function Declaration

```
uint32_t setType2ScreenReg(pD, regNum, regVals);  

CEDI_PrivateData* pD;  

uint8_t regNum;  

const CEDI_T2Screen* regVals;
```

Parameter List

regNum [in]
 Type 2 register number, range 0 to CEDI_DesignCfg.num_type2_screeners-1

regVals [in]
 points to a CEDI_T2Screen struct with the match parameters to be written

pD [in]
 driver private state info specific to this instance

Description

Write Rx frame matching values to a Type 2 screening register, for allocating to a priority queue.

Return Value

EOK if successful

EINVAL if parameter invalid

7.118. getType2ScreenReg

Function Declaration

```
uint32_t getType2ScreenReg(pD, regNum, regVals);
```

```
CEDI_PrivateData* pD;  
uint8_t regNum;  
CEDI_T2Screen* regVals;
```

Parameter List

regNum [in]

Type 2 register number, range 0 to CEDI_DesignCfg.num_type2_screeners-1

regVals [out]

points to a CEDI_T2Screen struct for returning the match parameters

pD [in]

driver private state info specific to this instance

Description

Read Rx frame matching values from a Type 2 screening register.

Return Value

EOK if successful

EINVAL if parameter invalid

7.119. setType2EthertypeReg

Function Declaration

```
uint32_t setType2EthertypeReg(pD, index, eTypeVal);
```

```
CEDI_PrivateData* pD;  
uint8_t index;  
uint16_t eTypeVal;
```

Parameter List

index [in]

Type 2 ethertype register number, range 0 to CEDI_DesignCfg.num_scr2_ethertype_regs-1

eTypeVal [in]

Ethertype compare value to write

pD [in]

driver private state info specific to this instance

Description

Write the ethertype compare value at the given index in the Ethertype registers.

Return Value

EOK if successful

EINVAL if parameter invalid

7.120. getType2EthertypeReg

Function Declaration

```
uint32_t getType2EthertypeReg(pD, index, eTypeVal);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t index;
```

```
uint16_t* eTypeVal;
```

Parameter List

index [in]

Type 2 ethertype register number, range 0 to CEDI_DesignCfg.num_scr2_ethertype_regs-1

eTypeVal [out]

pointer for returning the Ethertype compare value

pD [in]

driver private state info specific to this instance

Description

Read the ethertype compare value at the given index in the Ethertype registers.

Return Value

EOK if successful

EINVAL if parameter invalid

7.121. setType2CompareReg

Function Declaration

```
uint32_t setType2CompareReg(pD, index, regVals);

CEDI_PrivateData* pD;
uint8_t index;
const CEDI_T2Compare* regVals;
```

Parameter List

<i>index</i>	[in]
	Type 2 compare register number, range 0 to CEDI_DesignCfg.num_scr2_compare_regs-1
<i>regVals</i>	[in]
	points to a CEDI_T2Compare struct with the compare parameters to be written
<i>pD</i>	[in]
	driver private state info specific to this instance

Description

Write the compare value at the given index in the Type 2 compare register.

Return Value

EOK if successful

EINVAL if parameter invalid

7.122. getType2CompareReg

Function Declaration

```
uint32_t getType2CompareReg(pD, index, regVals);

CEDI_PrivateData* pD;
uint8_t index;
CEDI_T2Compare* regVals;
```

Parameter List

<i>index</i>	[in]
	Type 2 compare register number, range 0 to CEDI_DesignCfg.num_scr2_compare_regs-1
<i>regVals</i>	[out]

points to a CEDI_T2Compare struct for returning the compare parameters

pD [in]

driver private state info specific to this instance

Description

Read the compare value at the given index in the Type 2 compare register.

Return Value

EOK if successful

EINVAL if parameter invalid

7.123. setPauseEnable

Function Declaration

```
void setPauseEnable(pD, enable);
```

```
CEDI_PrivateData* pD;
uint8_t enable;
```

Parameter List

enable [in]

if =1 then enable, if =0 then disable

pD [in]

driver private state info specific to this instance

Description

Enable/disable pausing after valid non-zero (non-PFC) pause frame received.

7.124. getPauseEnable

Function Declaration

```
uint32_t getPauseEnable(pD, enable);
```

```
CEDI_PrivateData* pD;
uint8_t* enable;
```

Parameter List

enable [out]

equal 1 if enabled, equal 0 if disabled

`pD` [in]

driver private state info specific to this instance

Description

Read enable/disable of pausing after valid non-zero (non-PFC) pause frame received.

Return Value

EOK for success

EINVAL for invalid pointer

7.125. txPauseFrame

Function Declaration

```
void txPauseFrame(pD);
```

```
CEDI_PrivateData* pD;
```

Parameter List

`pD` [in]

driver private state info specific to this instance

Description

Transmit a normal pause frame.

7.126. txZeroQPause

Function Declaration

```
void txZeroQPause(pD);
```

```
CEDI_PrivateData* pD;
```

Parameter List

`pD` [in]

driver private state info specific to this instance

Description

Transmit a pause frame with zero quantum value.

7.127. getRxPauseQuantum

Function Declaration

```
uint32_t getRxPauseQuantum(pD, value);

CEDI_PrivateData* pD;
uint16_t* value;
```

Parameter List

value [out]
 returns transmit pause quantum value

pD [in]
 driver private state info specific to this instance

Description

Return the current value of received pause quantum.

Return Value

EOK for success
 EINVAL for invalid pointer

7.128. setTxPauseQuantum

Function Declaration

```
uint32_t setTxPauseQuantum(pD, value, qpriority);

CEDI_PrivateData* pD;
uint16_t value;
uint8_t qpriority;
```

Parameter List

qpriority [in]
 quantum priority, the priority for the pause quantum

value [in]
 transmit pause quantum value

pD [in]
 driver private state info specific to this instance

Description

Set the pause quantum value to use when transmitting non-zero quantum pause frame.

Return Value

EOK for success

EINVAL if invalid parameter

7.129. getTxPauseQuantum

Function Declaration

```
uint32_t getTxPauseQuantum(pD, value, qpriority);
```

```
CEDI_PrivateData* pD;
```

```
uint16_t* value;
```

```
uint8_t qpriority;
```

Parameter List

qpriority [in]

quantum priority,the priority for the pause quantum

value [out]

returns transmit pause quantum value

pD [in]

driver private state info specific to this instance

Description

Read the non-zero transmit pause quantum value.

Return Value

EOK for success

EINVAL if invalid parameter

7.130. setCopyPauseDisable

Function Declaration

```
void setCopyPauseDisable(pD, disable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t disable;
```


Parameter List

`disable` [in]

if =1 disable, if =0 enable

`pD` [in]

driver private state info specific to this instance

Description

Disable/Enable copying of valid Rx pause frames to memory.

7.131. `getCopyPauseDisable`

Function Declaration

```
uint32_t getCopyPauseDisable(pD, disable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t* disable;
```

Parameter List

`disable` [out]

if =1 disabled, if =0 enabled

`pD` [in]

driver private state info specific to this instance

Description

Read disable/enable state for copying of valid Rx pause frames to memory.

Return Value

EOK for success

EINVAL for invalid pointer

7.132. `setPfcPriorityBasedPauseRx`

Function Declaration

```
void setPfcPriorityBasedPauseRx(pD, enable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t enable;
```

Parameter List

`enable` [in]

if =1 enable, if =0 then disable

`pD` [in]

driver private state info specific to this instance

Description

Enable/disable PFC negotiation and reception of priority based pause frames.

7.133. `getPfcPriorityBasedPauseRx`

Function Declaration

```
uint32_t getPfcPriorityBasedPauseRx(pD, enable);
```

`CEDI_PrivateData*` *pD*;

`uint8_t*` *enable*;

Parameter List

`enable` [out]

returns combined enabled Rx state and negotiated state

`pD` [in]

driver private state info specific to this instance

Description

Read enable/disable state of reception for priority based pause frames, and negotiation state.

value can be decoded with two constants: if (OR with `CEDI_PFC_PBP_RX_EN`) not equal 0 then PFC priority based pause frame Rx enabled if (OR with `CEDI_PFC_PBP_NEG`) not equal 0 then PFC priority based pause has been negotiated

Return Value

EOK for success

EINVAL for invalid pointer

7.134. `txPfcPriorityBasedPause`

Function Declaration

```
uint32_t txPfcPriorityBasedPause(pD);
```

```
CEDI_PrivateData* pD;
```

Parameter List

pD [in]

driver private state info specific to this instance

Description

Transmit PFC Priority Based Pause Frame, taking field values as defined by `setTxPfcPauseFrameFields`.

Return Value

EOK for success

EINVAL if NULL pointer or not using full duplex mode or if transmission is disabled (mechanical success)

7.135. setTxPfcPauseFrameFields

Function Declaration

```
uint32_t setTxPfcPauseFrameFields(pD, priEnVector, zeroQSelVector);
```

```
CEDI_PrivateData* pD;  
uint8_t priEnVector;  
uint8_t zeroQSelVector;
```

Parameter List

priEnVector [in]

written to priority enable vector of PFC priority based pause frame

zeroQSelVector [in]

selects pause quantum fields to set to zero

pD [in]

driver private state info specific to this instance

Description

Set priority enable vector and zero quantum select vector fields of the Tx PFC Pause Frame (see User Guide for description).

Return Value

EOK if successful

EINVAL if pD equal NULL

7.136. getTxPfcPauseFrameFields

Function Declaration

```
uint32_t getTxPfcPauseFrameFields(pD, priEnVector, zeroQSelVector);

CEDI_PrivateData* pD;
uint8_t* priEnVector;
uint8_t* zeroQSelVector;
```

Parameter List

priEnVector	[out]	pointer for returning priority enable vector to be written to PFC priority based pause frame
zeroQSelVector	[out]	pointer for returning vector selecting pause quantum fields to set to zero
pD	[in]	driver private state info specific to this instance

Description

Read priority enable vector and zero quantum select vector fields selected by setTxPfcPauseFrameFields.

Return Value

EOK if successful

EINVAL if any parameter equal NULL

7.137. setEnableMultiPfcPauseQuantum

Function Declaration

```
uint32_t setEnableMultiPfcPauseQuantum(pD, enMultiPfcPause);

CEDI_PrivateData* pD;
uint8_t enMultiPfcPause;
```

Parameter List

enMultiPfcPause	[in]	set this field to 1 to enable multiple PFC pause quanta.
pD	[in]	

driver private state info specific to this instance

Description

Set Enable bit for multiple PFC pause quantums, one per pause priority.

Return Value

EOK if successful

EINVAL if pD equal NULL

7.138. getEnableMultiPfcPauseQuantum

Function Declaration

```
uint32_t getEnableMultiPfcPauseQuantum(pD, enMultiPfcPause);

CEDI_PrivateData* pD;
uint8_t* enMultiPfcPause;
```

Parameter List

enMultiPfcPause [out]

equal 1 if multiple PFC pause quantums enabled

pD [in]

driver private state info specific to this instance

Description

Get the Enable bit for multiple PFC pause quantums, one per pause priority.

Return Value

EOK if successful

EINVAL if any parameter equal NULL

7.139. setUnicastPtpDetect

Function Declaration

```
void setUnicastPtpDetect(pD, enable);

CEDI_PrivateData* pD;
uint8_t enable;
```

Parameter List

`enable` [in]

if =1 enable, if =0 then disable

`pD` [in]

driver private state info specific to this instance

Description

Enable/disable detection of unicast PTP frames.

7.140. `getUnicastPtpDetect`

Function Declaration

```
uint32_t getUnicastPtpDetect(pD, enable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t* enable;
```

Parameter List

`enable` [out]

equal 1 if enabled, equal 0 if disabled

`pD` [in]

driver private state info specific to this instance

Description

Read enable/disable state for detection of unicast PTP frames.

Return Value

EOK for success

EINVAL for invalid pointer

7.141. `setPtpRxUnicastIpAddr`

Function Declaration

```
uint32_t setPtpRxUnicastIpAddr(pD, rxAddr);
```

```
CEDI_PrivateData* pD;
```

```
uint32_t rxAddr;
```

Parameter List

rxAddr [in]

IP destination address

pD [in]

driver private state info specific to this instance

Description

Set Unicast IP destination address for detection of PTP Rx frames.

Unicast PTP frame recognition must be disabled.

Return Value

EOK if successful

ENOTSUP if unicast PTP frame recognition enabled

EINVAL if pD = NULL

7.142. getPtpRxUnicastIpAddr

Function Declaration

```
uint32_t getPtpRxUnicastIpAddr(pD, rxAddr);
```

```
CEDI_PrivateData* pD;
```

```
uint32_t* rxAddr;
```

Parameter List

rxAddr [out]

returns IP destination address

pD [in]

driver private state info specific to this instance

Description

Read Unicast IP destination address for detection of PTP Rx frames.

Return Value

EOK for success

EINVAL for invalid pointer

7.143. setPtpTxUnicastIpAddr

Function Declaration

```
uint32_t setPtpTxUnicastIpAddr(pD, txAddr);

CEDI_PrivateData* pD;
uint32_t txAddr;
```

Parameter List

txAddr [in]
 IP destination address

pD [in]
 driver private state info specific to this instance

Description

Set Unicast IP destination address for detection of PTP Tx frames.

Unicast PTP frame recognition must be disabled.

Return Value

EOK if successful

ENOTSUP if unicast PTP frame recognition enabled

EINVAL if *pD* = NULL

7.144. getPtpTxUnicastIpAddr

Function Declaration

```
uint32_t getPtpTxUnicastIpAddr(pD, txAddr);

CEDI_PrivateData* pD;
uint32_t* txAddr;
```

Parameter List

txAddr [out]
 returns IP destination address

pD [in]
 driver private state info specific to this instance

Description

Returns Unicast IP destination address for detection of PTP Tx frames.

Return Value

EOK for success

EINVAL for invalid pointer

7.145. set1588Timer

Function Declaration

```
uint32_t set1588Timer(pD, timeVal);
```

```
CEDI_PrivateData* pD;  
const CEDI_1588TimerVal* timeVal;
```

Parameter List

pD [in]

driver private state info specific to this instance

timeVal [in]

pointer to CEDI_1588TimerVal struct with values to write to timer registers

Description

Write 1588 timer registers.

Return Value

EOK for success

ENOTSUP if tsu equal 0

EINVAL if either parameter equal NULL, or nanosecs >0x3FFFFFFF

7.146. get1588Timer

Function Declaration

```
uint32_t get1588Timer(pD, timeVal);
```

```
CEDI_PrivateData* pD;  
CEDI_1588TimerVal* timeVal;
```

Parameter List

`pD` [in]

driver private state info specific to this instance

`timeVal` [out]

pointer to a `CEDI_1588TimerVal` struct for returning timer registers contents

Description

Read 1588 timer registers.

Return Value

EOK for success

ENOTSUP if tsu equal 0

EINVAL if either parameter NULL

7.147. `adjust1588Timer`

Function Declaration

```
uint32_t adjust1588Timer(pD, nSecAdjust);
```

```
CEDI_PrivateData* pD;
```

```
int32_t nSecAdjust;
```

Parameter List

`nSecAdjust` [in]

nanoseconds to adjust timer by: - if =1, enable time stamp storing, else disable (restore normal operation)

`pD` [in]

driver private state info specific to this instance

Description

Adjust the 1588 timer by adding or subtracting the specified number of nanoseconds.

Return Value

EOK for success

ENOTSUP if tsu equal 0

EINVAL if invalid parameter

7.148. set1588TimerInc

Function Declaration

```
uint32_t set1588TimerInc(pD, incSettings);
```

```
CEDI_PrivateData* pD;  
const CEDI_TimerIncrement* incSettings;
```

Parameter List

incSettings [in]

pointer to CEDI_TimerIncrement struct for defining how much to increment the timer each clock cycle

pD [in]

driver private state info specific to this instance

Description

Set 1588 timer initial and alternative increments, and when to switch to alternative increment.

The initial, or normal, increment is how many nanoseconds are added to the timer value on each pclk or tsu_clk cycle (if tsu_clk equal 1 in DesignCfgReg5). If altIncCount>0, after altIncCount increments the altNanoSInc value is used for one cycle, then the increment returns to the initial value. See EMAC User Guide [01] for further description.

Return Value

EOK for success

ENOTSUP if tsu equal 0

EINVAL if either parameter equal NULL

7.149. get1588TimerInc

Function Declaration

```
uint32_t get1588TimerInc(pD, incSettings);
```

```
CEDI_PrivateData* pD;  
CEDI_TimerIncrement* incSettings;
```

Parameter List

incSettings [out]

pointer to a CEDI_TimerIncrement struct for returning how the timer increment values

`pD` [in]

driver private state info specific to this instance

Description

Read 1588 timer increment settings.

Return Value

EOK for success

ENOTSUP if tsu equal 0

7.150. setTsuTimerCompVal

Function Declaration

```
uint32_t setTsuTimerCompVal(pD, timeVal);
```

```
CEDI_PrivateData* pD;  
const CEDI_TsuTimerVal* timeVal;
```

Parameter List

`pD` [in]

driver private state info specific to this instance

`timeVal` [in]

pointer to CEDI_TsuTimerVal struct with values to write to comparison registers

Description

Write TSU timer comparison value.

Return Value

EOK for success

ENOTSUP if tsu equal 0, or nanosecs >0x003FFFFFF

7.151. getTsuTimerCompVal

Function Declaration

```
uint32_t getTsuTimerCompVal(pD, timeVal);
```

```
CEDI_PrivateData* pD;
```

```
CEDI_TsuTimerVal* timeVal;
```

Parameter List

pD [in]

driver private state info specific to this instance

timeVal [out]

pointer to a CEDI_TsuTimerVal struct for returning comparison registers settings

Description

Read TSU timer comparison value.

Return Value

EOK for success

ENOTSUP if tsu equal 0

EINVAL if either param equal NULL

7.152. getPtpFrameTxTime

Function Declaration

```
uint32_t getPtpFrameTxTime(pD, timeVal);
```

```
CEDI_PrivateData* pD;
```

```
CEDI_1588TimerVal* timeVal;
```

Parameter List

pD [in]

driver private state info specific to this instance

timeVal [out]

pointer to a CEDI_1588TimerVal struct for returning latched timer registers contents.

Description

Read 1588 timer value latched when SFD of PTP transmit primary event crosses MII interface.

Return Value

EOK for success

ENOTSUP if tsu equal 0

7.153. getPtpFrameRxTime

Function Declaration

```
uint32_t getPtpFrameRxTime(pD, timeVal);
```

```
CEDI_PrivateData* pD;  
CEDI_1588TimerVal* timeVal;
```

Parameter List

pD [in]

driver private state info specific to this instance

timeVal [out]

pointer to a CEDI_1588TimerVal struct for returning latched timer registers contents.

Description

Read 1588 timer value latched when SFD of PTP receive primary event crosses MII interface.

Return Value

EOK for success

ENOTSUP if tsu equal 0

7.154. getPtpPeerFrameTxTime

Function Declaration

```
uint32_t getPtpPeerFrameTxTime(pD, timeVal);
```

```
CEDI_PrivateData* pD;  
CEDI_1588TimerVal* timeVal;
```

Parameter List

pD [in]

driver private state info specific to this instance

timeVal [out]

pointer to a CEDI_1588TimerVal struct for returning latched timer registers contents.

Description

Read 1588 timer value latched when SFD of PTP transmit peer event crosses MII interface.

Return Value

EOK for success

ENOTSUP if tsu equal 0

7.155. getPtpPeerFrameRxTime

Function Declaration

```
uint32_t getPtpPeerFrameRxTime(pD, timeVal);
```

```
CEDI_PrivateData* pD;  
CEDI_1588TimerVal* timeVal;
```

Parameter List

pD [in]

driver private state info specific to this instance

timeVal [out]

pointer to a CEDI_1588TimerVal struct for returning latched timer registers contents.

Description

Read 1588 timer value latched when SFD of PTP receive peer event crosses MII interface.

Return Value

EOK for success

ENOTSUP if tsu equal 0

7.156. get1588SyncStrobeTime

Function Declaration

```
uint32_t get1588SyncStrobeTime(pD, timeVal);
```

```
CEDI_PrivateData* pD;  
CEDI_1588TimerVal* timeVal;
```

Parameter List

pD [in]

driver private state info specific to this instance

timeVal [out]

pointer to a CEDI_1588TimerVal struct for returning latched timer registers contents.

Description

Read nanoseconds and seconds (lo & hi words) from 1588 Timer Sync Strobe registers.

Return Value

EOK for success

ENOTSUP if tsu equal 0

7.157. setExtTsuPortEnable

Function Declaration

```
uint32_t setExtTsuPortEnable(pD, enable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t enable;
```

Parameter List

enable [in]

if =1 then enable, if =0 then disable

pD [in]

driver private state info specific to this instance

Description

Enable/disable use of external time stamp port.

Return Value

EOK for success

ENOTSUP if ext_tsu_timer equal 0

EINVAL if invalid parameter

7.158. getExtTsuPortEnable

Function Declaration

```
uint32_t getExtTsuPortEnable(pD, enable);
```

```
CEDI_PrivateData* pD;
```



```
uint8_t* enable;
```

Parameter List

enable [out]

equal 1 if enabled, equal 0 if disabled

pD [in]

driver private state info specific to this instance

Description

Read enable status for use of external time stamp port.

Return Value

EOK for success

EINVAL for invalid pointer

7.159. set1588OneStepTxSyncEnable

Function Declaration

```
uint32_t set1588OneStepTxSyncEnable(pD, enable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t enable;
```

Parameter List

enable [in]

if =1 then enable, if =0 then disable

pD [in]

driver private state info specific to this instance

Description

Enable/disable one-step Tx sync mode in which timestamp field of every transmitted 1588 sync frame is replaced with the current TSU timestamp.

Return Value

EOK if successful

ENOTSUP if tsu equal 0

EINVAL if invalid parameter

7.160. get1588OneStepTxSyncEnable

Function Declaration

```
uint32_t get1588OneStepTxSyncEnable(pD, enable);

CEDI_PrivateData* pD;
uint8_t* enable;
```

Parameter List

enable [out]

pointer for returning state equal 1 if enabled equal 0 if disabled.

pD [in]

driver private state info specific to this instance

Description

Read enable/disable state of one-step 1588 Tx sync mode.

Return Value

EOK for success

EINVAL for invalid pointer

7.161. setDescTimeStampMode

Function Declaration

```
uint32_t setDescTimeStampMode(pD, txMode, rxMode);

CEDI_PrivateData* pD;
CEDI_TxTsMode txMode;
CEDI_RxTsMode rxMode;
```

Parameter List

txMode [in]

TX Descriptor timestamp insertion mode

rxMode [in]

RX Descriptor timestamp insertion mode

pD [in]

driver private state info specific to this instance

Description

Set the descriptor time stamp mode.

Return Value

EOK for success

EINVAL if pD is NULL/parameters out of range

7.162. getDescTimeStampMode

Function Declaration

```
uint32_t getDescTimeStampMode(pD, txMode, rxMode);
```

```
CEDI_PrivateData* pD;  
CEDI_TxTsMode* txMode;  
CEDI_RxTsMode* rxMode;
```

Parameter List

txMode [out]

TX Descriptor timestamp insertion mode

rxMode [out]

RX Descriptor timestamp insertion mode

pD [in]

driver private state info specific to this instance

Description

Get the descriptor time stamp mode.

Return Value

EOK for success

EINVAL if pD is NULL

7.163. setStoreRxTimeStamp

Function Declaration

```
uint32_t setStoreRxTimeStamp(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t enable;
```

Parameter List

enable [in]
if =1 then enable, if =0 then disable

pD [in]
driver private state info specific to this instance

Description

Enable/disable storing of nanoseconds field of Rx time stamp in CRC field of received frame.

Return Value

EOK for success
ENOTSUP if tsu equal 0

7.164. getStoreRxTimeStamp

Function Declaration

```
uint32_t getStoreRxTimeStamp(pD, enable);  
  
CEDI_PrivateData* pD;  
uint8_t* enable;
```

Parameter List

enable [out]
equal 1 if time stamp storing enabled equal 0 if disabled

pD [in]
driver private state info specific to this instance

Description

Read Enable/disable state for storing nanoseconds Rx time stamp in CRC field of received frame.

Return Value

EOK for success
EINVAL for invalid pointer

7.165. resetPcs

Function Declaration

```
uint32_t resetPcs(pD);
CEDI_PrivateData* pD;
```

Parameter List

pD [in]
driver private state info specific to this instance

Description

Does a PCS reset.

After this, software can monitor getPcsReady to determine when this has finished.

Return Value

EOK for success

ENOTSUP if no_pcs defined

7.166. getPcsReady

Function Declaration

```
uint32_t getPcsReady(pD, ready);
CEDI_PrivateData* pD;
uint8_t* ready;
```

Parameter List

ready [out]
pointer for returning 1 when PCS ready, i.e. reset has gone low or 0 if still in reset.

pD [in]
driver private state info specific to this instance

Description

Read PCS software reset bit, which goes high when resetPcs is called, and returns low when the reset has completed, provided PCS is enabled.

Return Value

EOK for success

ENOTSUP if PCS not enabled

EINVAL for invalid pointer

7.167. startAutoNegotiation

Function Declaration

```
uint32_t startAutoNegotiation(pD, advDat);

CEDI_PrivateData* pD;
const CEDI_AnAdvPage* advDat;
```

Parameter List

advDat [in]
 pointer to CEDI_AnAdvPage struct for setting the advertised abilities register

pD [in]
 driver private state info specific to this instance

Description

Enables and restarts auto-negotiation after writing the specified abilities to the auto-negotiation advertisement register.

After this, normal progress of auto-negotiation will lead to a lpPageRx callback, which includes the ability register data received from the link partner.

Return Value

EOK for success

ENOTSUP if no_pcs defined or in SGMII mode

EBUSY if auto-negotiation already in progress

EINVAL if NULL parameter, invalid struct fields or completion event not enabled

7.168. setAutoNegEnable

Function Declaration

```
uint32_t setAutoNegEnable(pD, enable);

CEDI_PrivateData* pD;
uint8_t enable;
```

Parameter List

enable [in]

if =1 then enable, if =0 then disable

pD [in]

driver private state info specific to this instance

Description

Enable/disable PCS auto-negotiation functionality.

If auto- negotiation is in progress, disabling will halt it. If enabling while underway, an error is returned.

Return Value

EBUSY if enable = 1 when auto-negotiation is under way

EOK for success

ENOTSUP if in SGMII mode

EINVAL if pD equal NULL

7.169. getAutoNegEnable

Function Declaration

```
uint32_t getAutoNegEnable(pD, enable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t* enable;
```

Parameter List

enable [out]

pointer for returning state, equal 1 if enabled equal 0 if disabled.

pD [in]

driver private state info specific to this instance

Description

Read enable/disable of PCS auto-negotiation functionality.

Return Value

EOK for success

ENOTSUP if (CEDI_DesignCfg.no_pcs==1)

EINVAL if NULL parameter

7.170. getLinkStatus

Function Declaration

```
uint32_t getLinkStatus(pD, status);
```

```
CEDI_PrivateData* pD;  
uint8_t* status;
```

Parameter List

status [out]

pointer returning 1 if link up (or sync) 0 if link down.

pD [in]

driver private state info specific to this instance

Description

If auto-negotiation is enabled, this reads the status of connection to link partner, else it reads the synchronisation status.

If link goes down, status remains low until this is read. The driver will remember the low condition when read from register (by this or other PCS functions), and this will only update to current state when this function is called (i.e. after reading the value to return), or when link status is reported via the AnComplete callback. Therefore, if getLinkStatus reports link down, the function should be called again if the current status is required.

Return Value

EOK for success

EINVAL for invalid pointer

7.171. getAnRemoteFault

Function Declaration

```
uint32_t getAnRemoteFault(pD, status);
```

```
CEDI_PrivateData* pD;  
uint8_t* status;
```

Parameter List

status [out]

pointer returning 1 if remote fault indicated by link partner, 0 otherwise.

pD [in]

driver private state info specific to this instance

Description

Read the remote fault status reported by link partner.

Driver will remember the high state when read from register, and only reset when this function is called.

Return Value

EOK for success

EINVAL for invalid pointer

7.172. getAnComplete

Function Declaration

```
uint32_t getAnComplete(pD, status);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t* status;
```

Parameter List

status [out]

pointer returning 1 if auto-negotiation complete 0 if incomplete.

pD [in]

driver private state info specific to this instance

Description

Read the auto-negotiation complete status.

Completion will also be reported by the anComplete callback, provided the CEDI_EV_PCS_AN_COMPLETE interrupt is enabled.

Return Value

EOK for success

ENOTSUP if (CEDI_DesignCfg.no_pcs==1)

EINVAL for invalid pointer

7.173. setAnAdvPage

Function Declaration

```
uint32_t setAnAdvPage(pD, advDat);
```

```
CEDI_PrivateData* pD;
```

```
const CEDI_AnAdvPage* advDat;
```

Parameter List

advDat [in]
 - pointer to advertisement register data

pD [in]
 driver private state info specific to this instance

Description

Set advertisement base page fields.

Return Value

EOK for success

ENOTSUP if (CEDI_DesignCfg.no_pcs==1)

EINVAL if NULL parameter

7.174. getAnAdvPage

Function Declaration

```
uint32_t getAnAdvPage(pD, advDat);
```

```
CEDI_PrivateData* pD;  
CEDI_AnAdvPage* advDat;
```

Parameter List

advDat [out]
 - pointer for returning the advertisement register data

pD [in]
 driver private state info specific to this instance

Description

Get advertisement base page fields, extracted into a CEDI_AnAdvPage struct.

Return Value

EOK for success

ENOTSUP if (CEDI_DesignCfg.no_pcs==1)

EINVAL if NULL parameter

7.175. getLpAbilityPage

Function Declaration

```
uint32_t getLpAbilityPage(pD, lpAbl);
```

```
CEDI_PrivateData* pD;  
CEDI_LpAbilityPage* lpAbl;
```

Parameter List

lpAbl [out]

pointer for returning the link partner ability data

pD [in]

driver private state info specific to this instance

Description

Get link partner ability page fields, extracted into a CEDI_LpAbilityPage struct.

This is a struct of two different structs, since the format depends on whether SGMII mode is in use (network config bit 27)

Return Value

EOK for success

ENOTSUP if (CEDI_DesignCfg.no_pcs==1)

EINVAL if NULL parameter

7.176. getPageRx

Function Declaration

```
uint32_t getPageRx(pD);
```

```
CEDI_PrivateData* pD;
```

Parameter List

pD [in]

driver private state info specific to this instance

Description

Read flag from auto-negotiation expansion register, indicating a base or next page has been received from the link partner.

Return Value

1 if page received and no_pcs not defined

0 if page not received or invalid pointer

7.177. setNextPageTx

Function Declaration

```
uint32_t setNextPageTx(pD, npDat);
```

```
CEDI_PrivateData* pD;  
const CEDI_AnNextPage* npDat;
```

Parameter List

npDat [in]

- pointer to struct containing next page data to transmit

pD [in]

driver private state info specific to this instance

Description

Set next page register fields for auto-negotiation with fields specified in a CEDI_AnNextPage struct.

Return Value

EOK for success

ENOTSUP if (CEDI_DesignCfg.no_pcs==1)

EINVAL if NULL parameter or invalid struct fields

7.178. getNextPageTx

Function Declaration

```
uint32_t getNextPageTx(pD, npDat);
```

```
CEDI_PrivateData* pD;  
CEDI_AnNextPage* npDat;
```

Parameter List

npDat [out]

- pointer to struct for returning next page register data.

pD [in]

driver private state info specific to this instance

Description

Read next page register field data.

Return Value

EOK for success

ENOTSUP if (CEDI_DesignCfg.no_pcs==1)

EINVAL if NULL parameter

7.179. getLpNextPage

Function Declaration

```
uint32_t getLpNextPage(pD, npDat);
```

CEDI_PrivateData* pD;

CEDI_LpNextPage* npDat;

Parameter List

npDat [out]

- pointer to CEDI_LpNextPage struct for returning link partner next page data.

pD [in]

driver private state info specific to this instance

Description

Read next page data received from link partner.

Return Value

EOK for success

ENOTSUP if (CEDI_DesignCfg.no_pcs==1)

EINVAL if NULL parameter

7.180. getPhyId

Function Declaration

```
uint32_t getPhyId(pD, phyId);
```

```
CEDI_PrivateData* pD;  
uint32_t* phyId;
```

Parameter List

pD [in]

driver private state info specific to this instance

phyId [out]

composed of PCS PHY upper ID (= EMAC Module ID) in the upper 16 bits, and PCS PHY lower ID (= EMAC Rev) in the lower 16 bits.

Description

Get PCS PHY upper ID (= EMAC Module ID) & PCS PHY lower ID (= EMAC Rev).

Return Value

EOK for success

EINVAL for invalid pointer

7.181. setMdioEnable

Function Declaration

```
uint32_t setMdioEnable(pD, enable);  
  
CEDI_PrivateData* pD;  
uint8_t enable;
```

Parameter List

enable [in]

if =1 enable MDIO interface, if =0 then disable it

pD [in]

driver private state info specific to this instance

Description

Enable/disable MDIO interface.

Return Value

EOK if successful

EINVAL if invalid parameter

7.182. getMdioEnable

Function Declaration

```
uint32_t getMdioEnable(pD);
```

```
CEDI_PrivateData* pD;
```

Parameter List

pD [in]

driver private state info specific to this instance

Description

Get en/disable state of MDIO interface.

Return Value

1 if MDIO interface enabled

0 if MDIO interface disabled or *pD* equal NULL, or NULL parameter

7.183. phyStartMdioWrite

Function Declaration

```
uint32_t phyStartMdioWrite(pD, flags, phyAddr, devReg, addrData);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t flags;
```

```
uint8_t phyAddr;
```

```
uint8_t devReg;
```

```
uint16_t addrData;
```

Parameter List

addrData [in]

register address (if CEDI_MDIO_FLG_SET_ADDR) or data to write

phyAddr [in]

PHY address

flags [in]

Combination of 2 bit-flags: if CEDI_MDIO_FLG_CLAUSE_45 present, specifies clause 45 PHY (else clause 22), if CEDI_MDIO_FLG_SET_ADDR present, specifies a set address operation (else do a write operation) - if not clause 45, this will be ignored.

<code>pD</code>	[in]
	driver private state info specific to this instance
<code>devReg</code>	[in]
	device type (clause 45) or register address (clause 22) - enum <code>CEDI_MdioDevType</code> is available to specify the device type

Description

Initiate a write or set address operation on the MDIO interface.

Clause 45 devices require a call to set the register address (if this is changing since last access), and then a write or read operation. The command writes to the shift register, which starts output on the MDIO interface. Write completion is signalled by the `phyManComplete` callback, or by polling `getMdioIdle`.

Return Value

EOK if successful

EINVAL if invalid parameter

7.184. `phyStartMdioRead`

Function Declaration

```
uint32_t phyStartMdioRead(pD, flags, phyAddr, devReg);
```

```
CEDI_PrivateData* pD;  
uint8_t flags;  
uint8_t phyAddr;  
uint8_t devReg;
```

Parameter List

<code>phyAddr</code>	[in]
	PHY address
<code>flags</code>	[in]
	Combination of 2 bit-flags: if <code>CEDI_MDIO_FLG_CLAUSE_45</code> present, specifies clause 45 PHY (else clause 22), If <code>CEDI_MDIO_FLG_INC_ADDR</code> present, and clause 45, then address will be incremented after the read operation.
<code>pD</code>	[in]
	driver private state info specific to this instance
<code>devReg</code>	[in]
	device type (clause 45) or register address (clause 22) - enum <code>CEDI_MdioDevType</code> is available to specify the device type

Description

Initiate a read operation on the MDIO interface.

If clause 45, the register address will need to be set by a preceding `phyStartMdioWrite` call, unless same as for last operation. Completion is signalled by the `phyManComplete` callback, which will return the read data by a pointer parameter. Alternatively polling `getMdioIdle` will indicate when the operation completes, then `getMdioReadDat` will return the data.

Return Value

EOK if successful

EINVAL if invalid parameter

7.185. `getMdioReadData`

Function Declaration

```
uint32_t getMdioReadData(pD, readData);

CEDI_PrivateData* pD;
uint16_t* readData;
```

Parameter List

<code>readData</code>	[out]	PHY read data, from in lower 16 bits of maintenance register
<code>pD</code>	[in]	driver private state info specific to this instance

Description

Get data read from the PHY during a read operation.

Return Value

EOK for success

EINVAL for invalid pointer

7.186. `getMdioIdle`

Function Declaration

```
uint32_t getMdioIdle(pD);

CEDI_PrivateData* pD;
```

Parameter List

`pD` [in]

driver private state info specific to this instance

Description

Read status of PHY management logic.

Return Value

1 for idle (mechanical success)

0 for busy, or `pD` equal NULL

7.187. readStats

Function Declaration

```
uint32_t readStats(pD);
```

```
CEDI_PrivateData* pD;
```

Parameter List

`pD` [in]

driver private state info specific to this instance

Description

Copy the statistics registers contents into the `statsRegs` struct reserved by client.

Return Value

EOK for success

ENOTSUP if `no_stats` defined

7.188. clearStats

Function Declaration

```
uint32_t clearStats(pD);
```

```
CEDI_PrivateData* pD;
```

Parameter List

`pD` [in]

driver private state info specific to this instance

Description

Clear the statistics registers.

Return Value

EOK for success

ENOTSUP if no_stats defined

7.189. takeSnapshot

Function Declaration

```
uint32_t takeSnapshot(pD);
```

```
CEDI_PrivateData* pD;
```

Parameter List

pD [in]

driver private state info specific to this instance

Description

Record snapshot of current statistics counts into snapshot registers.

Return Value

EOK for success

ENOTSUP if no_snapshot or no_stats defined

7.190. setReadSnapshot

Function Declaration

```
uint32_t setReadSnapshot(pD, enable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t enable;
```

Parameter List

enable [in]

if =1 then enable, if =0 then disable

pD [in]

driver private state info specific to this instance

Description

Define action taken when statistics registers are read.

Return Value

EOK for success

ENOTSUP if no_snapshot or no_stats defined

7.191. getReadSnapshot

Function Declaration

```
uint32_t getReadSnapshot(pD, enable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t* enable;
```

Parameter List

enable [out]

pointer for returning ReadSnapshot flag: if =1 then read snapshot is enabled; if =0 registers will show current statistics

pD [in]

driver private state info specific to this instance

Description

Read state of ReadSnapshot flag.

Return Value

EOK for success

ENOTSUP if no_snapshot or no_stats defined

7.192. setWakeOnLanReg

Function Declaration

```
uint32_t setWakeOnLanReg(pD, regVals);
```

```
CEDI_PrivateData* pD;
```

```
const CEDI_WakeOnLanReg* regVals;
```

Parameter List

regVals [in]

pointer to CEDI_WakeOnLanReg struct for register values to set

pD [in]

driver private state info specific to this instance

Description

Set Wake On LAN (WOL) register using CEDI_WakeOnLanReg struct.

Return Value

EOK if successful

EINVAL if invalid parameter

7.193. getWakeOnLanReg

Function Declaration

```
uint32_t getWakeOnLanReg(pD, regVals);
```

```
CEDI_PrivateData* pD;
CEDI_WakeOnLanReg* regVals;
```

Parameter List

regVals [out]

pointer to CEDI_WakeOnLanReg struct for returning WOL register contents

pD [in]

driver private state info specific to this instance

Description

Read Wake On LAN register using CEDI_WakeOnLanReg struct.

Return Value

EOK if successful

EINVAL if invalid parameter

7.194. setLpiTxEnable

Function Declaration

```
uint32_t setLpiTxEnable(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t enable;
```

Parameter List

enable [in]

if =1 then enable, if =0 then disable

pD [in]

driver private state info specific to this instance

Description

Enable/disable low power indication (LPI) transmission.

Return Value

EOK if successful

EINVAL if invalid parameter

7.195. getLpiTxEnable

Function Declaration

```
uint32_t getLpiTxEnable(pD, enable);  
  
CEDI_PrivateData* pD;  
uint8_t* enable;
```

Parameter List

enable [out]

equal 1 if enabled, equal 0 if disabled

pD [in]

driver private state info specific to this instance

Description

Get LPI transmission status.

Return Value

EOK for success

EINVAL for invalid parameter

7.196. getLpiStats

Function Declaration

```
uint32_t getLpiStats(pD, lpiStats);

CEDI_PrivateData* pD;
CEDI_LpiStats* lpiStats;
```

Parameter List

lpiStats [out]
 pointer to a CEDI_LpiStats struct for returning the LPI statistics.

pD [in]
 driver private state info specific to this instance

Description

Read LPI transitions & time in both Tx and Rx directions.

The registers are all cleared on read.

Return Value

EOK if successful

EINVAL if invalid parameter

7.197. getDesignConfig

Function Declaration

```
uint32_t getDesignConfig(pD, hwCfg);

const CEDI_PrivateData* pD;
CEDI_DesignCfg* hwCfg;
```

Parameter List

hwCfg [out]
 pointer to struct for receiving configuration data

pD [in]
 driver private state info specific to this instance

Description

Copies all design configuration register fields into a CEDI_DesignCfg struct declared by client software.

Return Value

EOK for success

EINVAL if any parameters =NULL

7.198. setWriteStatsEnable

Function Declaration

```
uint32_t setWriteStatsEnable(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t enable;
```

Parameter List

enable [in]

if =1 enable writing, if =0 then disable

pD [in]

driver private state info specific to this instance

Description

Enable/disable writing to the statistics registers for debugging.

Return Value

EOK for success

EINVAL if no_stats defined

7.199. getWriteStatsEnable

Function Declaration

```
uint32_t getWriteStatsEnable(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t* enable;
```

Parameter List

enable [out]

pointer for returning state: equal 1 if writing enabled equal 0 if disabled.

pD [in]

driver private state info specific to this instance

Description

Read enable/disable state of writing to the statistics registers for debugging.

Return Value

EOK for success

EINVAL if no_stats defined

7.200. incStatsRegs

Function Declaration

```
uint32_t incStatsRegs(pD);
```

```
CEDI_PrivateData* pD;
```

Parameter List

pD [in]

driver private state info specific to this instance

Description

Increments all statistics registers contents by 1 for debugging.

Return Value

EOK for success

EINVAL if no_stats defined

7.201. setRxBackPressure

Function Declaration

```
void setRxBackPressure(pD, enable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t enable;
```

Parameter List

enable [in]

if =1 enable back pressure, if =0 then disable

`pD` [in]
 driver private state info specific to this instance

Description

Enable/disable back pressure to force collisions an all received frames in 10M or 100M half duplex mode.

7.202. getRxBackPressure

Function Declaration

```
uint32_t getRxBackPressure(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t* enable;
```

Parameter List

`enable` [out]
 equal 1 if enabled, equal 0 if disabled

`pD` [in]
 driver private state info specific to this instance

Description

Read Enable/disable state of back pressure to force collisions an all received frames in 10M or 100M half duplex mode.

Return Value

EOK for success

EINVAL for invalid pointer

7.203. setCollisionTest

Function Declaration

```
uint32_t setCollisionTest(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t enable;
```

Parameter List

`enable` [in]
 if =1 enable forcing collisions, if =0 then disable

`pD` [in]

driver private state info specific to this instance

Description

Enable/disable forcing collisions by PCS on transmit.

Return Value

EOK for success

ENOTSUP if `no_pcs` defined

7.204. `getCollisionTest`

Function Declaration

```
uint32_t getCollisionTest(pD, enable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t* enable;
```

Parameter List

`enable` [out]

pointer for returning state: equal 1 if forcing collisions enabled, equal 0 if disabled.

`pD` [in]

driver private state info specific to this instance

Description

Read enable/disable state for forcing collisions by PCS on transmit.

Return Value

EOK for success

ENOTSUP if `no_pcs` defined

7.205. `setRetryTest`

Function Declaration

```
void setRetryTest(pD, enable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t enable;
```

Parameter List

enable [in]

if =1 enable retry test, if =0 then disable

pD [in]

driver private state info specific to this instance

Description

Enable/disable functionality to set backoff between collisions to one slot time.

This helps test for too many retries, and also shortens pause countdown time in pause frame tests.

7.206. getRetryTest

Function Declaration

```
uint32_t getRetryTest(pD, enable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t* enable;
```

Parameter List

enable [out]

=1 if retry test enabled, =0 if disabled

pD [in]

driver private state info specific to this instance

Description

Read enable/disable status of retry test functionality.

Return Value

EOK for success

EINVAL for invalid pointer

7.207. writeUserOutputs

Function Declaration

```
uint32_t writeUserOutputs(pD, outVal);
```

```
CEDI_PrivateData* pD;  
uint16_t outVal;
```

Parameter List

pD [in]
 driver private state info specific to this instance

outVal [in]
 value to write to the user outputs

Description

Write to the user_out_width user outputs.

Return Value

EOK for success

ENOTSUP if user_io not defined

EINVAL if *pD* equal NULL

7.208. readUserOutputs

Function Declaration

```
uint32_t readUserOutputs(pD, outVal);  
  
CEDI_PrivateData* pD;  
uint16_t* outVal;
```

Parameter List

pD [in]
 driver private state info specific to this instance

outVal [out]
 pointer for returning value read from the user outputs

Description

Read from the user_out_width user outputs.

Return Value

EOK for success

ENOTSUP if user_io not defined

EINVAL if NULL parameters

7.209. setUserOutPin

Function Declaration

```
uint32_t setUserOutPin(pD, pin, state);
```

```
CEDI_PrivateData* pD;  
uint8_t pin;  
uint8_t state;
```

Parameter List

state [in]

level to set on the pin 1 for high, 0 for low

pin [in]

pin number to set, range 0 to user_out_width-1

pD [in]

driver private state info specific to this instance

Description

Set the state of the specified user output pin.

Return Value

EOK for success

ENOTSUP if user_io not defined

EINVAL if pin out of range, or pD equal NULL

7.210. readUserInputs

Function Declaration

```
uint32_t readUserInputs(pD, inVal);
```

```
CEDI_PrivateData* pD;  
uint16_t* inVal;
```

Parameter List

pD [in]

driver private state info specific to this instance

`inVal` [out]

pointer for returning value read from the user inputs

Description

Read from the `user_in_width` user inputs.

Return Value

EOK for success

ENOTSUP if `user_io` not defined

EINVAL if NULL parameter

7.211. getUserInPin

Function Declaration

```
uint32_t getUserInPin(pD, pin, state);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t pin;
```

```
uint8_t* state;
```

Parameter List

`state` [out]

pointer for returning level present on the pin, 1 for high, 0 for low

`pin` [in]

pin number to read

`pD` [in]

driver private state info specific to this instance

Description

Get the state of the specified user input pin.

Return Value

EOK for success

ENOTSUP if `user_io` not defined

EINVAL if pin out of range, or pD equal NULL

7.212. getMdioInState

Function Declaration

```
uint32_t getMdioInState(pD, state);
```

```
CEDI_PrivateData* pD;  
uint8_t* state;
```

Parameter List

state [out]

1 for high, 0 for low

pD [in]

driver private state info specific to this instance

Description

Read the state of the MDIO pin.

Return Value

EOK for success

EINVAL for invalid pointer

7.213. readReg

Function Declaration

```
uint32_t readReg(pD, offs, data);
```

```
const CEDI_PrivateData* pD;  
uint32_t offs;  
uint32_t* data;
```

Parameter List

offs [in]

address offset to register

data [out]

pointer for returning data read

pD [in]

driver private state info specific to this instance

Description

Direct read of register contents.

Return Value

EOK for success

EINVAL if offs out of range, or pD equal NULL

7.214. writeReg

Function Declaration

```
uint32_t writeReg(pD, offs, data);

const CEDI_PrivateData* pD;
uint32_t offs;
uint32_t data;
```

Parameter List

offs [in]

address offset to register

data [in]

data to write

pD [in]

driver private state info specific to this instance

Description

Direct write to register.

Return Value

EOK for success

EINVAL if offs out of range, or pD equal NULL

7.215. setTxQueueNum

Function Declaration

```
uint32_t setTxQueueNum(pD, numQueues);
```

```
CEDI_PrivateData* pD;  
uint8_t numQueues;
```

Parameter List

<i>numQueues</i>	[in]	number of TX priority queues
<i>pD</i>	[in]	driver private state info specific to this instance

Description

Change TX priority queues number.

Return Value

EOK for success

EINVAL if number of queues is bigger than available, or *pD* equal NULL

7.216. setRxQueueNum

Function Declaration

```
uint32_t setRxQueueNum(pD, numQueues);  
  
CEDI_PrivateData* pD;  
uint8_t numQueues;
```

Parameter List

<i>numQueues</i>	[in]	number of RX priority queues
<i>pD</i>	[in]	driver private state info specific to this instance

Description

Change RX priority queues number.

Return Value

EOK for success

EINVAL if number of queues is bigger than available, or *pD* equal NULL

7.217. setTxQueueSegAlloc

Function Declaration

```
uint32_t setTxQueueSegAlloc(pD, queueSegAlloc);

CEDI_PrivateData* pD;
const CEDI_QueueSegAlloc* queueSegAlloc;
```

Parameter List

<i>queueSegAlloc</i>	[in]	new configuration of segment allocation
<i>pD</i>	[in]	driver private state info specific to this instance

Description

configure number of memory segments assigned to a Tx queue

Return Value

EOK for success

EINVAL if segment allocation configuration is wrong, or *pD* equal NULL

7.218. setTxQueueScheduler

Function Declaration

```
uint32_t setTxQueueScheduler(pD, queueNum, schedType);

CEDI_PrivateData* pD;
uint8_t queueNum;
CEDI_TxSchedType schedType;
```

Parameter List

<i>schedType</i>	[in]	new scheduler type
<i>pD</i>	[in]	driver private state info specific to this instance
<i>queueNum</i>	[in]	priority queue to configure

Description

function configures scheduler options for a TX queue.

It is possible to select one of four schedulers.

Return Value

EOK for success

EINVAL if TX queue scheduler configuration is wrong, or pD equal NULL

7.219. getTxQueueScheduler

Function Declaration

```
uint32_t getTxQueueScheduler(pD, queueNum, schedType);

CEDI_PrivateData* pD;
uint8_t queueNum;
CEDI_TxSchedType* schedType;
```

Parameter List

<i>schedType</i>	[out]	returned scheduler type
<i>pD</i>	[in]	driver private state info specific to this instance
<i>queueNum</i>	[in]	priority queue to read configuration

Description

get scheduler type for a TX queue.

Return Value

EOK for success

EINVAL for invalid pointer

7.220. setDwrrWeighting

Function Declaration

```
uint32_t setDwrrWeighting(pD, queueNum, dwrrWeightingVal);

CEDI_PrivateData* pD;
uint8_t queueNum;
```

```
uint8_t dwrrWeightingVal;
```

Parameter List

<i>dwrrWeightingVal</i>	[in]	new dwrr weighting value
<i>pD</i>	[in]	driver private state info specific to this instance
<i>queueNum</i>	[in]	priority queue to configure

Description

function configures DWRR weight for DWRR scheduling algorithm for a queue

Return Value

EOK for success

EINVAL if *pD* equal NULL

7.221. setEtsBandAlloc

Function Declaration

```
uint32_t setEtsBandAlloc(pD, queueNum, etsBandAlloc);
```

```
CEDI_PrivateData* pD;  
uint8_t queueNum;  
uint8_t etsBandAlloc;
```

Parameter List

<i>etsBandAlloc</i>	[in]	new ets bandwidth allocation value
<i>pD</i>	[in]	driver private state info specific to this instance
<i>queueNum</i>	[in]	priority queue to configure ETS bandwidth allocation

Description

function configures ETS bandwidth allocation for ETS scheduling algorithm for a queue

Return Value

EOK for success

EINVAL if pD equal NULL

7.222. getDwrrWeighting

Function Declaration

```
uint32_t getDwrrWeighting(pD, queueNum, dwrrWeightingVal);
```

```
CEDI_PrivateData* pD;  
uint8_t queueNum;  
uint8_t* dwrrWeightingVal;
```

Parameter List

dwrrWeightingVal	[out]	
		returned dwrr weighting value
pD	[in]	
		driver private state info specific to this instance
queueNum	[in]	
		priority queue to get value from

Description

function gets DWRR weight for DWRR scheduling algorithm for a queue

Return Value

EOK for success

EINVAL if pD equal NULL

7.223. getEtsBandAlloc

Function Declaration

```
uint32_t getEtsBandAlloc(pD, queueNum, etsBandAlloc);
```

```
CEDI_PrivateData* pD;  
uint8_t queueNum;  
uint8_t* etsBandAlloc;
```

Parameter List

etsBandAlloc	[out]
--------------	---------

	returned ets bandwidth allocation value
pD	[in] driver private state info specific to this instance
queueNum	[in] priority queue to get ETS bandwidth allocation value from

Description

function gets ETS bandwidth allocation for ETS scheduling algorithm for a queue

Return Value

EOK for success

EINVAL if pD equal NULL

7.224. setEnstTimeConfig

Function Declaration

```
uint32_t setEnstTimeConfig(pD, queueNum, enstTimeConfig);
```

```
CEDI_PrivateData* pD;  
uint8_t queueNum;  
const CEDI_EnstTimeConfig* enstTimeConfig;
```

Parameter List

enstTimeConfig	[in] new timing configuration
pD	[in] driver private state info specific to this instance
queueNum	[in] priority queue to configure

Description

set timing configuration for Enhancement for Scheduled Traffic

Return Value

EOK for success

EINVAL if TX queue time scheduler configuration is wrong, or pD equal NULL

7.225. getEnstTimeConfig

Function Declaration

```
uint32_t getEnstTimeConfig(pD, queueNum, enstTimeConfig);
```

```
CEDI_PrivateData* pD;  
uint8_t queueNum;  
CEDI_EnstTimeConfig* enstTimeConfig;
```

Parameter List

<i>enstTimeConfig</i>	[out]	new timing configuration
<i>pD</i>	[in]	driver private state info specific to this instance
<i>queueNum</i>	[in]	priority queue to get configuration

Description

get timing configuration for Enhancement for Scheduled Traffic

Return Value

EOK for success

EINVAL if *pD* equal NULL

7.226. setEnstEnable

Function Declaration

```
uint32_t setEnstEnable(pD, queue, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t queue;  
uint8_t enable;
```

Parameter List

<i>queue</i>	[in]	priority queue to enable/disable ENST time scheduling
<i>enable</i>	[in]	if 0 then disable ENST, if 1 then enable ENST

`pD` [in]
driver private state info specific to this instance

Description

Function enables or disables ENST time scheduling for queue.

Return Value

EOK for success

EINVAL if `pD` equal NULL or if ENST is not supported for particular queue

7.227. `getEnstEnable`

Function Declaration

```
uint32_t getEnstEnable(pD, queue, enable);

CEDI_PrivateData* pD;
uint8_t queue;
uint8_t* enable;
```

Parameter List

`queue` [in]
priority queue to get status of ENST time scheduling

`enable` [out]
if 1 then ENST is enabled, if 0 then ENST is disabled

`pD` [in]
driver private state info specific to this instance

Description

Function gets configuration of ENST time scheduling for queue state can be 1 - means enabled, 0 - means disabled.

Return Value

EOK for success

EINVAL if `pD` equal NULL or if ENST is not supported for particular queue

7.228. `getEnstSupported`

Function Declaration

```
uint32_t getEnstSupported(pD, supported);
```

```
CEDI_PrivateData* pD;  
uint8_t* supported;
```

Parameter List

supported [out]

value "1" means that EnST is supported. "0" otherwise

pD [in]

driver private state info specific to this instance

Description

Functions checks if core supports EnST.

Return Value

EOK for success

EINVAL if *pD* equal NULL or if *supported* equal NULL

7.229. setReportingBadFCS

Function Declaration

```
uint32_t setReportingBadFCS(pD, enable);
```

```
CEDI_PrivateData* pD;  
uint8_t enable;
```

Parameter List

enable [in]

if 1 enable reporting FCS/CRC, if 0 disable reporting FCS/CRC

pD [in]

driver private state info specific to this instance

Description

function configures report FCS/CRC error in receive buffer descriptor,

Return Value

EOK for success

EINVAL if *pD* equal NULL

7.230. getReportingBadFCS

Function Declaration

```
uint32_t getReportingBadFCS(pD, enable);

CEDI_PrivateData* pD;
uint8_t* enable;
```

Parameter List

enable [out]
if 1 reporting FCS/CRC i enabled, if 0 reporting FCS/CRC is disabled

pD [in]
driver private state info specific to this instance

Description

Function gets state of reporting FCS/CRC error in receive buffer descriptor,.

Return Value

EOK for success
EINVAL if *pD* equal NULL

7.231. setPtpSingleStep

Function Declaration

```
uint32_t setPtpSingleStep(pD, enable);

CEDI_PrivateData* pD;
uint8_t enable;
```

Parameter List

enable [in]
if 1 enable ptp one step correction, if 0 disable PTP one step correction

pD [in]
driver private state info specific to this instance

Description

Function sets enables or disables 1588 One Step Correction Field Update feature in controller,.

Return Value

EOK for success

EINVAL if pD equal NULL

7.232. getPtpSingleStep

Function Declaration

```
uint32_t getPtpSingleStep(pD, enable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t* enable;
```

Parameter List

enable [out]

if 1 means that ptp one step correction is enabled, if 0 means that ptp one step correction is disabled

pD [in]

driver private state info specific to this instance

Description

Function gets configuration enables of 1588 One Step Correction Field Update feature.

Return Value

EOK for success

EINVAL if pD equal NULL

7.233. setMiiOnRgmii

Function Declaration

```
uint32_t setMiiOnRgmii(pD, enable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t enable;
```

Parameter List

enable [in]

1 - enable MII operation on RGMII interface, 0 - enable MII operation on RGMII interface

pD [in]

driver private state info specific to this instance

Description

Function enables or disables feature whis allows to execute MII operation on RGMII interface.

Return Value

EOK for success

EINVAL if pD equal NULL

7.234. getMiiOnRgmii

Function Declaration

```
uint32_t getMiiOnRgmii(pD, enable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t* enable;
```

Parameter List

enable [out]

1 - MII operation on RGMII interface is enabled, 0 - MII operation on RGMII interface is disabled

pD [in]

driver private state info specific to this instance

Description

Function gets configuration of feature allowing to execute MII operation on RGMII interface.

Return Value

EOK for success

EINVAL if pD equal NULL

7.235. setCbsIdleSlope

Function Declaration

```
uint32_t setCbsIdleSlope(pD, queueNum, idleSlope);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t queueNum;
```

```
uint32_t idleSlope;
```

Parameter List

idleSlope [in]

	new idle slop value
<code>pD</code>	[in]
	driver private state info specific to this instance
<code>queueNum</code>	[in]
	priority queue to set new idle slop value

Description

Function sets a new idle slope value for the queue.

Return Value

EOK for success

EINVAL if there is only one active queue

7.236. getCbsIdleSlope

Function Declaration

```
uint32_t getCbsIdleSlope(pD, queueNum, idleSlope);

CEDI_PrivateData* pD;
uint8_t queueNum;
uint32_t* idleSlope;
```

Parameter List

<code>idleSlope</code>	[out]
	new idle slop value
<code>pD</code>	[in]
	driver private state info specific to this instance
<code>queueNum</code>	[in]
	priority queue to set new idle slop value

Description

Function gets idle slope value for the queue.

Return Value

EOK for success

EINVAL if there is only one active queue

7.237. getLinkFaultIndication

Function Declaration

```
uint32_t getLinkFaultIndication(pD, linkFault);

CEDI_PrivateData* pD;
CEDI_LinkFaultIndication* linkFault;
```

Parameter List

linkFault	[out]
	pointer to read link fault status
pD	[in]
	driver private state info specific to this instance

Description

Function gets link fault indication status.

Return Value

EOK for success

ENOTSUP if feature is not available

EINVAL for invalid linkFault

7.238. setFrameEliminationTagSize

Function Declaration

```
uint32_t setFrameEliminationTagSize(pD, tagSize);

CEDI_PrivateData* pD;
CEDI_RedundancyTagSize tagSize;
```

Parameter List

tagSize	[in]
	tag size 4 or 6 as enum
pD	[in]
	driver private state info specific to this instance

Description

Function sets frame elimination tag size (4 or 6).

Return Value

EOK for success

ENOTSUP if feature is not available

EINVAL for invalid tagSize

7.239. getFrameEliminationTagSize

Function Declaration

```
uint32_t getFrameEliminationTagSize(pD, tagSize);
```

```
CEDI_PrivateData* pD;
```

```
CEDI_RedundancyTagSize* tagSize;
```

Parameter List

tagSize [out]

pointer to read current tag size

pD [in]

driver private state info specific to this instance

Description

Function gets frame elimination tag size.

Return Value

EOK for success

ENOTSUP if feature is not available

EINVAL if tagSize equal NULL

7.240. setFrameEliminationEnable

Function Declaration

```
uint32_t setFrameEliminationEnable(pD, queueNum, enable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t queueNum;
```



```
uint8_t enable;
```

Parameter List

enable [in]
 if 1 then enable frame elimination, if 0 disable it.

pD [in]
 driver private state info specific to this instance

queueNum [in]
 priority queue to enable disable elimination frames

Description

Function enable/disable elimination of received frames functionality.

Return Value

EOK for success

EINVAL for invalid queueNum

7.241. getFrameEliminationEnable

Function Declaration

```
uint32_t getFrameEliminationEnable(pD, queueNum, enable);

const CEDI_PrivateData* pD;
uint8_t queueNum;
uint8_t* enable;
```

Parameter List

enable [out]
 if 1 then frame elimination is enabled, if 0 it is disabled

pD [in]
 driver private state info specific to this instance

queueNum [in]
 priority queue to check elimination frames

Description

Function returns state of elimination of received frames functionality.

Return Value

EOK for success

EINVAL for invalid queueNum

7.242. setFrameEliminationSeqRecRstTmrEnable

Function Declaration

```
uint32_t setFrameEliminationSeqRecRstTmrEnable(pD, queueNum, enable);

const CEDI_PrivateData* pD;
uint8_t queueNum;
uint8_t enable;
```

Parameter List

enable [in]

1 - enable sequence recovery reset timer. 0 otherwise.

pD [in]

driver private state info specific to this instance

queueNum [in]

priority queue to enable/disable elimination frames

Description

Function enable/disable sequence recovery reset timer for elimination of received frames functionality.

Return Value

EOK for success

EINVAL for invalid queueNum

7.243. setFrameEliminationSeqRecRstTmrEnable

Function Declaration

```
uint32_t setFrameEliminationSeqRecRstTmrEnable(pD, queueNum, enable);

const CEDI_PrivateData* pD;
uint8_t queueNum;
uint8_t* enable;
```

Parameter List

enable [out]

1 - sequence recovery reset timer is enabled. 0 otherwise.

pD [in]

driver private state info specific to this instance

queueNum [in]

priority queue to check elimination frames

Description

Function returns state of sequence recovery reset timer for elimination of received frames functionality.

Return Value

EOK for success

EINVAL for invalid queueNum

7.244. setFrameEliminationConfig

Function Declaration

```
uint32_t setFrameEliminationConfig(pD, queueNum, fec);

const CEDI_PrivateData* pD;
uint8_t queueNum;
const CEDI_FrameEliminationConfig* fec;
```

Parameter List

fec [in]

configuration of frame elimination

pD [in]

driver private state info specific to this instance

queueNum [in]

priority queue to configure elimination frames feature

Description

Function sets set configuration of frame elimination feature.

Return Value

EOK for success

EINVAL for invalid queueNum

7.245. getFrameEliminationConfig

Function Declaration

```
uint32_t getFrameEliminationConfig(pD, queueNum, fec);

const CEDI_PrivateData* pD;
uint8_t queueNum;
CEDI_FrameEliminationConfig* fec;
```

Parameter List

<i>fec</i>	[out]	configuration of frame elimination
<i>pD</i>	[in]	driver private state info specific to this instance
<i>queueNum</i>	[in]	priority queue to get frame elimination configuration

Description

Function gets configuration of frame elimination feature.

Return Value

EOK for success

EINVAL for invalid queueNum

7.246. setFrameEliminationTagConfig

Function Declaration

```
uint32_t setFrameEliminationTagConfig(pD, fetc);

CEDI_PrivateData* pD;
const CEDI_FrameEliminationTagConfig* fetc;
```

Parameter List

<i>fetc</i>	[in]	tag configuration of frame elimination
<i>pD</i>	[in]	

driver private state info specific to this instance

Description

Function sets tag configuration of frame elimination feature.

Return Value

EOK for success

EINVAL if fetc equal NULL

7.247. setFrameEliminationTagConfig

Function Declaration

```
uint32_t setFrameEliminationTagConfig(pD, fetc);

CEDI_PrivateData* pD;
CEDI_FrameEliminationTagConfig* fetc;
```

Parameter List

fetc [out]

tag configuration of frame elimination

pD [in]

driver private state info specific to this instance

Description

Function gets tag configuration of frame elimination feature.

Return Value

EOK for success

EINVAL if fetc equal NULL

7.248. setFrameEliminationTimeoutConfig

Function Declaration

```
uint32_t setFrameEliminationTimeoutConfig(pD, timeout);

CEDI_PrivateData* pD;
uint16_t timeout;
```

Parameter List

<code>timeout</code>	[in]
	Count of 8192 rx_clk periods
<code>pD</code>	[in]
	driver private state info specific to this instance

Description

Function sets configuration of timeout period.

Return Value

EOK for success

EINVAL if pD equal NULL

7.249. getFrameEliminationStats

Function Declaration

```
uint32_t getFrameEliminationStats(pD, queueNum, stats);

const CEDI_PrivateData* pD;
uint8_t queueNum;
CEDI_FrameEliminationStats* stats;
```

Parameter List

<code>stats</code>	[out]
	Frame elimination statistics
<code>pD</code>	[in]
	driver private state info specific to this instance
<code>queueNum</code>	[in]
	priority queue to get statistics of FRER

Description

Function gets frame elimination statistics.

Return Value

EOK for success

EINVAL for invalid queueNum

7.250. getEmac

Function Declaration

```
uint32_t getEmac(pD, *emacPrivateData);

CEDI_PrivateData* pD;
CEDI_PrivateData* *emacPrivateData;
```

Parameter List

emacPrivateData	[out]	express MAC private data object
pD	[in]	driver private state info specific to this instance

Description

Function gets express MAC object.

Return Value

EOK for success

EINVAL if pD equal NULL

7.251. getPmac

Function Declaration

```
uint32_t getPmac(pD, *pmacPrivateData);

CEDI_PrivateData* pD;
CEDI_PrivateData* *pmacPrivateData;
```

Parameter List

pmacPrivateData	[out]	pre-emption MAC private data object
pD	[in]	driver private state info specific to this instance

Description

Function gets pre-emptive MAC object.

Return Value

EOK for success

EINVAL if pD equal NULL

7.252. getMacType

Function Declaration

```
uint32_t getMacType(pD, macType);
```

```
const CEDI_PrivateData* pD;
```

```
CEDI_MacType* macType;
```

Parameter List

macType [out]

type of MAC private data object

pD [in]

driver private state info specific to this instance

Description

Function gets type of private data object.

Return Value

EOK for success

EINVAL if macType equal NULL

7.253. setPreemptionConfig

Function Declaration

```
uint32_t setPreemptionConfig(pD, preCfg);
```

```
CEDI_PrivateData* pD;
```

```
const CEDI_PreemptionConfig* preCfg;
```

Parameter List

preCfg [in]

configuration of preemption

pD [in]

driver private state info specific to this instance

Description

Function sets configuration of preemption feature.

Return Value

EOK for success

EINVAL if preCfg equal NULL

7.254. getPreemptionConfig

Function Declaration

```
uint32_t getPreemptionConfig(pD, preCfg);
```

CEDI_PrivateData* *pD*;

CEDI_PreemptionConfig* *preCfg*;

Parameter List

preCfg [out]

read configuration of preemption

pD [in]

driver private state info specific to this instance

Description

Function gets configuration of preemption feature.

Return Value

EOK for success

EINVAL if preCfg equal NULL

7.255. preemptionRestartVerification

Function Declaration

```
uint32_t preemptionRestartVerification(pD);
```

CEDI_PrivateData* *pD*;

Parameter List

pD [in]

driver private state info specific to this instance

Description

Function initiate the verification procedure.

Return Value

EOK for success

EINVAL if pD equal NULL

7.256. setMmslEventEnable

Function Declaration

```
uint32_t setMmslEventEnable(pD, events, enable);
```

```
CEDI_PrivateData* pD;
```

```
uint32_t events;
```

```
uint8_t enable;
```

Parameter List

enable [in]

if =1 enable the events, if =0 then disable

events [in]

OR'd combination of bit-flags selecting the events to be enabled or disabled

pD [in]

driver private state info specific to this instance

Description

Enable or disable the specified MMSL interrupts.

Return Value

EOK for success

EINVAL if pD equal NULL

7.257. getMmslEventEnable

Function Declaration

```
uint32_t getMmslEventEnable(pD, events);
```

```
CEDI_PrivateData* pD;
```

```
uint32_t* events;
```

Parameter List

events [out]
 returned enabled events

pD [in]
 driver private state info specific to this instance

Description

Read the specified MMSL interrupts OR the returned event value with the event bit-flags to determine if event(s) are enabled.

Return Value

EOK for success

EINVAL if pD equal NULL

7.258. readMmslStats

Function Declaration

```
uint32_t readMmslStats(pD, mmslStats);
```

```
CEDI_PrivateData* pD;
```

```
CEDI_MmslStats* mmslStats;
```

Parameter List

mmslStats [out]
 mmsl statistics

pD [in]
 driver private state info specific to this instance

Description

reads MMSL statistics

Return Value

EOK for success

ENOTSUP if no_stats defined

EINVAL if pD equal NULL

7.259. readMmslStatus

Function Declaration

```
uint32_t readMmslStatus(pD, mmslStatus);

CEDI_PrivateData* pD;
CEDI_MmslStatus* mmslStatus;
```

Parameter List

<i>mmslStatus</i>	[out]	
		mmsl status
<i>pD</i>	[in]	
		driver private state info specific to this instance

Description

reads MMSL status

Return Value

EOK for success

ENOTSUP if no_stats defined

EINVAL if *pD* equal NULL

7.260. setLockupConfig

Function Declaration

```
uint32_t setLockupConfig(pD, lockupCfg);

CEDI_PrivateData* pD;
const CEDI_LockupConfig* lockupCfg;
```

Parameter List

<i>lockupCfg</i>	[in]	
		lockup detection and recovery configuration
<i>pD</i>	[in]	
		driver private state info specific to this instance

Description

Function sets lockup detection and recovery configuration.

Return Value

EOK for success

EINVAL if lockupCfg equal NULL

7.261. getLockupConfig

Function Declaration

```
uint32_t getLockupConfig(pD, lockupCfg);
```

CEDI_PrivateData* *pD*;

CEDI_LockupConfig* *lockupCfg*;

Parameter List

lockupCfg [out]

lockup detection and recovery configuration

pD [in]

driver private state info specific to this instance

Description

Function gets lockup detection and recovery configuration.

Return Value

EOK for success

EINVAL if lockupCfg equal NULL

7.262. setType1ScreenRegDropEnable

Function Declaration

```
uint32_t setType1ScreenRegDropEnable(pD, regNum, enable);
```

CEDI_PrivateData* *pD*;

uint8_t *regNum*;

uint8_t *enable*;

Parameter List

regNum [in]

Type 1 register number, range 0 to CEDI_DesignCfg.num_type1_screeners-1

enable [in]

if =1 then enable dropping, if =0 then disable dropping.

pD [in]

driver private state info specific to this instance

Description

Enable or disable dropping frames matching given type 1 screening register.

Return Value

EOK for success

ENOTSUP if feature is not available

EINVAL for invalid arguments values or NULL pointers

7.263. getType1ScreenRegDropEnable

Function Declaration

```
uint32_t getType1ScreenRegDropEnable(pD, regNum, enable);
```

```
CEDI_PrivateData* pD;
```

```
uint8_t regNum;
```

```
uint8_t* enable;
```

Parameter List

regNum [in]

Type 1 register number, range 0 to CEDI_DesignCfg.num_type1_screeners-1

enable [out]

if =1 then enable dropping, if =0 then disable dropping.

pD [in]

driver private state info specific to this instance

Description

Get status (enabled/disabled) of dropping frames matching given type 1 screening registers.

Return Value

EOK for success

ENOTSUP if feature is not available

EINVAL for invalid arguments values or NULL pointers

7.264. setType2ScreenRegDropEnable

Function Declaration

```
uint32_t setType2ScreenRegDropEnable(pD, regNum, enable);

CEDI_PrivateData* pD;
uint8_t regNum;
uint8_t enable;
```

Parameter List

regNum [in]
Type 2 register number, range 0 to CEDI_DesignCfg.num_type2_screeners-1

enable [in]
if =1 then enable dropping, if =0 then disable dropping.

pD [in]
driver private state info specific to this instance

Description

Enable or disable dropping frames matching given type 2 screening register.

Return Value

EOK for success

ENOTSUP if feature is not available

EINVAL for invalid arguments values or NULL pointers

7.265. getType2ScreenRegDropEnable

Function Declaration

```
uint32_t getType2ScreenRegDropEnable(pD, regNum, enable);

CEDI_PrivateData* pD;
uint8_t regNum;
uint8_t* enable;
```

Parameter List

regNum [in]
Type 2 register number, range 0 to CEDI_DesignCfg.num_type2_screeners-1

enable [out]

if =1 then enable dropping, if =0 then disable dropping.

pD [in]

driver private state info specific to this instance

Description

Get status (enabled/disabled) of dropping frames matching given type 2 screening registers.

Return Value

EOK for success

ENOTSUP if feature is not available

EINVAL for invalid arguments values or NULL pointers

7.266. setRxQFlushConfig

Function Declaration

```
uint32_t setRxQFlushConfig(pD, queueNum, rxQFlushConfig);
```

CEDI_PrivateData* *pD*;

uint8_t *queueNum*;

CEDI_RxQFlushConfig* *rxQFlushConfig*;

Parameter List

rxQFlushConfig [in]

Rx queue flush configuration.

pD [in]

driver private state info specific to this instance.

queueNum [in]

number of the Rx queue.

Description

Sets RX queue flush configuration.

Return Value

EOK for success

ENOTSUP if feature is not available

EINVAL for invalid arguments values or NULL pointers

7.267. getRxQFlushConfig

Function Declaration

```
uint32_t getRxQFlushConfig(pD, queueNum, rxQFlushConfig);

CEDI_PrivateData* pD;
uint8_t queueNum;
CEDI_RxQFlushConfig* rxQFlushConfig;
```

Parameter List

<i>rxQFlushConfig</i>	[out]	Rx queue flush configuration.
<i>pD</i>	[in]	driver private state info specific to this instance.
<i>queueNum</i>	[in]	number of the Rx queue.

Description

Gets RX queue flush configuration.

Return Value

EOK for success

ENOTSUP if feature is not available

EINVAL for invalid arguments values or NULL pointers

7.268. getRxDmaFlushedPacketsCount

Function Declaration

```
uint32_t getRxDmaFlushedPacketsCount(pD, count);

CEDI_PrivateData* pD;
uint16_t* count;
```

Parameter List

<i>count</i>	[out]	number of flushed RX packets.
--------------	---------	-------------------------------

`pD` [in]

driver private state info specific to this instance.

Description

Gets count of flushed RX packets.

Return Value

EOK for success

ENOTSUP if feature is not available

EINVAL for invalid arguments values or NULL pointers

7.269. setType2ScreenerRateLimit

Function Declaration

```
uint32_t setType2ScreenerRateLimit(pD, regNum, rateLimit);
```

`CEDI_PrivateData* pD;`

`uint8_t regNum;`

`CEDI_Type2ScreenerRateLimit* rateLimit;`

Parameter List

`regNum` [in]

Type 2 register number, range 0 to `CEDI_DesignCfg.num_type2_screeners-1`

`rateLimit` [in]

Configuration of rate limiting for given screening register.

`pD` [in]

driver private state info specific to this instance.

Description

Sets maximum receive data rate for traffic matching type 2 screening register.

Setting interval time to '0' disables limiting for given type 2 screener.

Return Value

EOK for success

ENOTSUP if feature is not available

EINVAL for invalid arguments values or NULL pointers

7.270. getType2ScreenerRateLimit

Function Declaration

```
uint32_t getType2ScreenerRateLimit(pD, regNum, rateLimit);

CEDI_PrivateData* pD;
uint8_t regNum;
CEDI_Type2ScreenerRateLimit* rateLimit;
```

Parameter List

<i>regNum</i>	[in]
Type 2 register number, range 0 to CEDI_DesignCfg.num_type2_screeners-1	
<i>rateLimit</i>	[out]
Configuration of rate limiting for given screening register.	
<i>pD</i>	[in]
driver private state info specific to this instance.	

Description

Gets maximum receive data rate for traffic matching type 2 screening register.

Return Value

EOK for success

ENOTSUP if feature is not available

EINVAL for invalid arguments values or NULL pointers

7.271. getRxType2RateLimitTriggered

Function Declaration

```
uint32_t getRxType2RateLimitTriggered(pD, status);

CEDI_PrivateData* pD;
uint16_t* status;
```

Parameter List

<i>status</i>	[out]
ORed combination of bit flags for each type 2 screening register (bit 0 - screener 0, bit 15 - screener 15). If bit is set to '1', rate limiting mechanism for given screener has been triggered.	
<i>pD</i>	[in]

driver private state info specific to this instance.

Description

Gets status, whether or not rate limiting has been triggered for given type 2 screening register.

Return Value

EOK for success

ENOTSUP if feature is not available

EINVAL for invalid arguments values or NULL pointers

7.272. setRxWatermark

Function Declaration

```
uint32_t setRxWatermark(pD, rxHighWatermark, rxLowWatermark);
```

```
CEDI_PrivateData* pD;  
uint16_t rxHighWatermark;  
uint16_t rxLowWatermark;
```

Parameter List

rxLowWatermark [in]

If not set to zero and receive SRAM fill level falls below this value, a zero-length pause frame is transmitted.

pD [in]

driver private state info specific to this instance.

rxHighWatermark [in]

If not set to zero and receive SRAM fill level exceeds this value, a pause frame is transmitted.

Description

Set receive watermark levels, related to sending PAUSE frames.

Return Value

EOK for success

ENOTSUP if feature is not available

EINVAL for invalid arguments values or NULL pointers

7.273. getRxWatermark

Function Declaration

```
uint32_t getRxWatermark(pD, rxHighWatermark, rxLowWatermark);
```

```
CEDI_PrivateData* pD;
uint16_t* rxHighWatermark;
uint16_t* rxLowWatermark;
```

Parameter List

rxLowWatermark	[out]	If not set to zero and receive SRAM fill level falls below this value, a zero-length pause frame is transmitted.
pD	[in]	driver private state info specific to this instance.
rxHighWatermark	[out]	If not set to zero and receive SRAM fill level exceeds this value, a pause frame is transmitted.

Description

Get receive watermark levels, related to sending PAUSE frames.

Return Value

EOK for success

ENOTSUP if feature is not available

EINVAL for invalid arguments values or NULL pointers

7.274. setAxiQosConfig

Function Declaration

```
uint32_t setAxiQosConfig(pD, queueNum, axiQosConfig);
```

```
CEDI_PrivateData* pD;
uint8_t queueNum;
CEDI_AxiQosConfig* axiQosConfig;
```

Parameter List

axiQosConfig	[in]	configuration of AXI QoS to set
--------------	--------	---------------------------------

<code>pD</code>	[in]
	driver private state info specific to this instance.
<code>queueNum</code>	[in]
	number of Tx queue

Description

Set AXI quality of service configuration for a specific queue.

Return Value

EOK for success

ENOTSUP if feature is not available

EINVAL for invalid arguments values or NULL pointers

7.275. getAxiQosConfig

Function Declaration

```
uint32_t getAxiQosConfig(pD, queueNum, axiQosConfig);
```

```
CEDI_PrivateData* pD;  
uint8_t queueNum;  
CEDI_AxiQosConfig* axiQosConfig;
```

Parameter List

<code>axiQosConfig</code>	[out]
	read configuration of AXI QoS
<code>pD</code>	[in]
	driver private state info specific to this instance.
<code>queueNum</code>	[in]
	number of Tx queue

Description

Get AXI quality of service configuration for a specific queue.

Return Value

EOK for success

ENOTSUP if feature is not available

EINVAL for invalid arguments values or NULL pointers

7.276. getAsfInfo

Function Declaration

```
uint32_t getAsfInfo(regBase, asfInfo);  
  
uintptr_t regBase;  
CEDI_AsfInfo* asfInfo;
```

Parameter List

asfInfo [out]

returns information about ASF module. It informs if ASF is included to GEM, how many instances of ASF are included and what are the register base offsets if any. ASF module has own Core Driver which does not connected with GEM Core Driver.

regBase [in]

base address of MAC registers

Description

Get the information about ASF module.

Return Value

EOK on success

EINVAL if config is NULL, or hardware not present